Generation Models and 3D Reconstruction and Frontiers

Hao WANG

AI Thrust, Information Hub

Recap of last week

Computer Vision Tasks

Classification



CAT

No spatial extent

Semantic Segmentation



GRASS, CAT, TREE, SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

Instance Segmentation



DOG, DOG, CAT

Multiple Object

This image is CC0 public domain

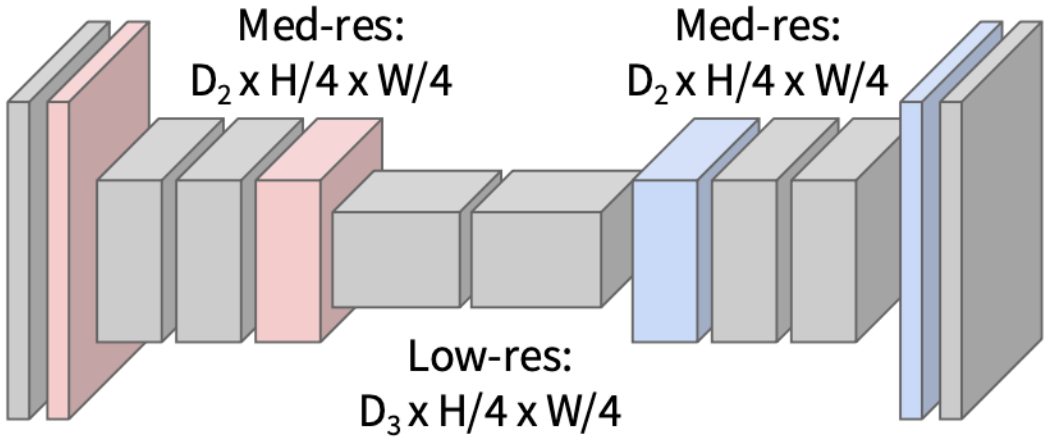
Semantic Segmentation Idea: Fully Convolutional

Downsampling:
Pooling, strided convolution

Design network as a bunch of convolutional layers, with downsampling and upsampling inside the network!

Upsampling:
???







Input:
3 x H x W

High-res:
D_1 x H/2 x W/2

High-res:
D_1 x H/2 x W/2

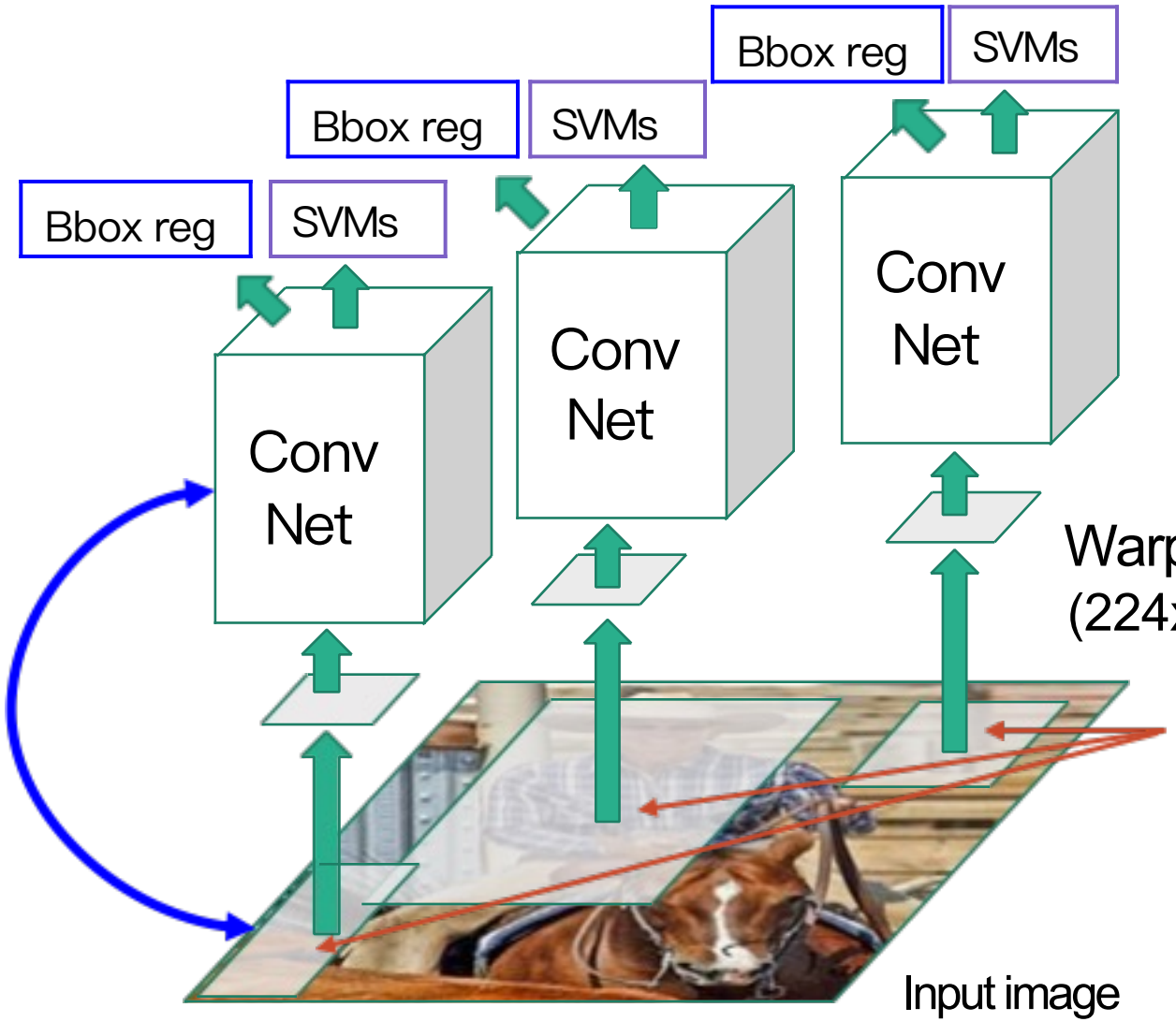
C x H x W

Predictions:
H x W

Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

"Slow" R-CNN

Predict "corrections" to the RoI: 4 numbers: (dx, dy, dw, dh)



Classify regions with SVMs

Forward each region through ConvNet (ImageNet-pretranied)

Warped image regions (224x224 pixels)

Regions of Interest (RoI) from a proposal method (~2k)

Input image

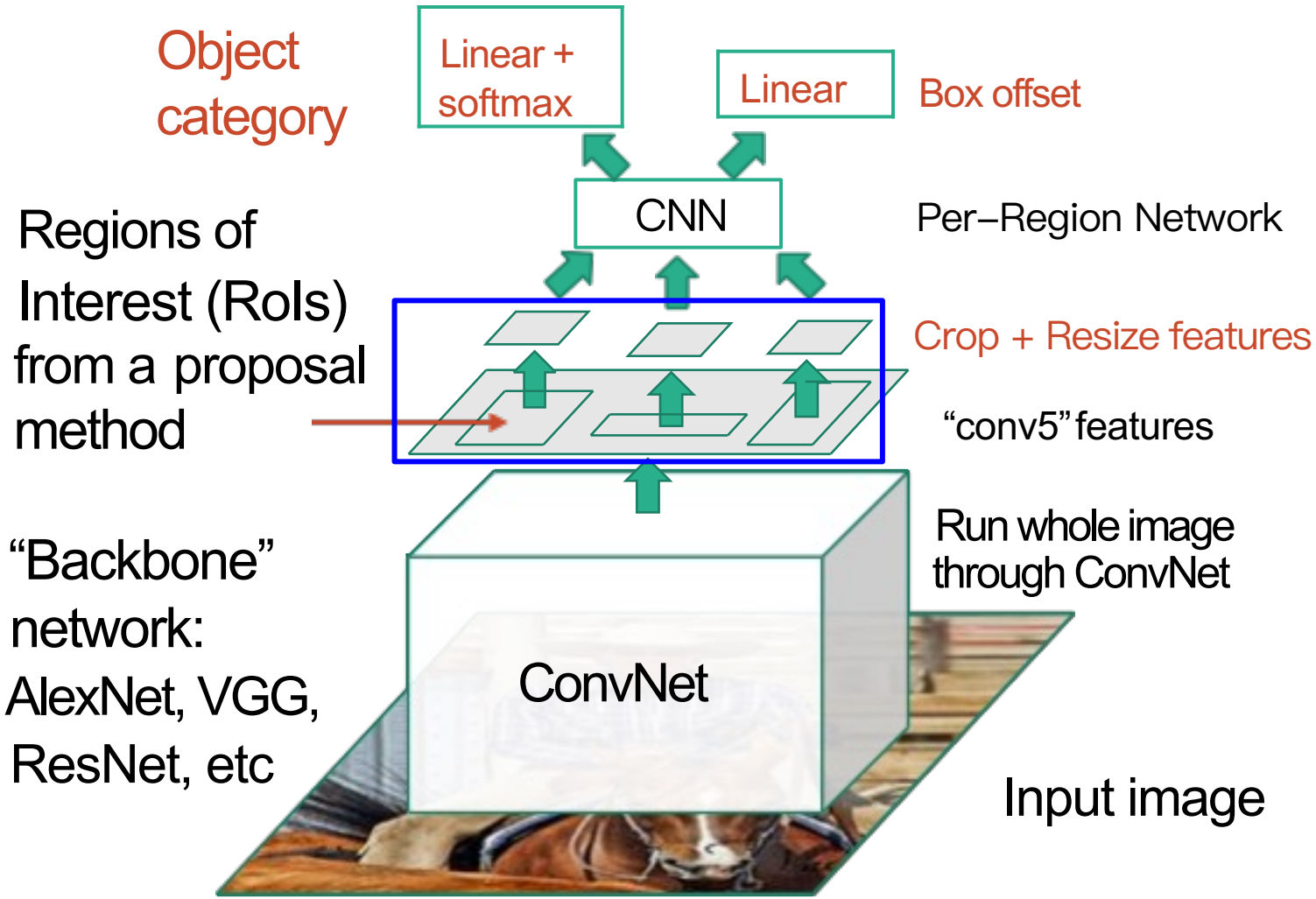
Problem: Very slow! Need to do ~2k independent forward passes for each image!

Idea: Pass the image through convnet before cropping! Crop the conv feature instead!

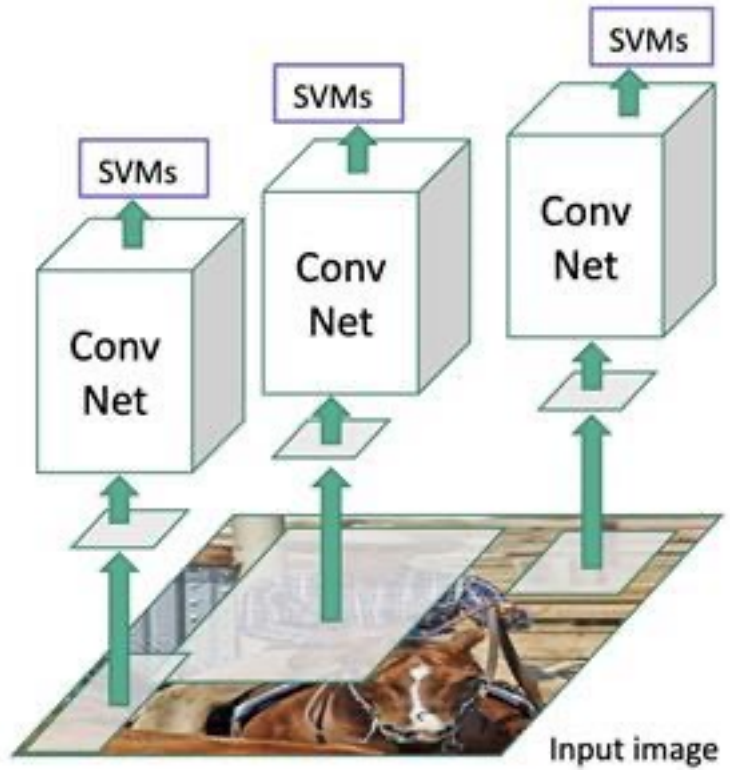
Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.

Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

Fast R-CNN

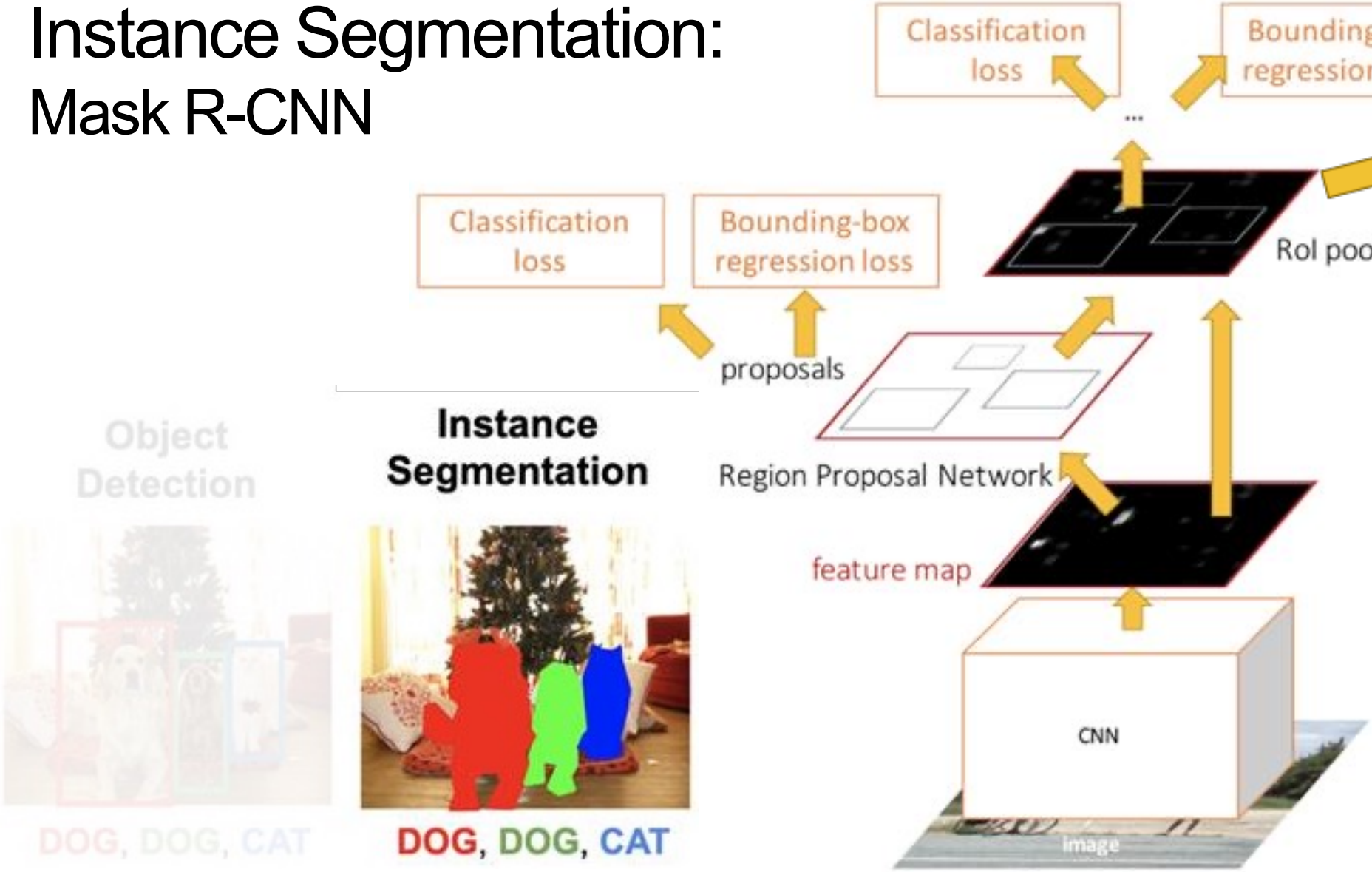


"Slow" R-CNN



Girshick, "Fast R-CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

Instance Segmentation:
Mask R-CNN



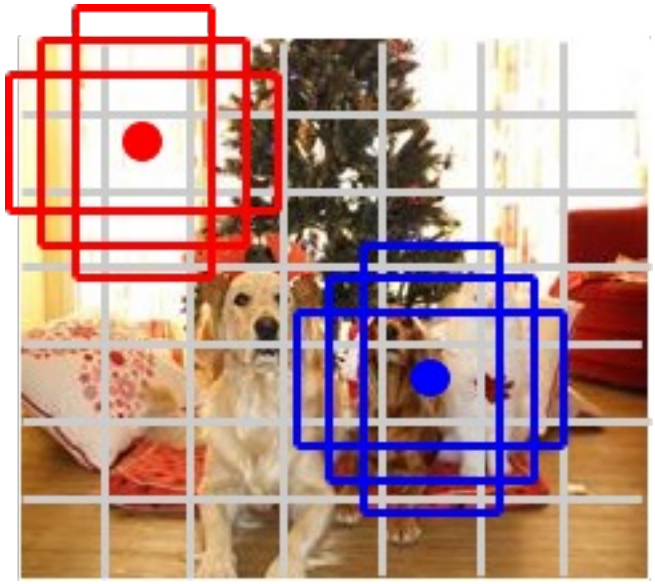
Add a small mask network that operates on each RoI and predicts a 28x28 binary mask

He et al, "Mask R-CNN", ICCV 2017

Single-Stage Object Detectors: YOLO / SSD / RetinaNet



Input image
3 x H x W



Divide image into grid
7 x 7

Image a set of base boxes centered at each grid cell
Here B = 3

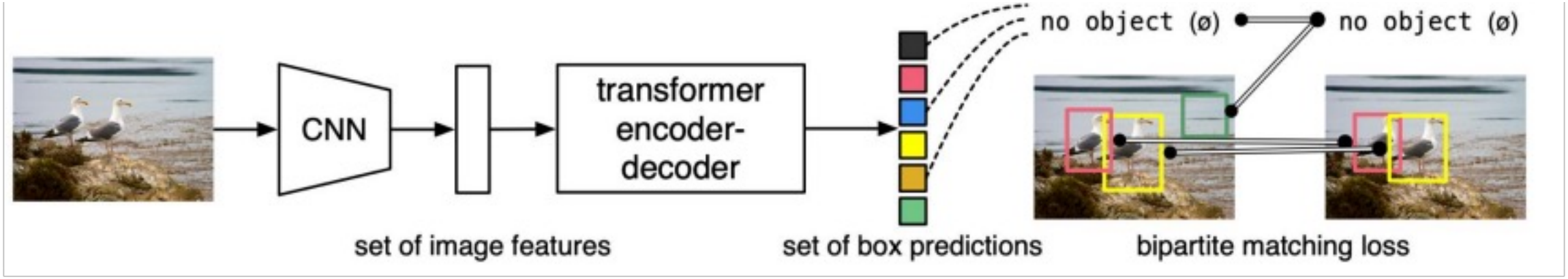
Within each grid cell:

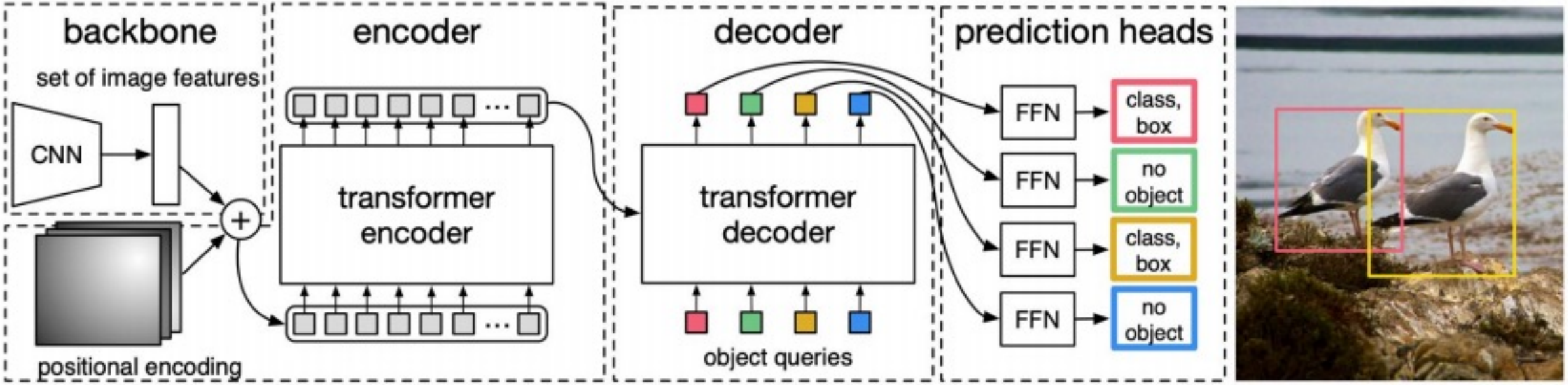
- Regress from each of the B base boxes to a final box with 5 numbers: (dx, dy, dh, dw, confidence)
- Predict scores for each of C classes (including background as a class)
- Looks a lot like RPN, but category-specific!

Output:
7 x 7 x (5 * B + C)

Redmon et al, "You Only Look Once: Unified, Real-Time Object Detection", CVPR 2016
Liu et al, "SSD: Single-Shot MultiBox Detector", ECCV 2016
Lin et al, "Focal Loss for Dense Object Detection", ICCV 2017

Object Detection with Transformers: DETR



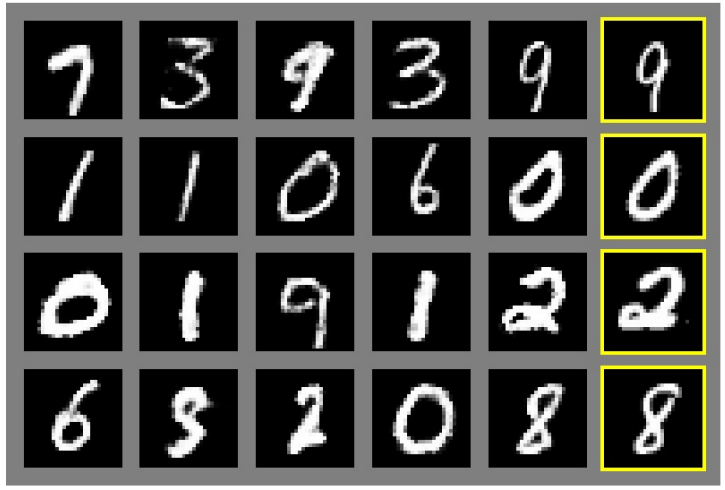


Carion et al, "End-to-End Object Detection with Transformers", ECCV 2020

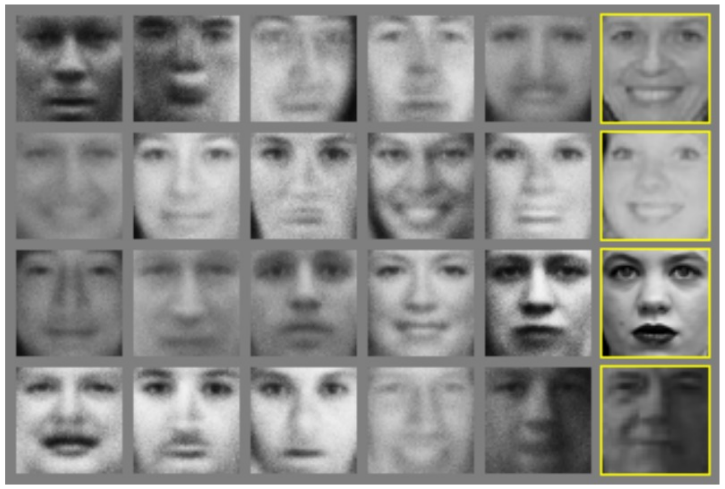
Do you know what is the best visual generation model at this moment?



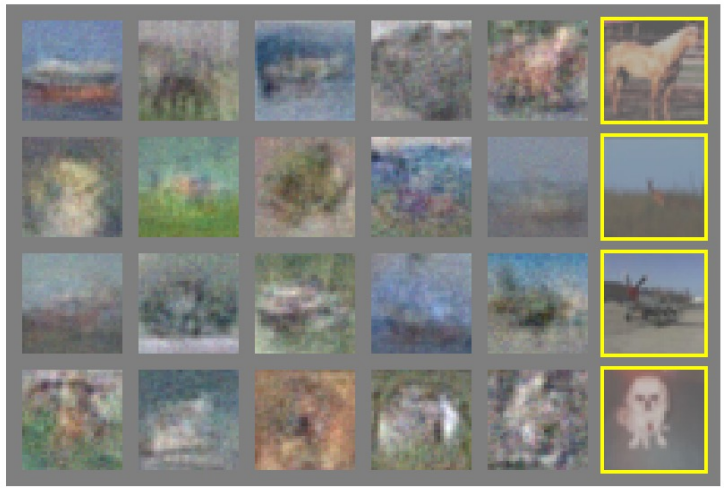
What it was in 12 years ago…



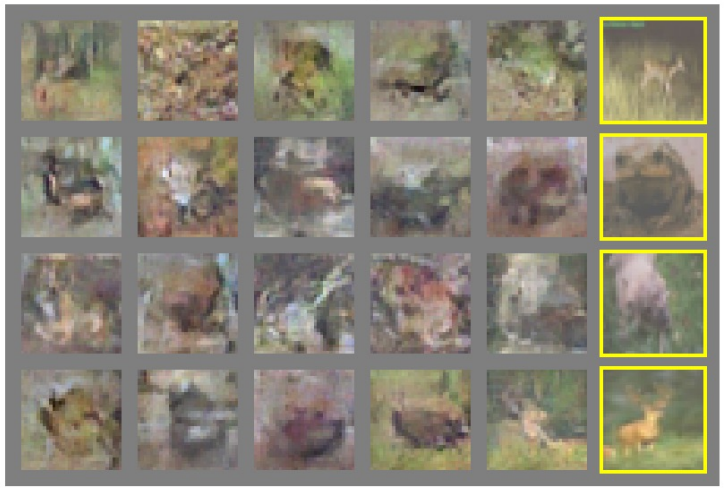
a)



b)

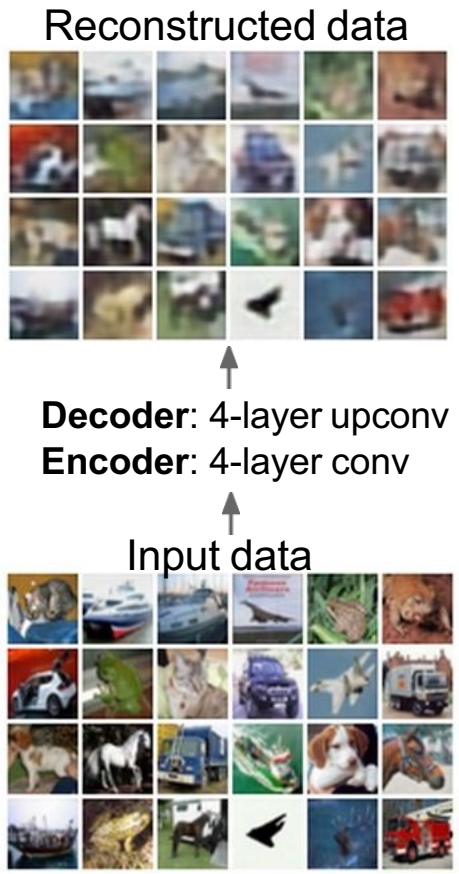


c)

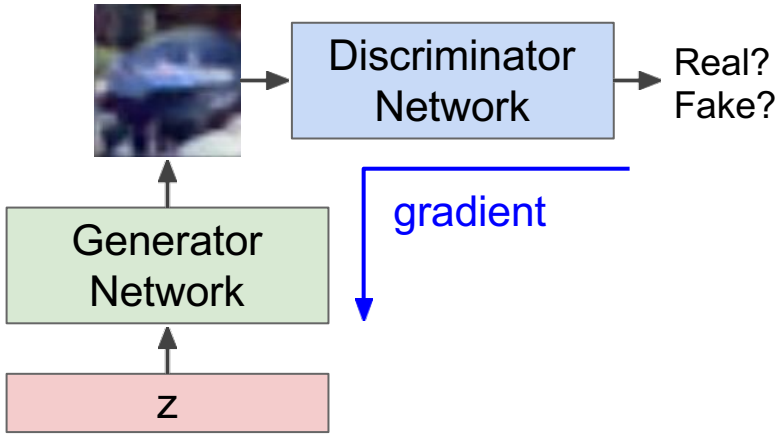


d)

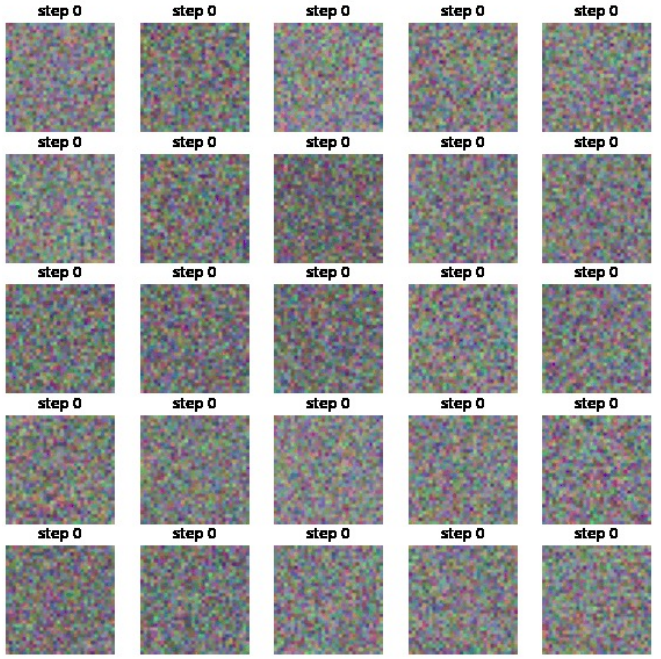
How we approach here?



Variational Autoencoder



Generative Adversarial Networks



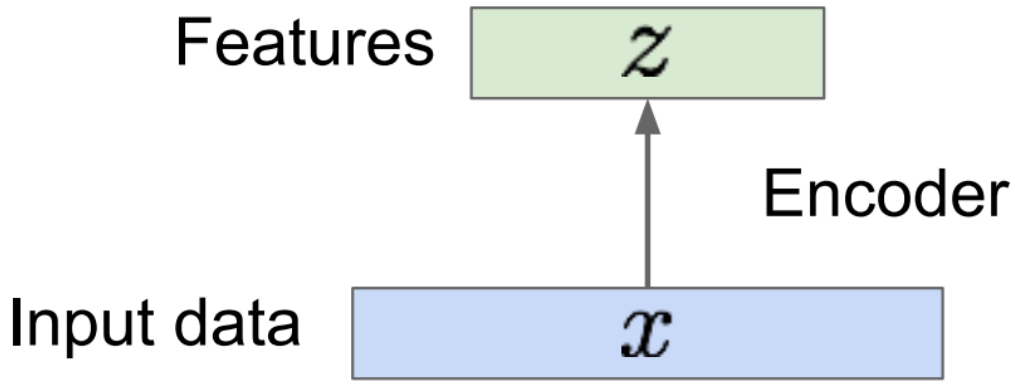
Diffusion Model

Variational Autoencoder

Some background first: Autoencoders

Unsupervised approach for learning a feature representation from unlabeled training data

With the encoder (CNN/dense layer), we extract the features of meaningful factors of variation in data



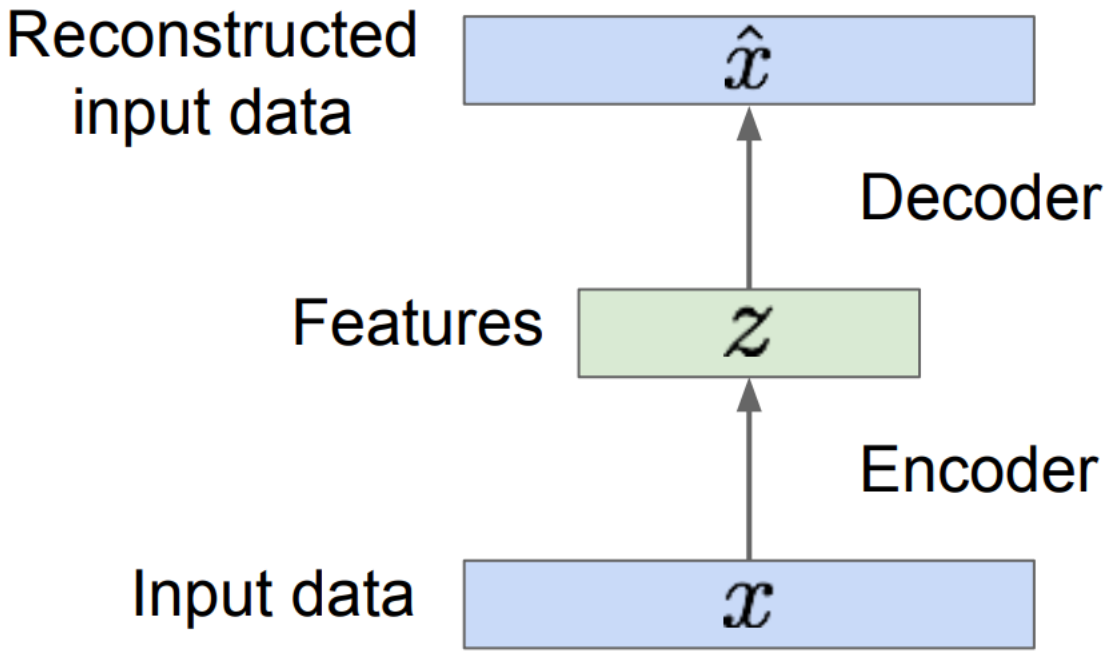


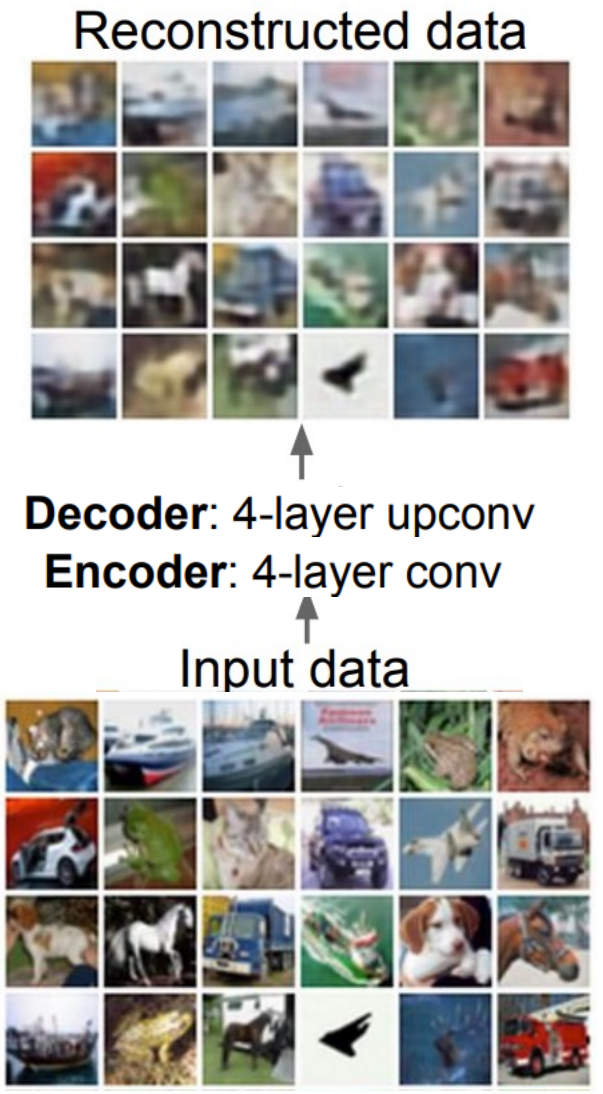
Some background first: Autoencoders

How to learn this feature representation?

Training objective:

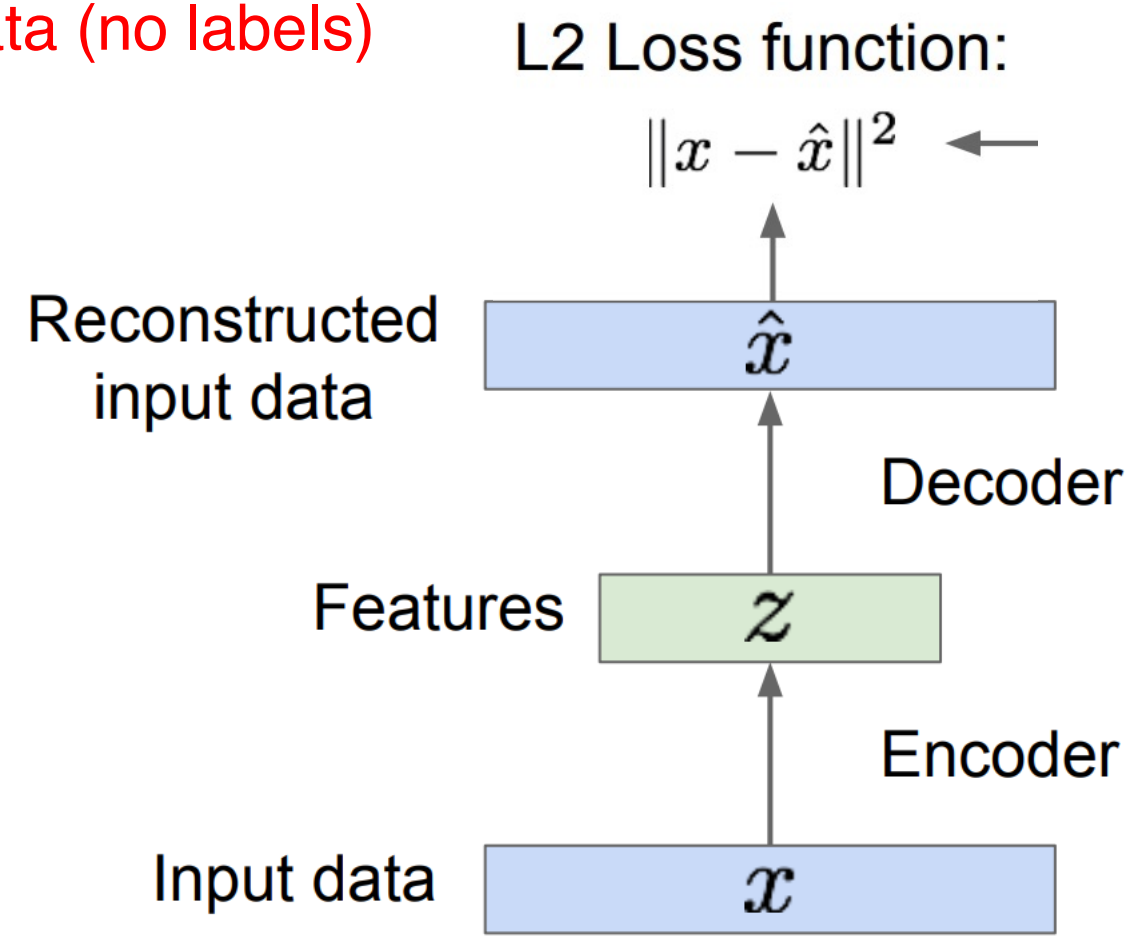
- Features can be used to reconstruct original data “Autoencoding” - encoding itself

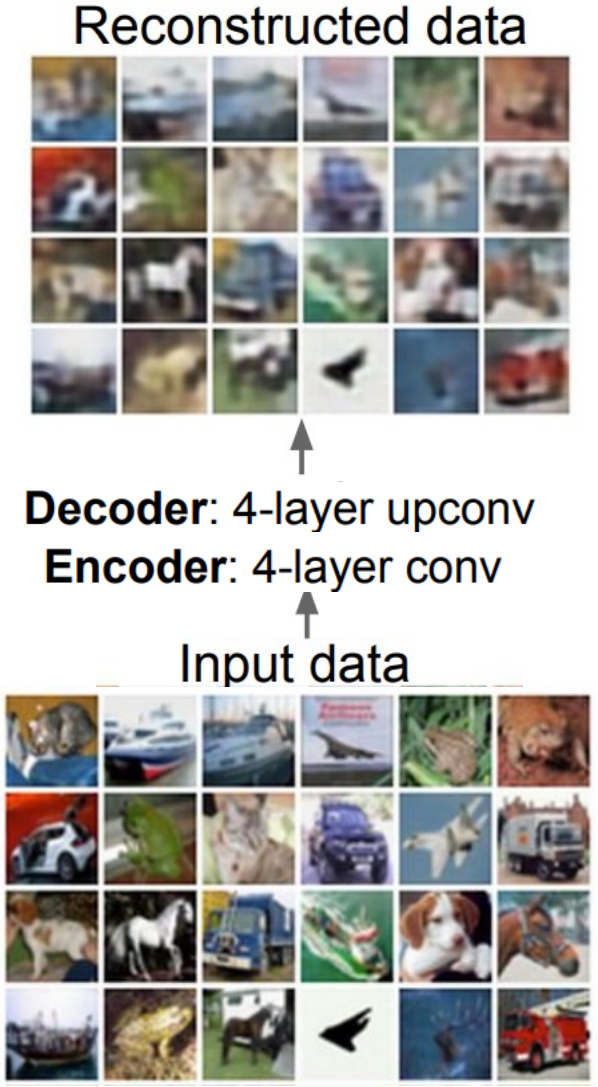




Some background first: Autoencoders

Learned features can be used to reconstruct original data (no labels)

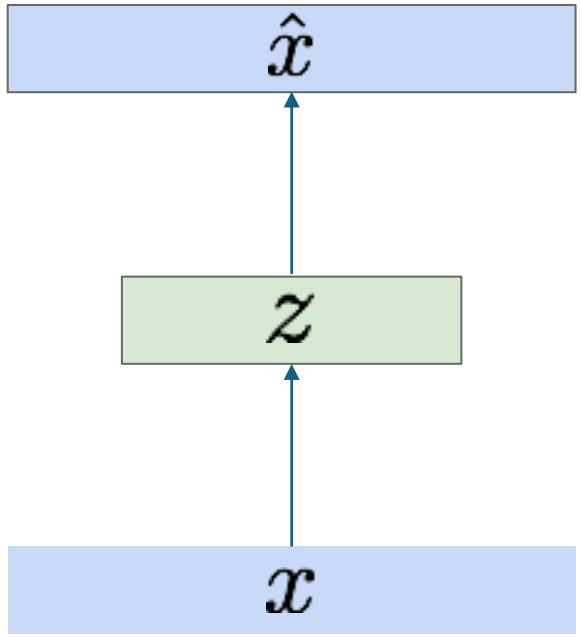




Some background first: Autoencoders

L2 Loss function:

$$\|x - \hat{x}\|^2$$



Feature

Input data

Applications:

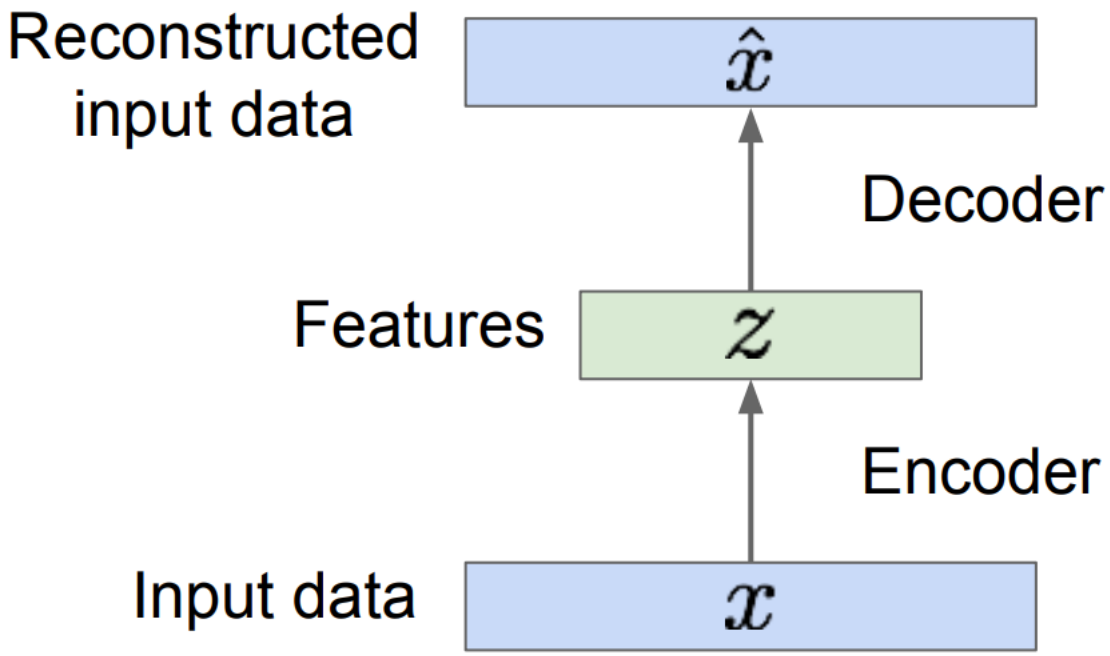
Dimensional reduction

Feature learning

Auxiliary learning

...

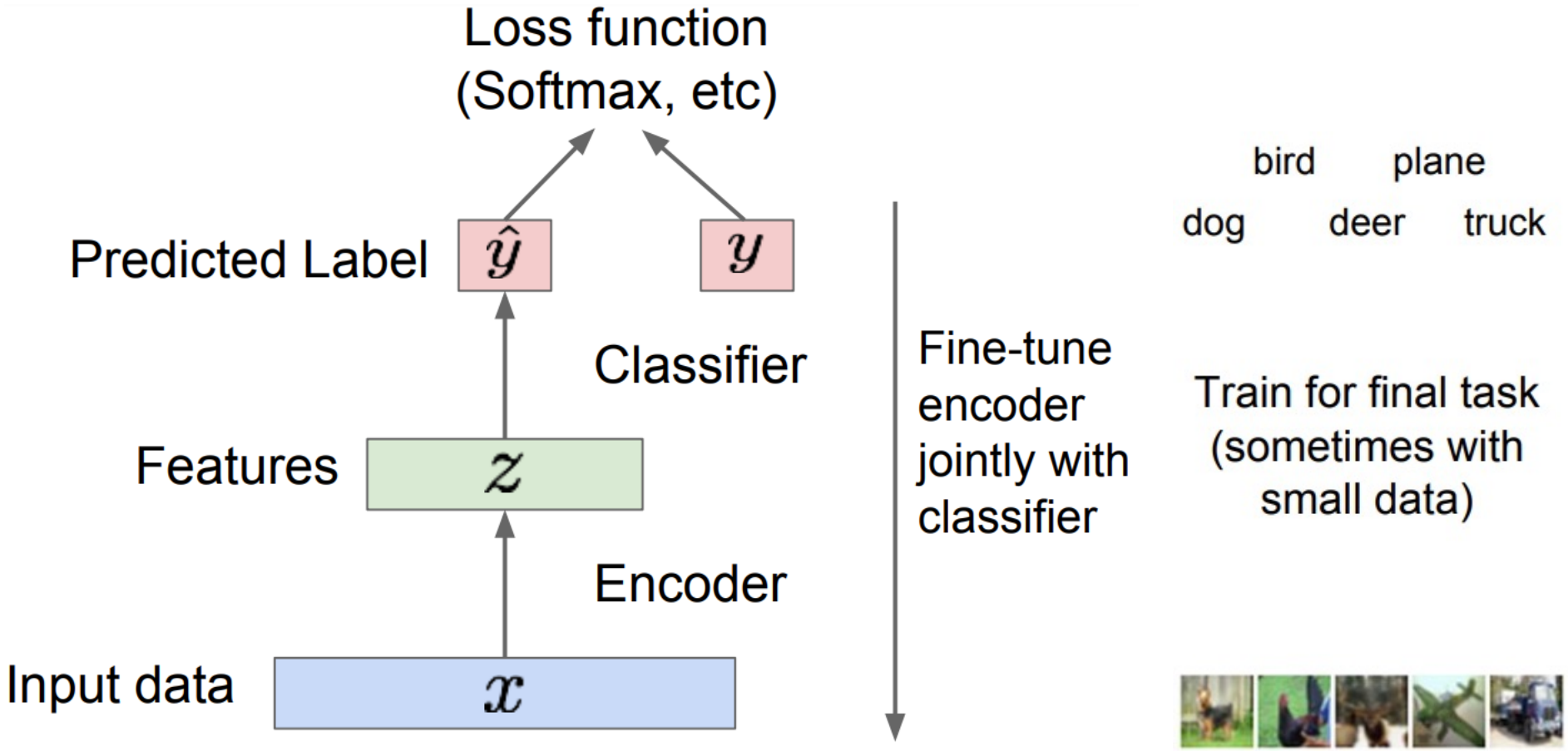
Some background first: Autoencoders



After training, throw away decoder

Some background first: Autoencoders

Encoder can be used to initialize a **supervised** model



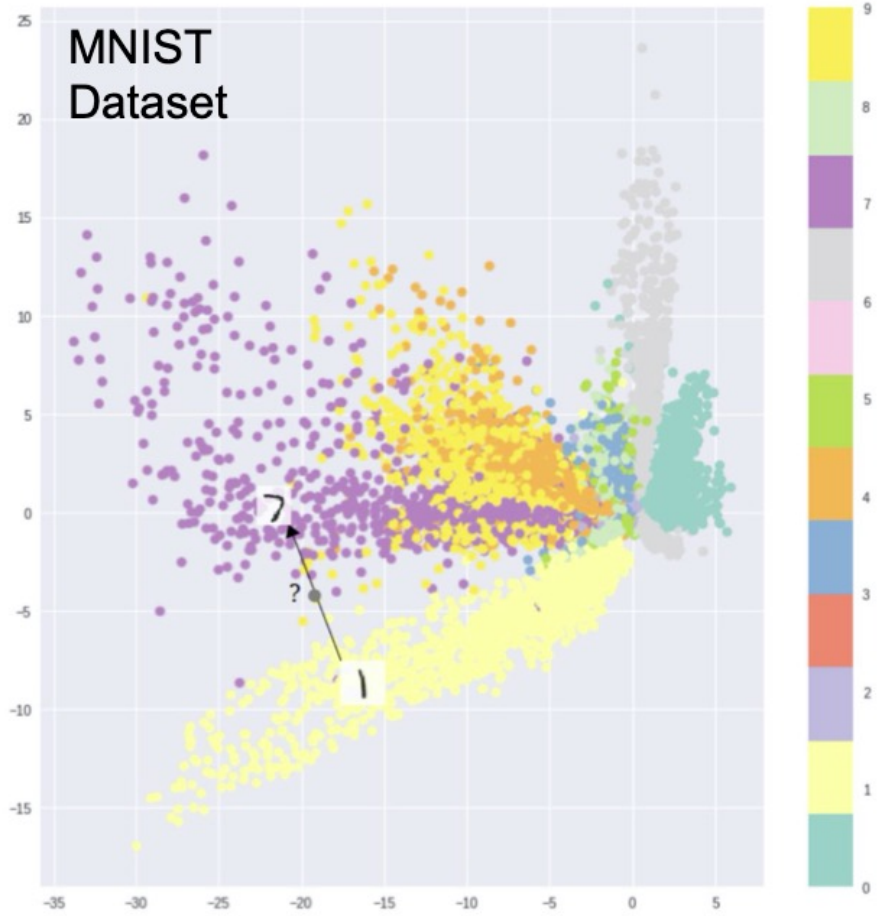
The problems of traditional autoencoder

Background :

- This is fine if you're just replicating the same images.
- If we don't want to replicate the same image you put in. You want to generate variations on an input image from a continuous latent space.

Problems :

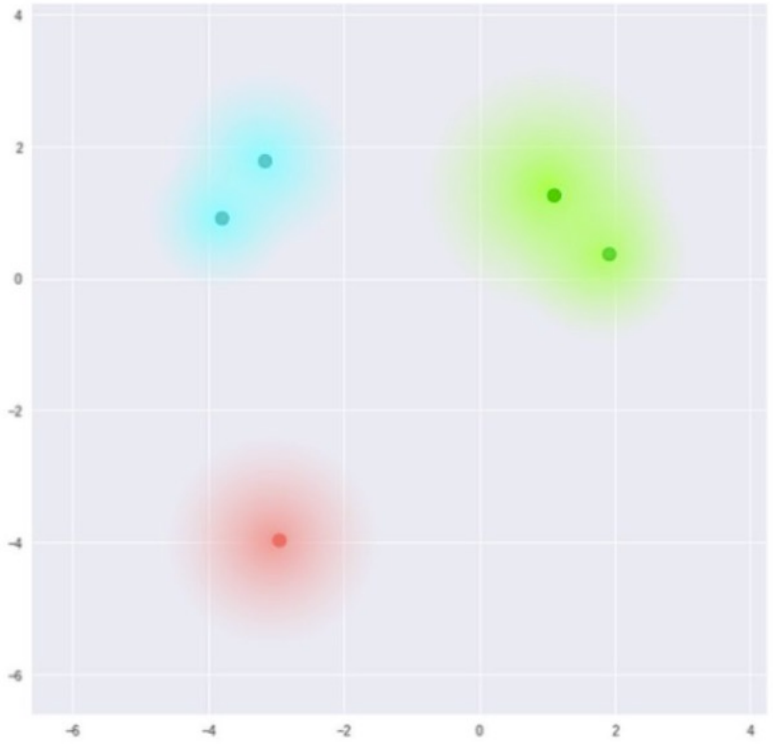
- The encoded vectors may not be continuous or allow easy interpolation.
- Because of discontinuities (e.g. gaps between clusters) the decoder will easily generate an unrealistic output.



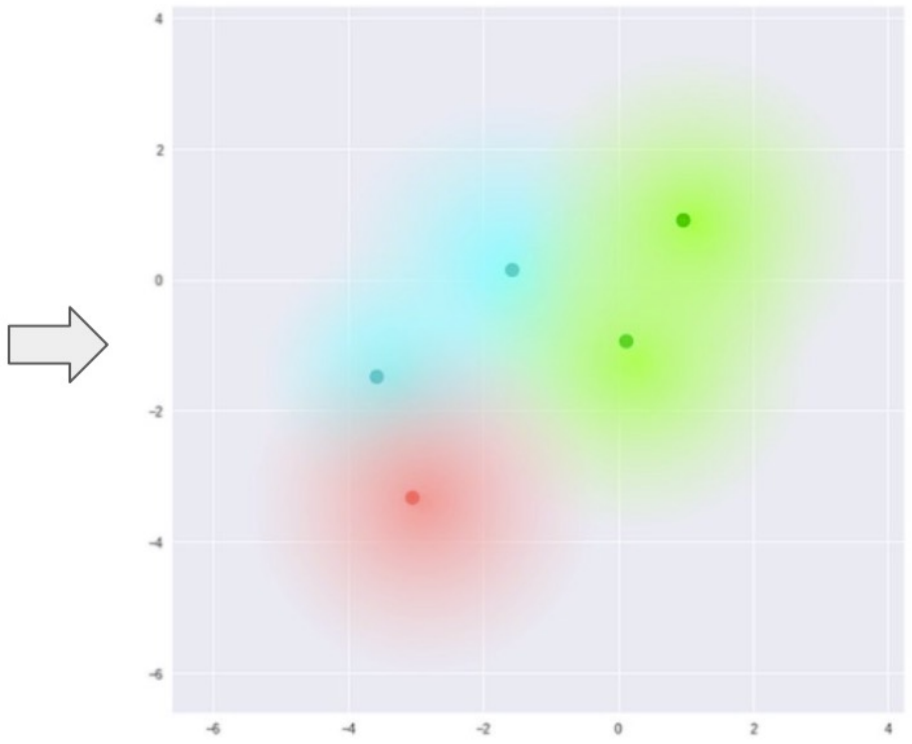
Visualizing the encodings from a 2D space

Intuitively Understanding Variational Autoencoders

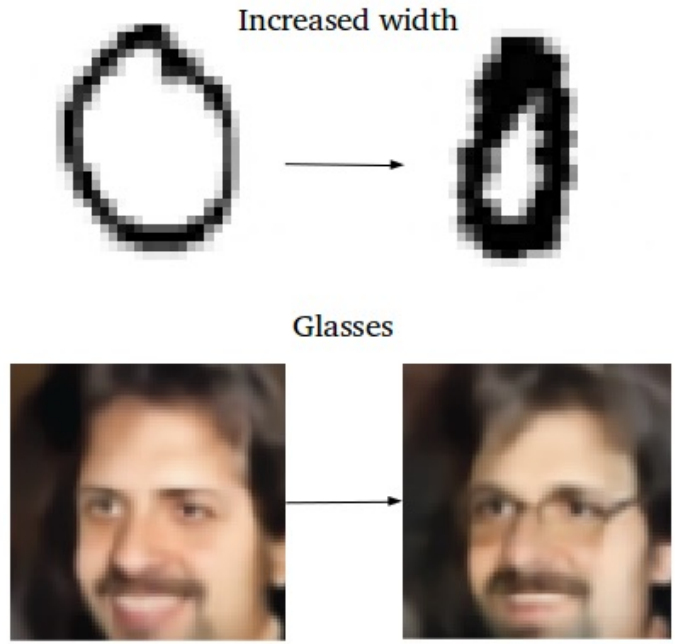
The problems of traditional autoencoder



Autoencoders



What we require

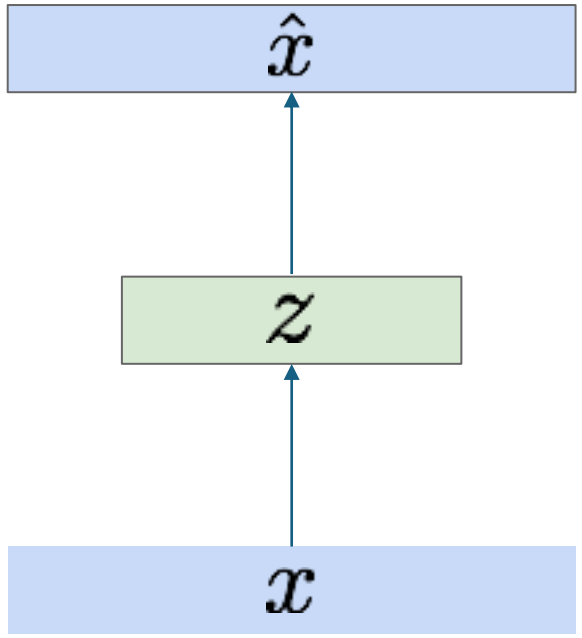


[Intuitively Understanding Variational Autoencoders]

Some background first: Autoencoders

L2 Loss function:

$$\|x - \hat{x}\|^2$$



Applications:

Dimensional reduction

Feature learning

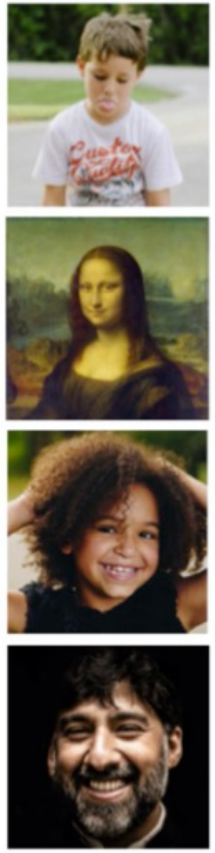
Auxiliary learning

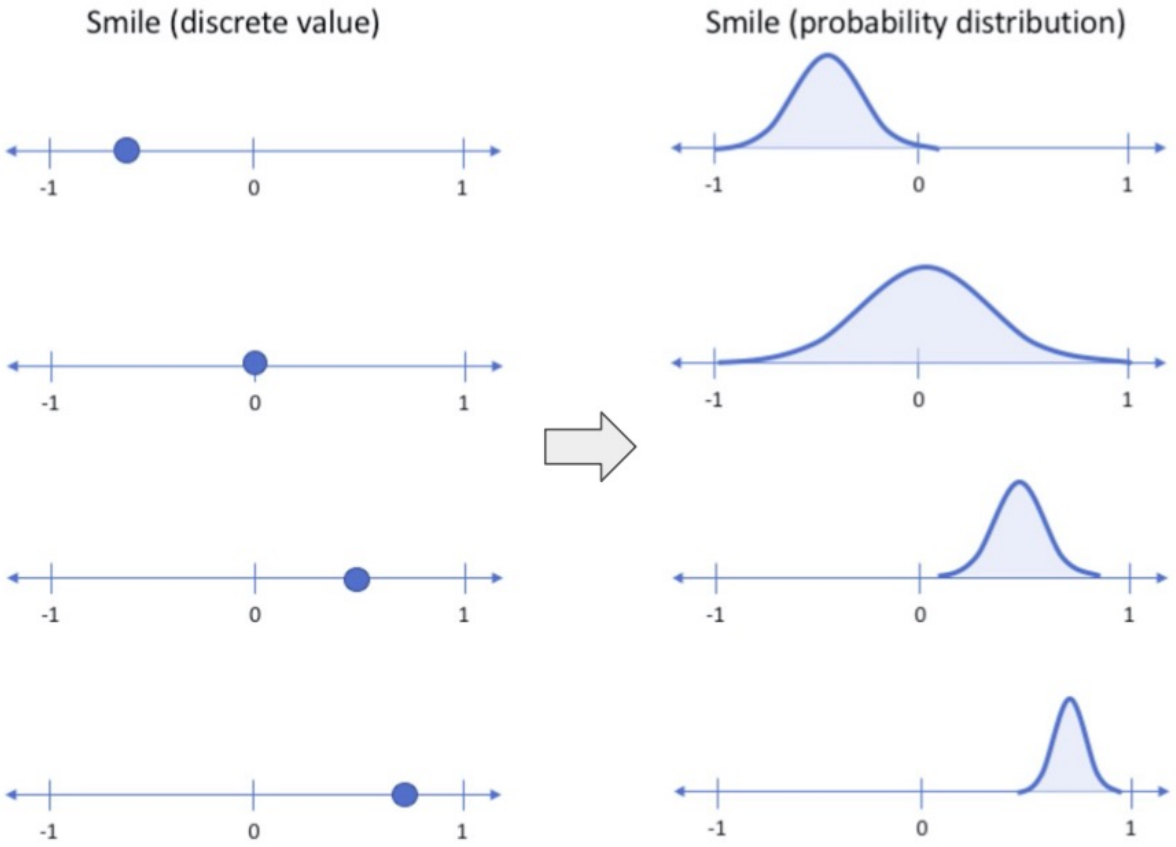
...

But we can't generate new images from an autoencoder.

How do we make autoencoder a **generative model**?

VAE: key difference with traditional autoencoder





Suppose:

Given a sample X_k, we have an exclusive distribution $P(Z|X_k)$ for X_k

How to construct a Gaussian Distribution?

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

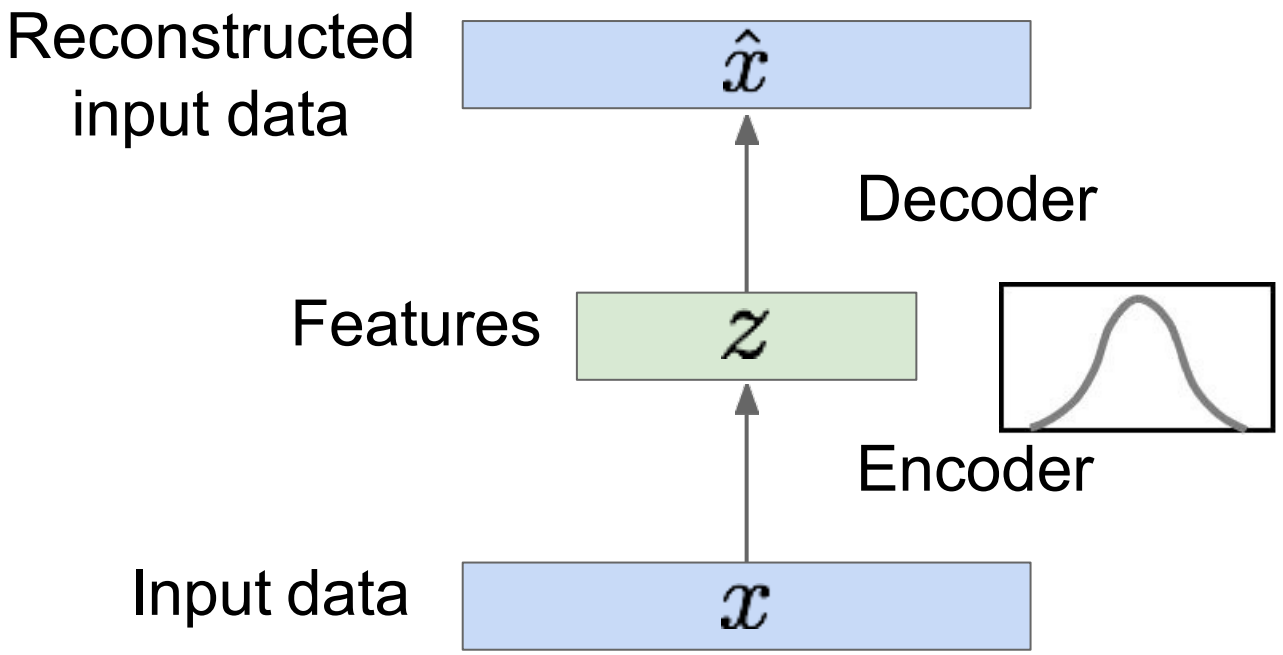
- μ = mean (center of the distribution)
- σ = standard deviation (spread of the distribution)

Variational autoencoders are a probabilistic twist on autoencoders
Sample from the mean and variance to compute latent sample

JJ's Variational autoencoders blog

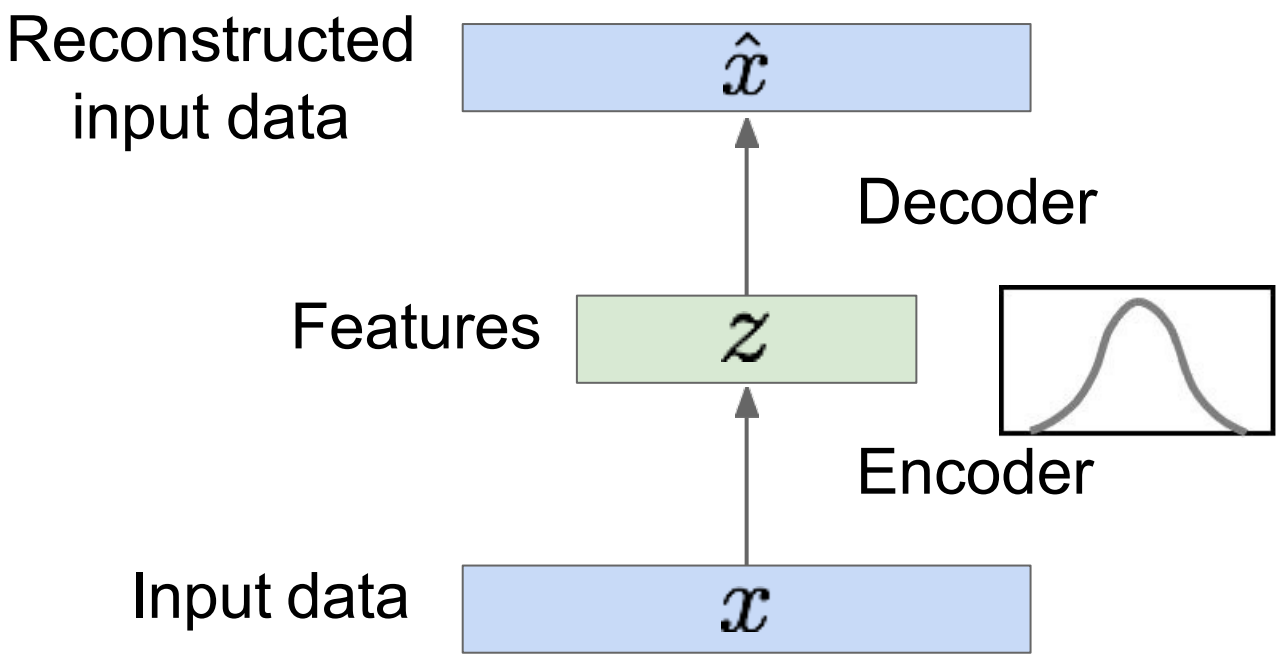
Variational Autoencoder

Suppose the latent features follow Gaussian distribution.



Variational Autoencoder

Suppose the latent features follow Gaussian distribution



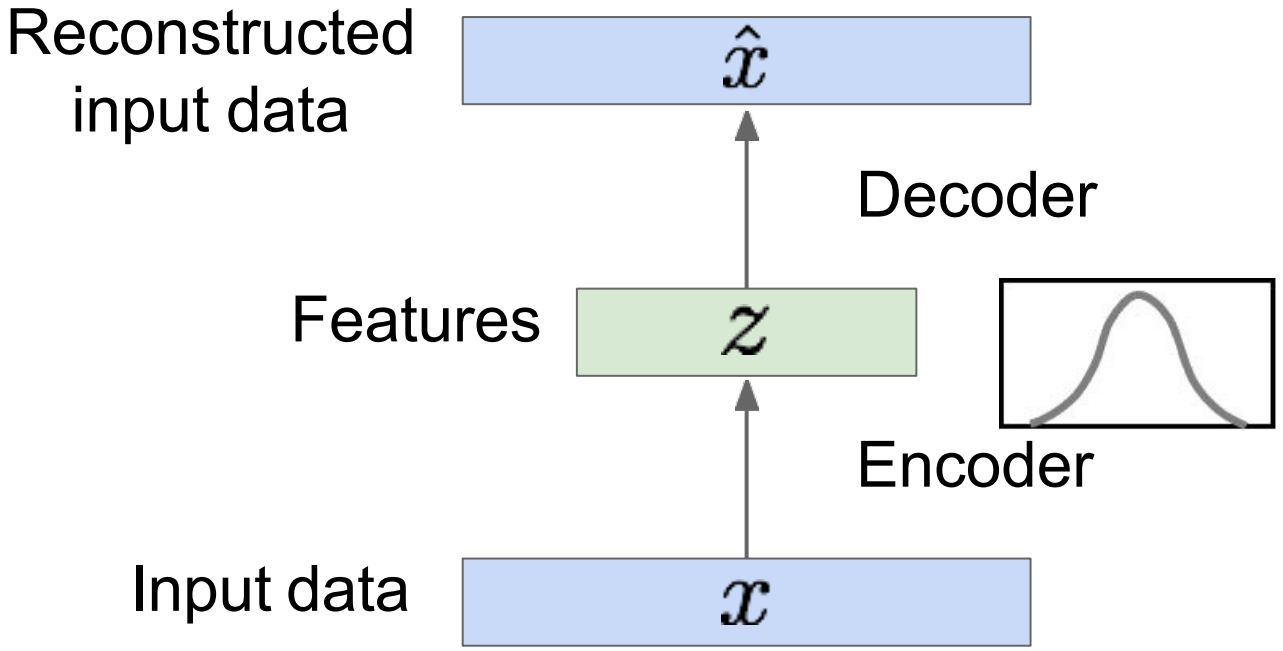
Then generation is done by

1. Sample z from the Gaussian distribution.

Variational Autoencoder

Suppose the latent features follow Gaussian distribution





Then generation is done by

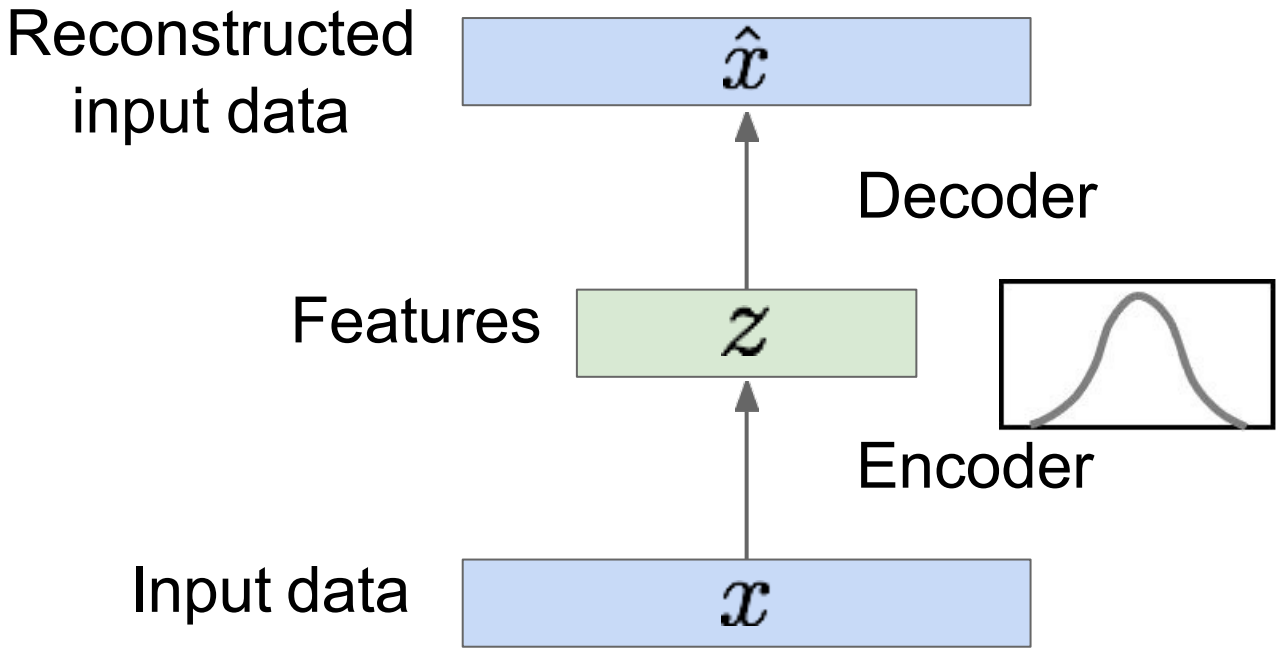
1. Sample z from the Gaussian distribution.

2. Forward z to the decoder to get the generated results.

Variational Autoencoder

Suppose the latent features follow Gaussian distribution





Then generation is done by

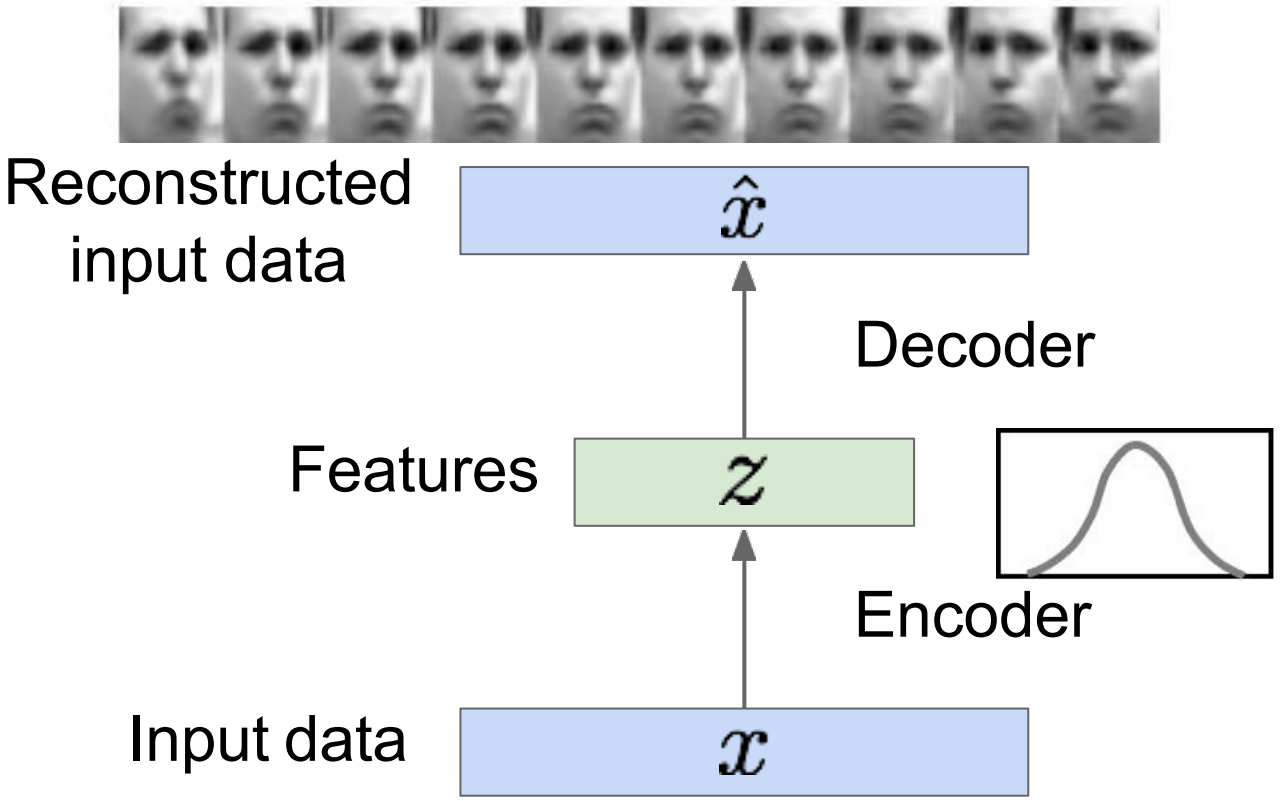
1. Sample z from the Gaussian distribution.

2. Forward z to the decoder to get the generated results.

The key issue: How can we impose the features z to follow Gaussian distribution?

Variational Autoencoder

The key issue: How can we impose the features z to follow Gaussian distribution?



Regularization:

$$L = \left|\left|x - \hat{x}\right|\right|^2 + D_{KL}\left(q(z|x)||p(z)\right)$$

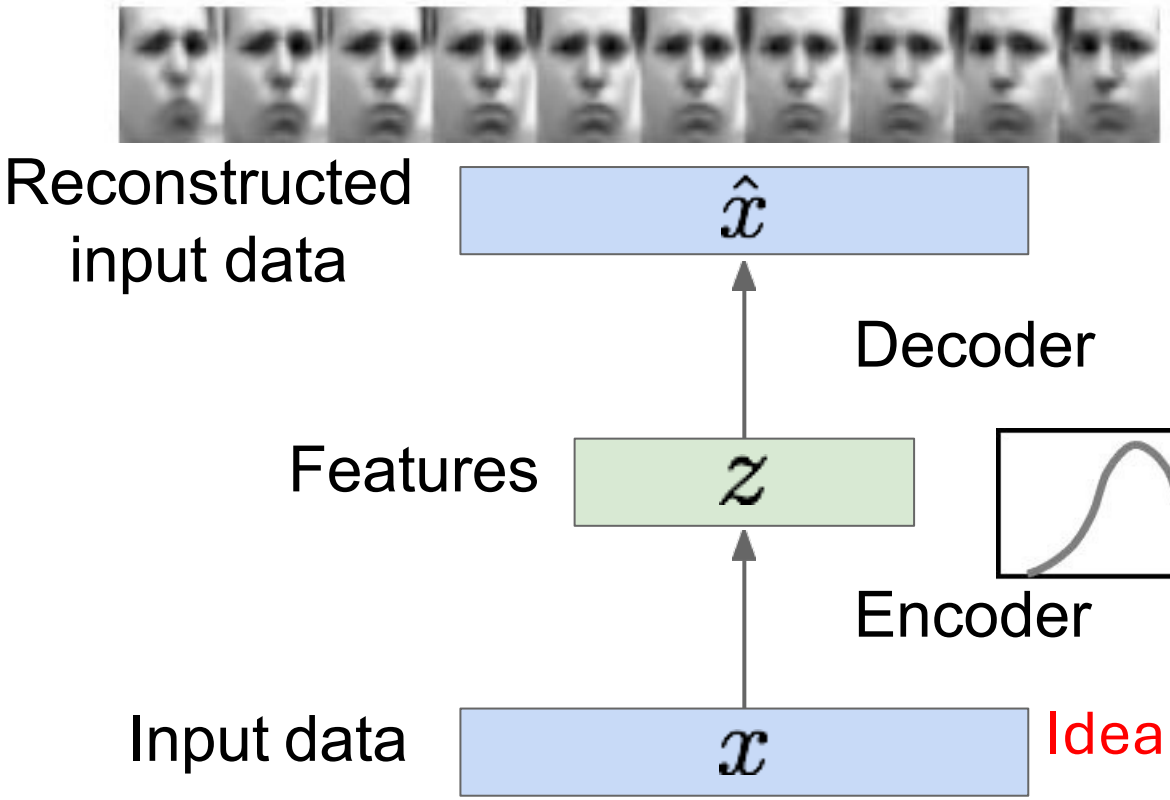
Here D_{KL} means the KL divergence.
$q(z|x)$: the latent distribution.
$p(z)$: standard Gaussian distribution.

How to compute $D_{KL}\left(q(z|x)||p(z)\right)$?

Difficulty: z is determinristic. But $p(z)$ is a distribution.
How to measure their loss?

Variational Autoencoder

The key issue: How can we impose the features z to follow Gaussian distribution?



Regularization:

$$L = \left|\left|x - \hat{x}\right|\right|^2 + D_{KL}\left(q(z|x)||p(z)\right)$$

Here D_{KL} means the KL divergence.
$q(z|x)$: the latent distribution.
$p(z)$: standard Gaussian distribution.

How to compute $D_{KL}\left(q(z|x)||p(z)\right)$?

Idea: Reparameterization trick.

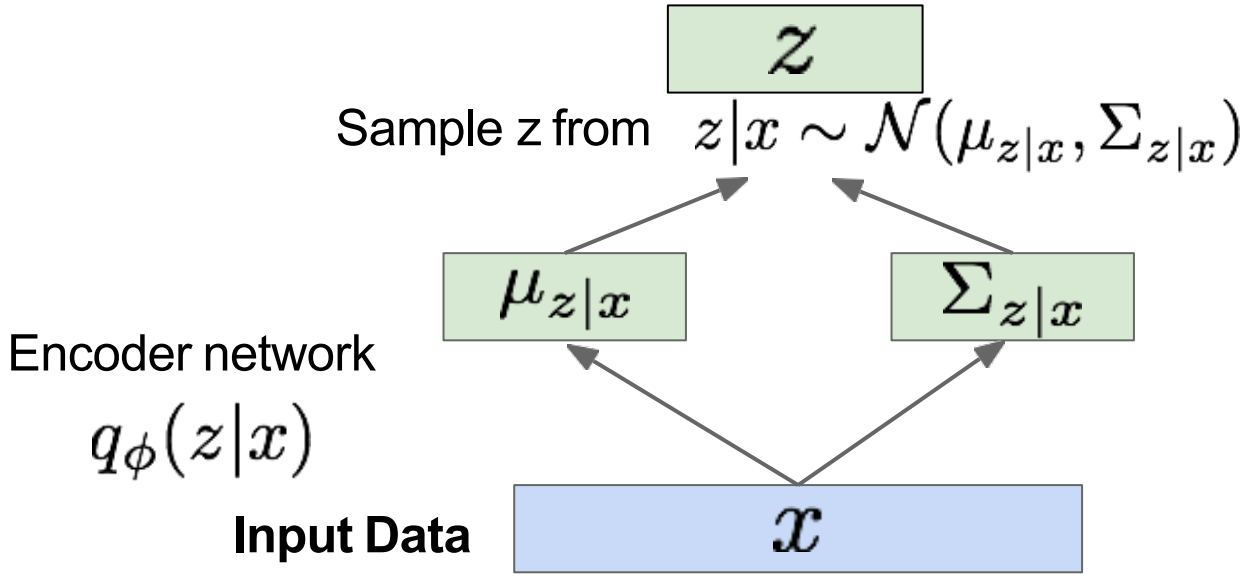
$$z = \mu + \sigma \cdot \varepsilon$$

where ε is sampled from standar Gaussian distribution.

Variational Autoencoders

Now the regularization term can be computed by

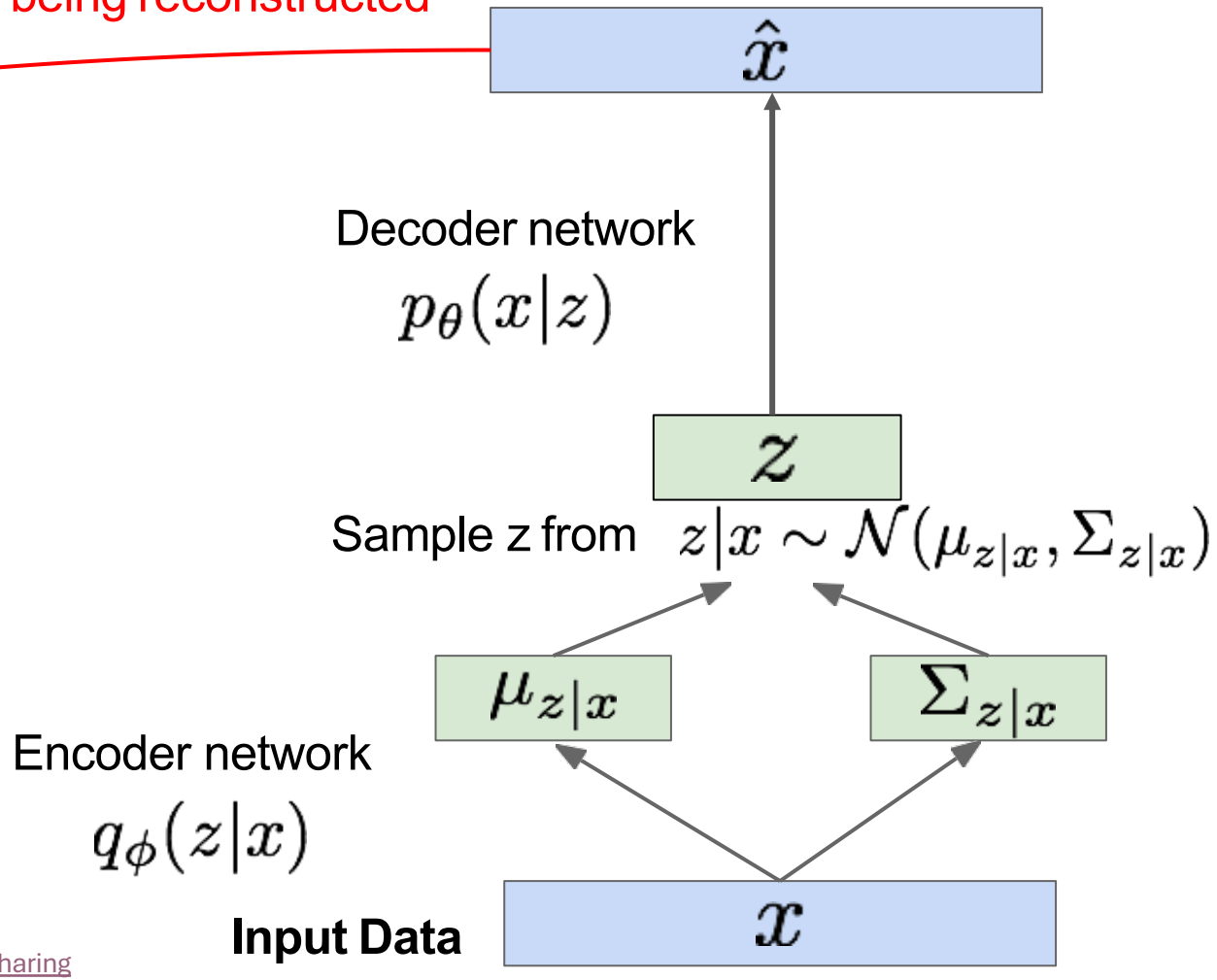
$$D_{KL}\big(q(z|x)||p(z)\big) = \frac{1}{2}\sum_{i=1}^{n}\big(\Sigma_i^2 + \mu_i^2 - 1 - \log\Sigma_i^2\big)$$



Variational Autoencoders

Maximize likelihood of original input being reconstructed

$$L = \left|\left|x - \hat{x}\right|\right|^2 + D_{KL}\left(q(z|x)||p(z)\right)$$



https://colab.research.google.com/drive/1C_264kIFH7p8g8p7eNDtf5sgt25o4dXb?usp=sharing

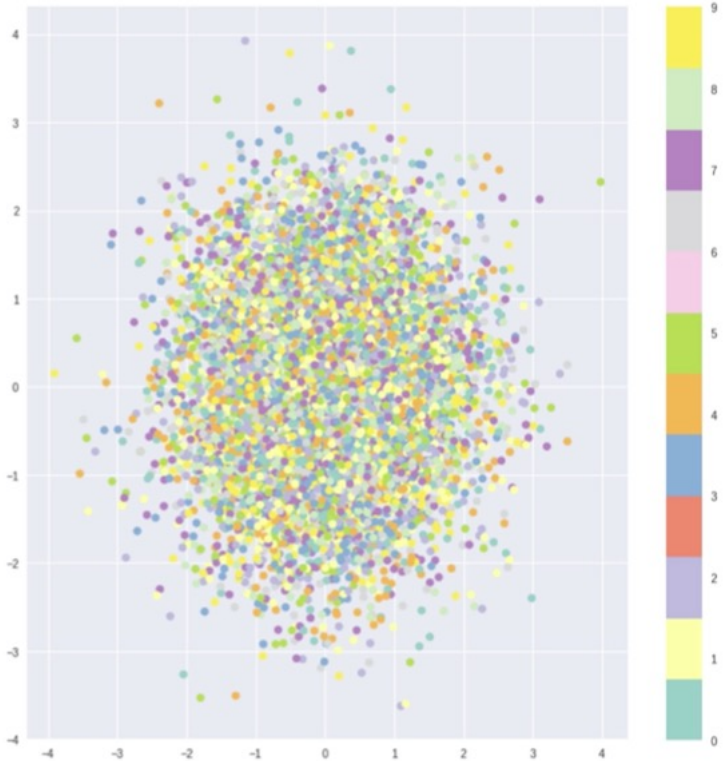
Priors on the latent distribution

Inferred latent distribution

Fixed prior on laten distribution

$$D(q_\phi(z|x) \,||\, p(z)) = -\frac{1}{2}\sum_{j=0}^{k-1}(\sigma_j + \mu_j^2 - 1 - \log \sigma_j)$$

KL-divergence between the two distributions



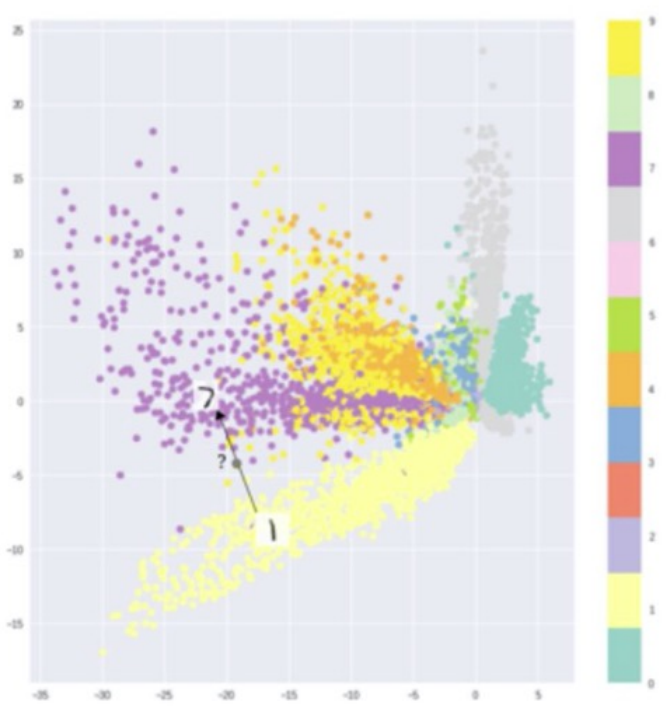
Common choice of prior:

$p(z) = \mathcal{N}(\mu = 0, \sigma^2 = 1)$

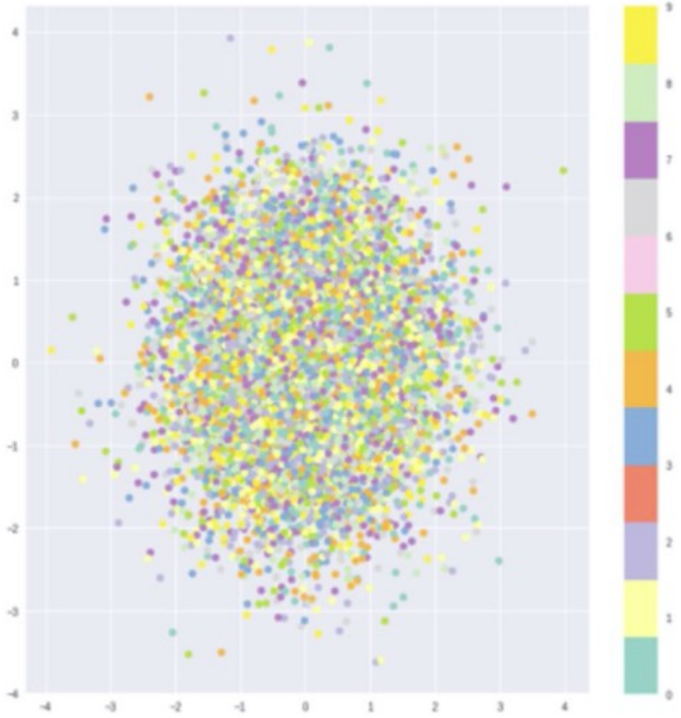
- Encourages encodings to distribute encodings evenly around the center of the latent space
- Penalize the network when it tries to “cheat” by clustering points in specific regions (ie. memorizing the data)

VAE Optimization

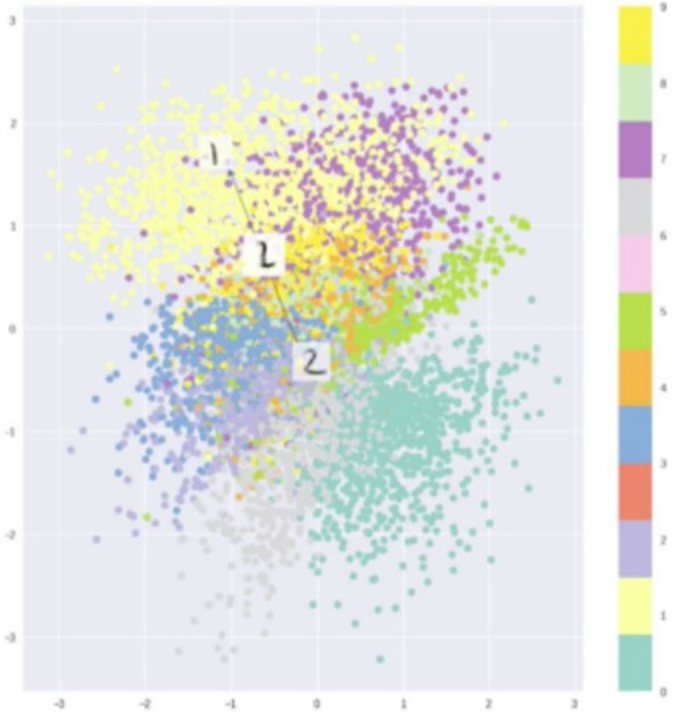
Only reconstruction loss



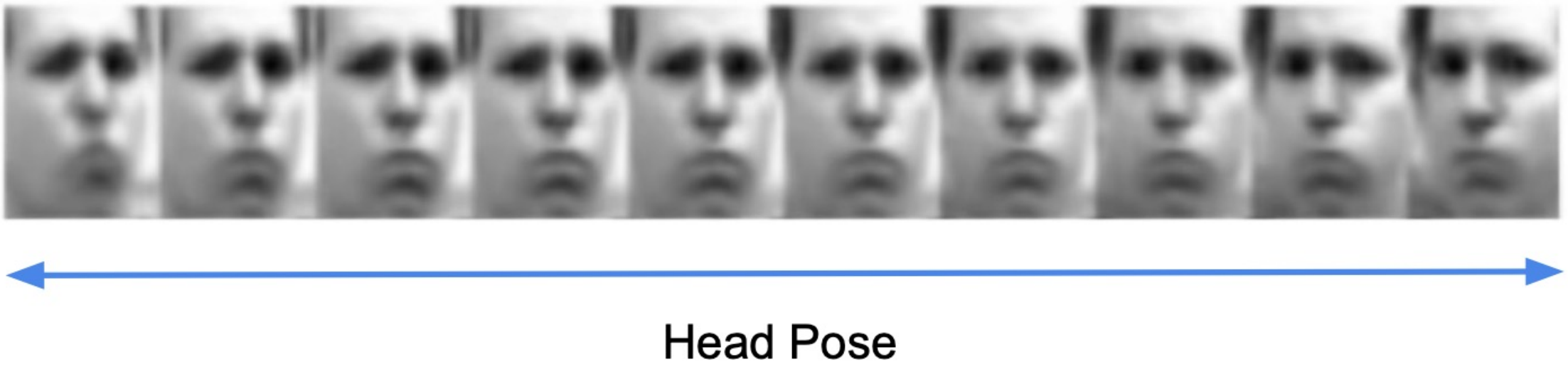
Only KL divergence



Combination

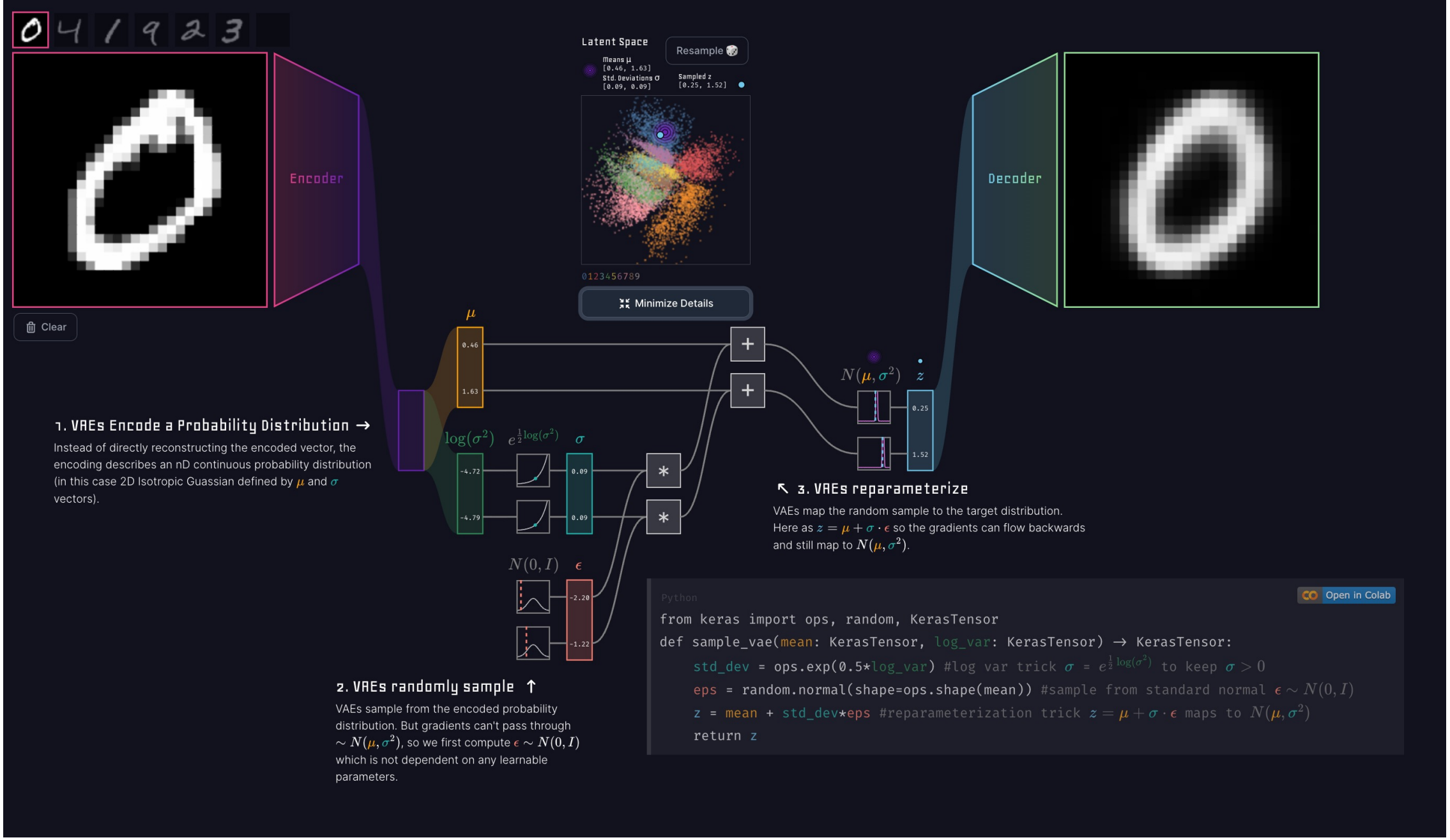


VAE: Latent perturbation



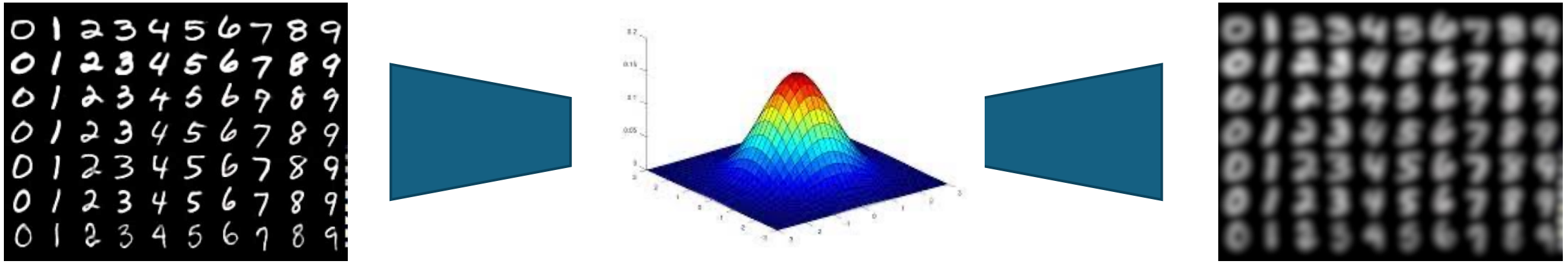
Different dimension of z codes different interpretable latent features

https://colab.research.google.com/drive/1cBumzL8jnfENDMykNFsx06os-xZv2pat



https://www.donnybertucci.com/project/vae-explainer

Why Blurry?

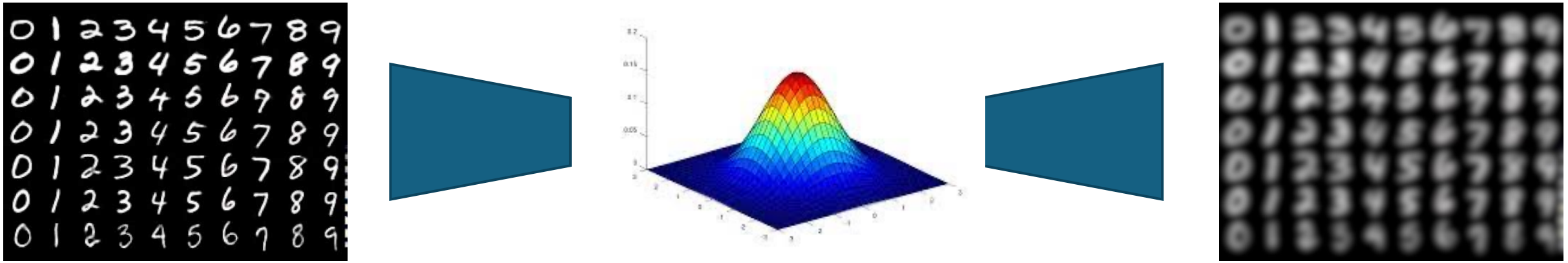


One reason: the **averaging effect.**
Our data usually has multiple modes (e.g., 0~9 in mnist)
VAE forces the **multi-mode** data distribution to a **unit-mode** Gaussian distribution.
Thus, it may represent these modes by averaging their characteristics.
During generation stage, it may produce samples “between” different modes, which could be blurry.

Besides, the **pixel-wise reconstruction loss** will also lead to averaging effect.

Generative Adversarial Networks (GANs)

VAE: generated samples have limited diversity.

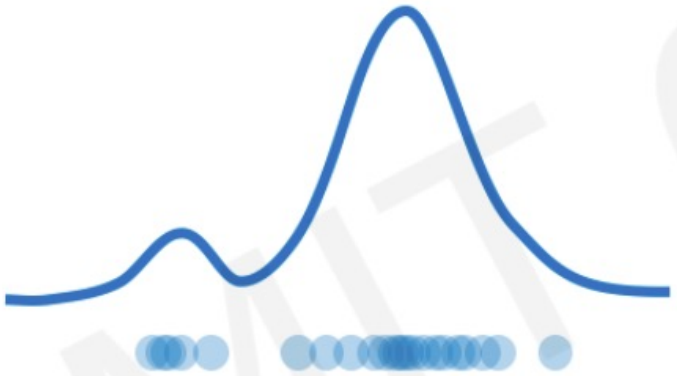


GAN fixes this problem.

Generative Models

Goal: Take training samples as input from some distribution and learn a model representing that distribution

Density Estimation

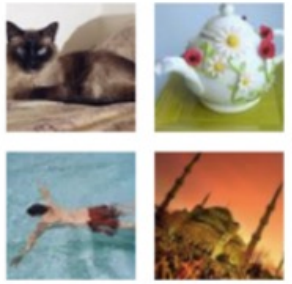


Sample Generation



Input samples

Training data ~ $P_{data}(x)$



Generated samples

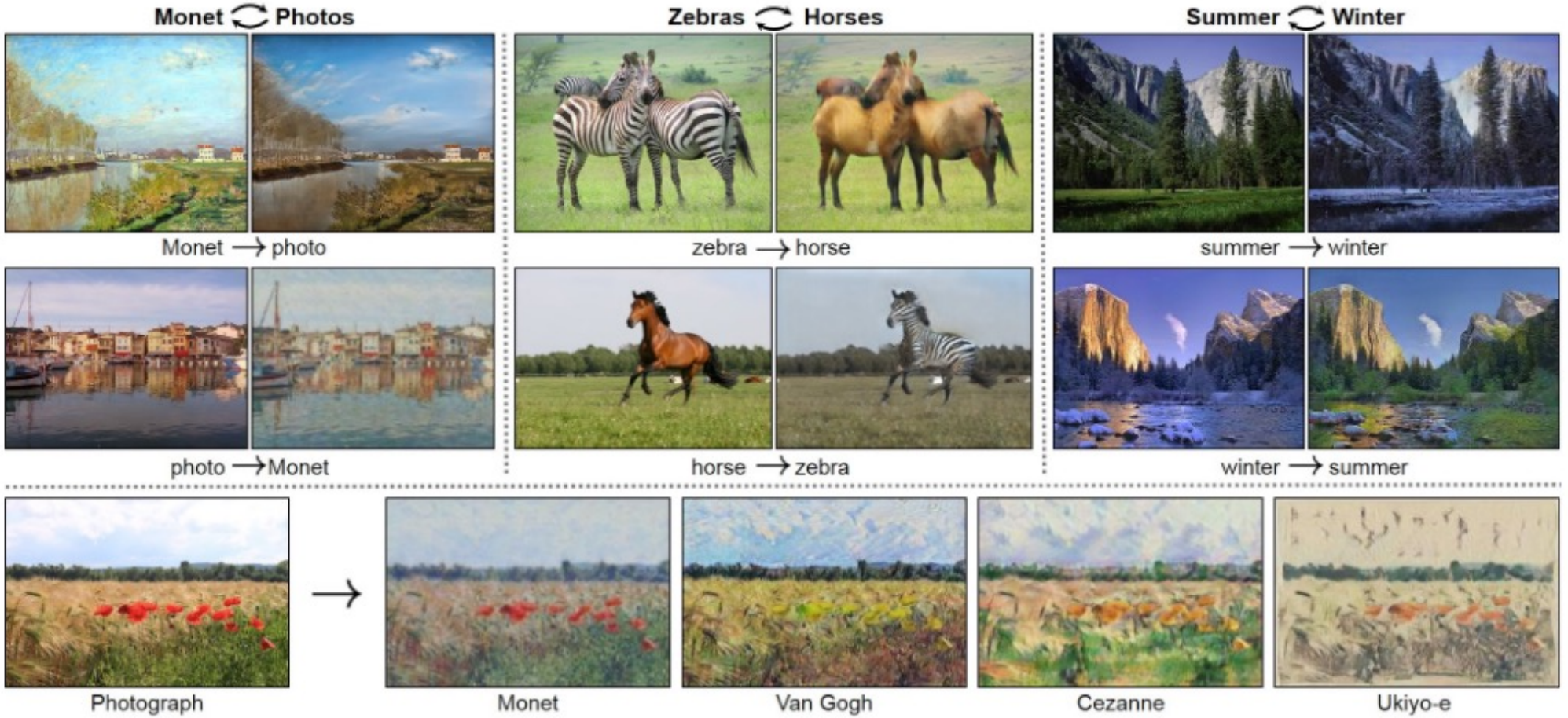
Generated ~ $P_{model}(x)$

How can we learn $P_{model}(x)$ similar to $P_{data}(x)$?

MIT 6.S191

Generative Models

Given training data, generate new samples from the same distribution

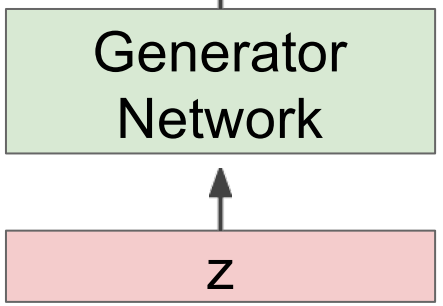


Generative Adversarial Networks

Ian Goodfellow et al., “Generative Adversarial Nets”, NIPS 2014

Output: Sample from training distribution





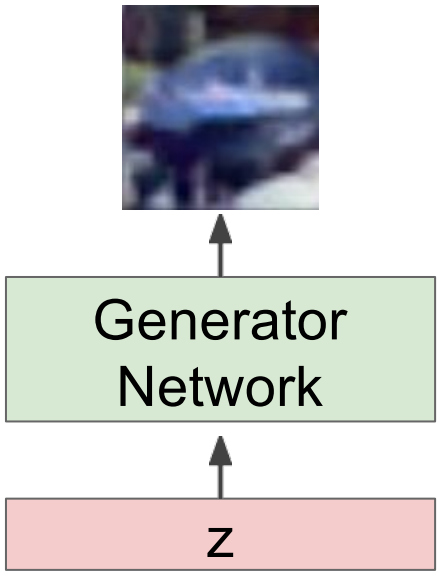
Input: Random noise

Generative Adversarial Networks

Ian Goodfellow et al., “Generative Adversarial Nets”, NIPS 2014

we don’t know which sample z maps to which training image -> can’t learn by reconstructing training images

Output: Sample from training image distribution



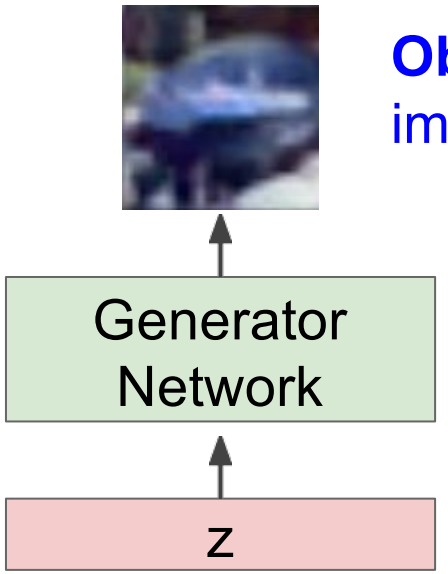
Input: Random noise from Gaussian distribution

Generative Adversarial Networks

Ian Goodfellow et al., “Generative Adversarial Nets”, NIPS 2014

we don’t know which sample z maps to which training image -> can’t learn by reconstructing training images

Output: Sample from training image distribution



Objective: generated images should look “real”

Input: Random noise from Gaussian distribution

Generative Adversarial Networks

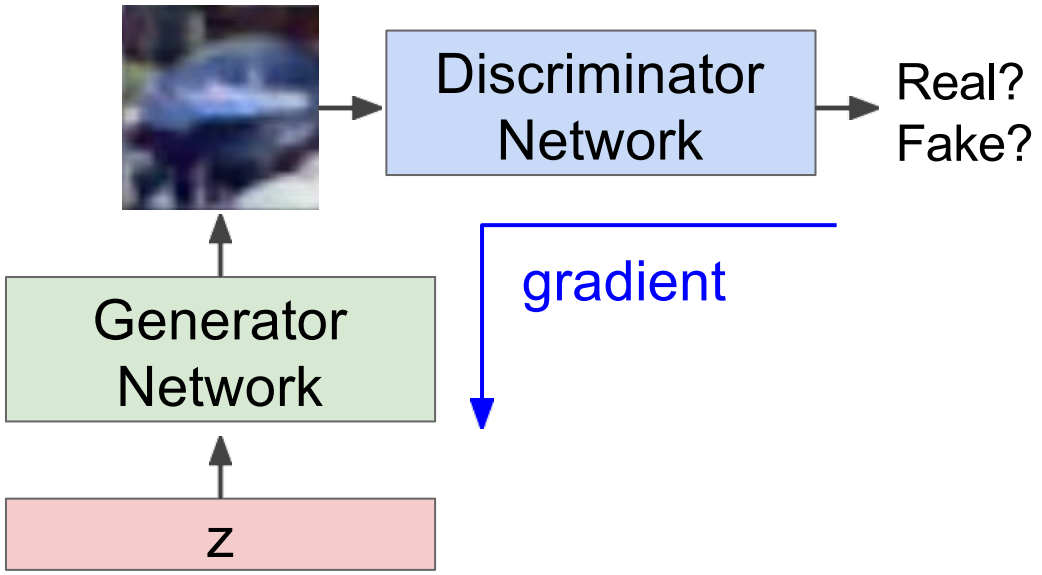
Ian Goodfellow et al., “Generative Adversarial Nets”, NIPS 2014

we don’t know which sample z maps to which training image -> can’t learn by reconstructing training images

Solution: Use a discriminator network to tell whether the generate image is within data distribution (“real”) or not

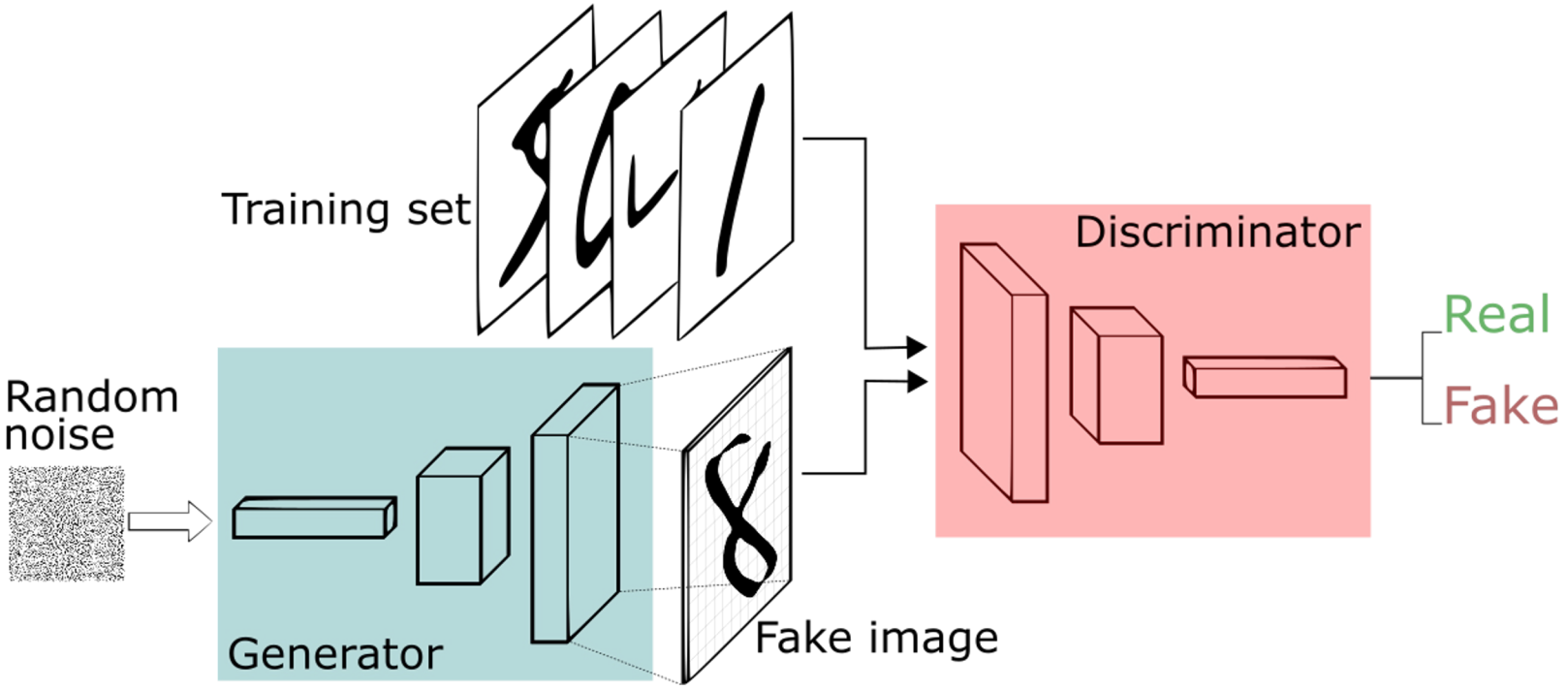
Output: Sample from training image distribution

Input: Random noise from Gaussian distribution



Training GANs: Two-player game

GANs are a way to make a generative model by having two neural networks compete with each other.



For each batch (step)

1. Train D, fix G
2. Train G, fix D, try to fool D, tell D that fake image is real

Training GANs: Two-player game

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Discriminator network: try to distinguish between real and fake images
Generator network: try to fool the discriminator by generating real-looking images

Train jointly in **minimax game**

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Generator objective

Discriminator objective

Training GANs: Two-player game

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Discriminator network: try to distinguish between real and fake images
Generator network: try to fool the discriminator by generating real-looking images

Train jointly in **minimax game**

Discriminator outputs likelihood in (0,1) of real image

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Discriminator output for real data x

Discriminator output for generated fake data G(z)

Training GANs: Two-player game

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Discriminator network: try to distinguish between real and fake images
Generator network: try to fool the discriminator by generating real-looking images

Train jointly in **minimax game**

Discriminator outputs likelihood in (0,1) of real image

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Discriminator output for real data x

Discriminator output for generated fake data G(z)

Training GANs: Two-player game

Ian Goodfellow et al., "Generative Adversarial Nets", NIPS 2014

Discriminator network: try to distinguish between real and fake images
Generator network: try to fool the discriminator by generating real-looking images

Train jointly in **minimax game**

Discriminator outputs likelihood in (0,1) of real image

Minimax objective function:

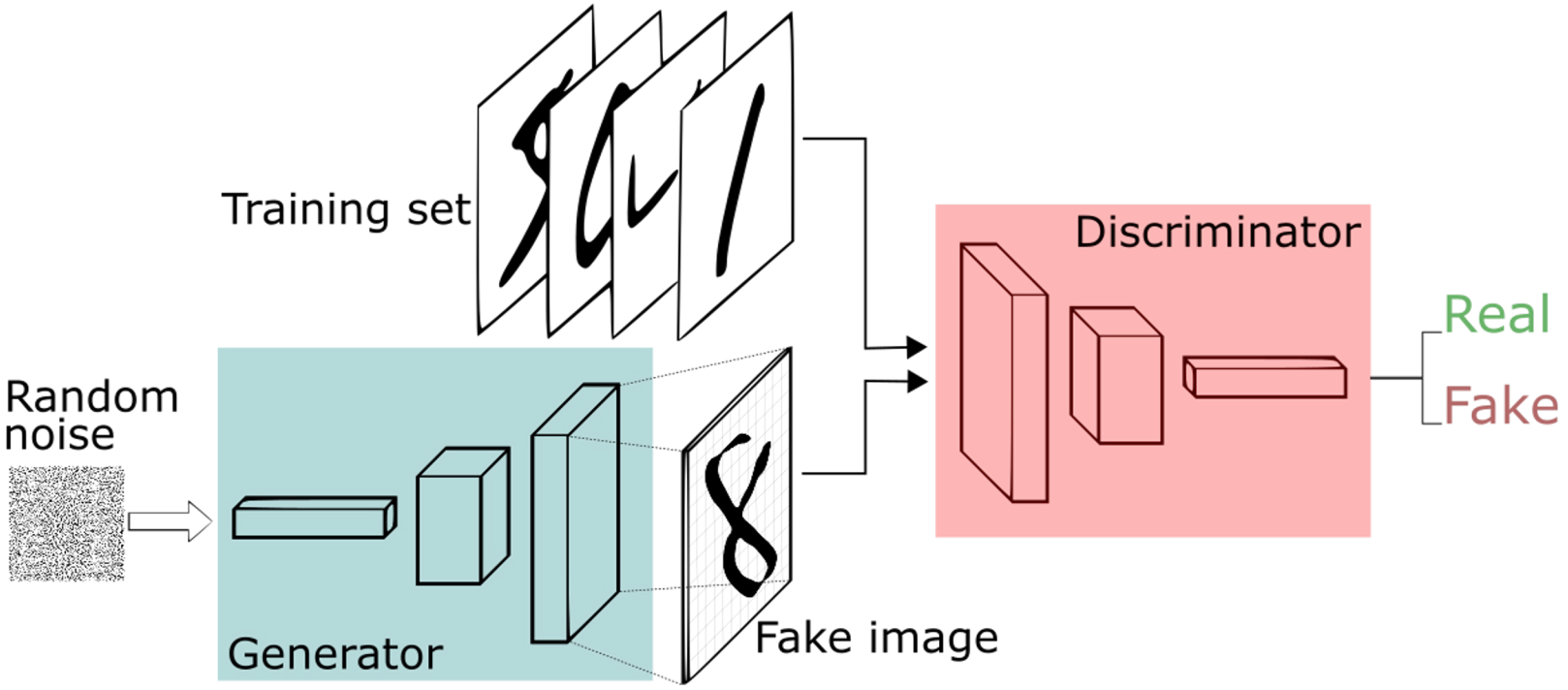
$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Discriminator output for real data x

Discriminator output for generated fake data G(z)

Training GANs : Two-player game

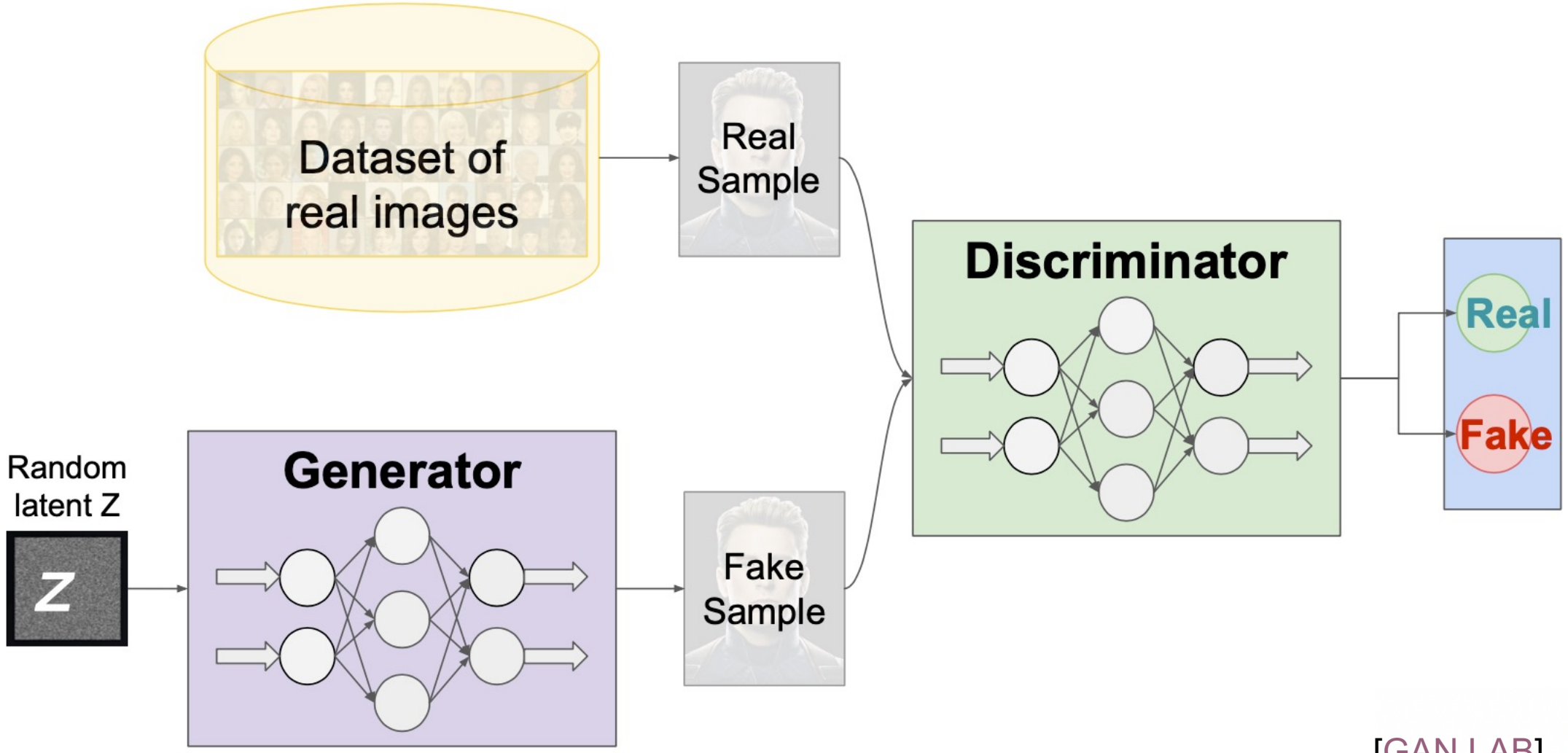
GANs are a way to make a generative model by having two neural networks compete with each other.



For each batch (step)

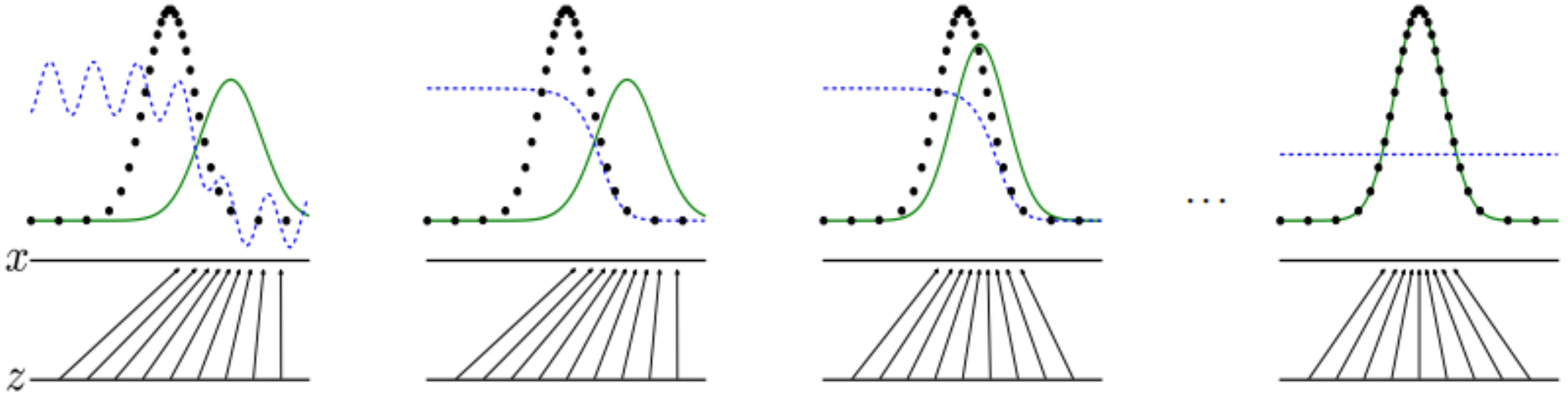
1. Train D, fix G
2. Train G, fix D, try to fool D, tell D that fake image is real

GANs : Creating Images



[GAN LAB]

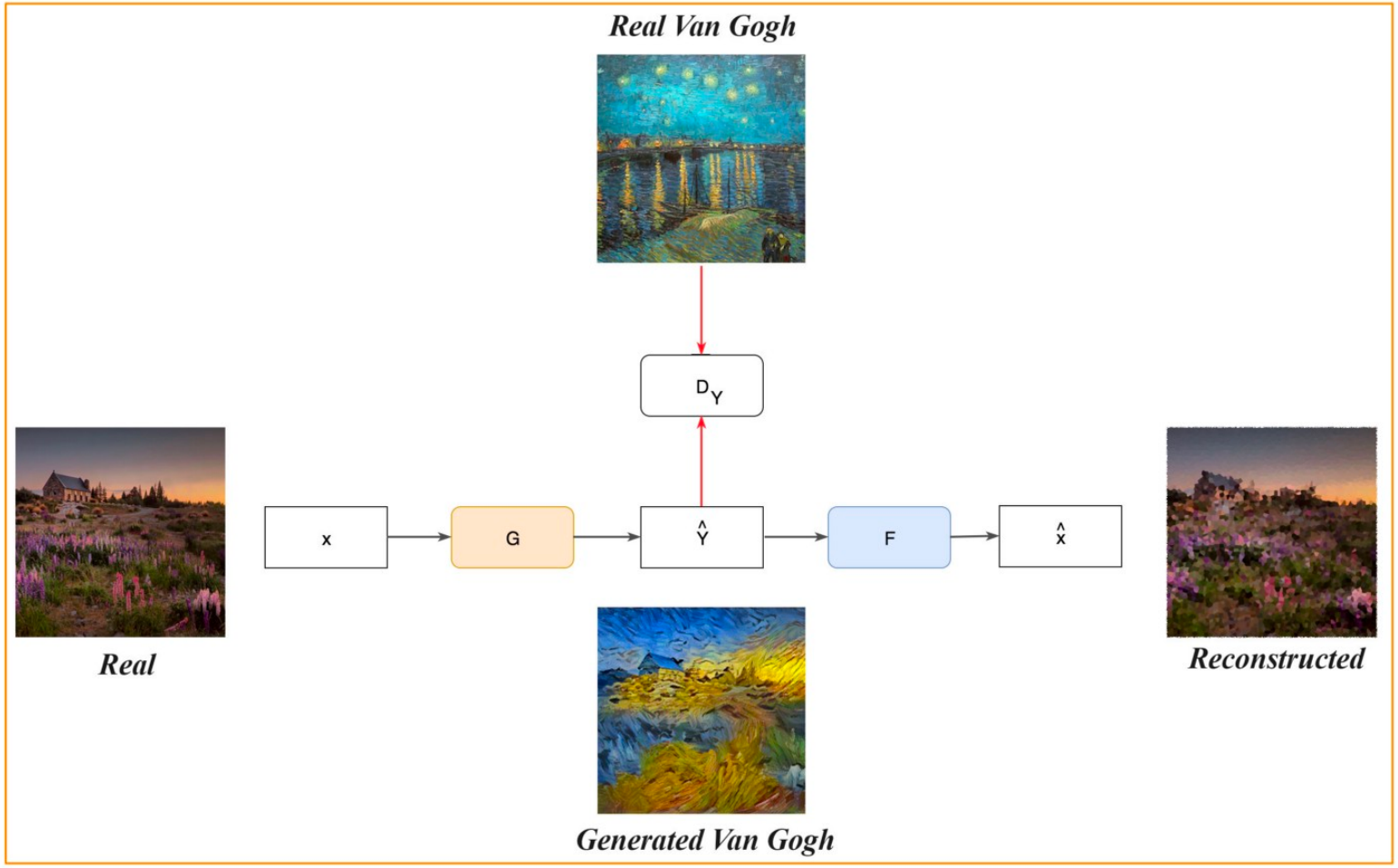
GANs : Learn a distribution



The first figure ever published in a GAN paper, illustrating a GAN learning to map uniform noise to the normal distribution.

- Black dots are the real data points
- Green curve is the distribution generated by the GAN,
- Blue curve is the discriminator's confidence that a sample in that region is real.
- Here **x** denotes the sample space and **z** the latent space. (Source: Goodfellow 2014)

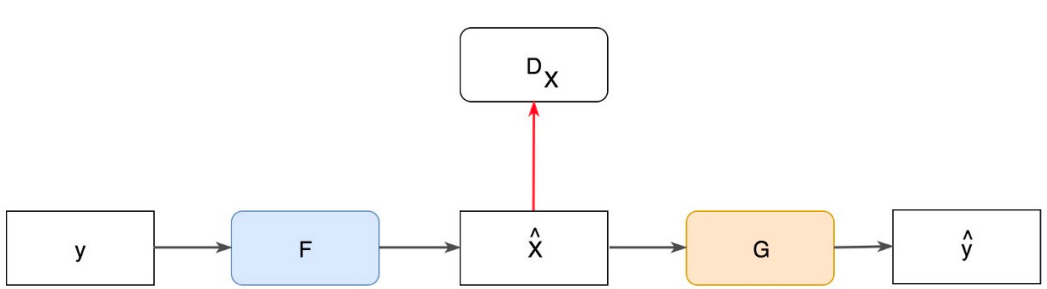
CycleGAN

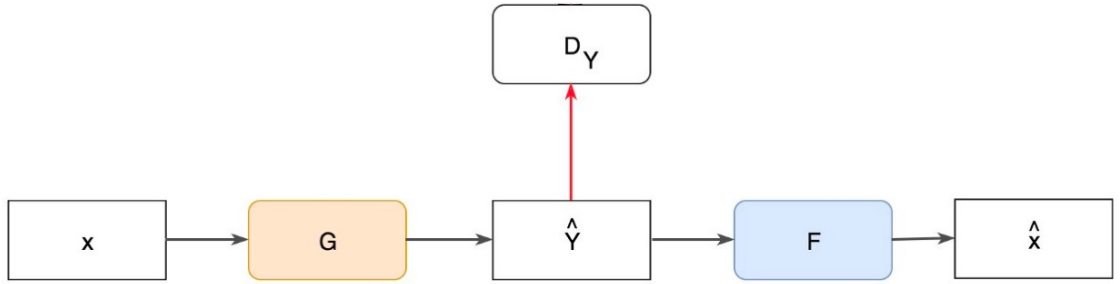


We build three networks

- A generator ***G*** to convert a real image to a Van Gogh style picture.
- A generator ***F*** to convert a Van Gogh style picture to a real image.
- A discriminator ***D*** to identify real or generated Van Gogh pictures.

CycleGAN





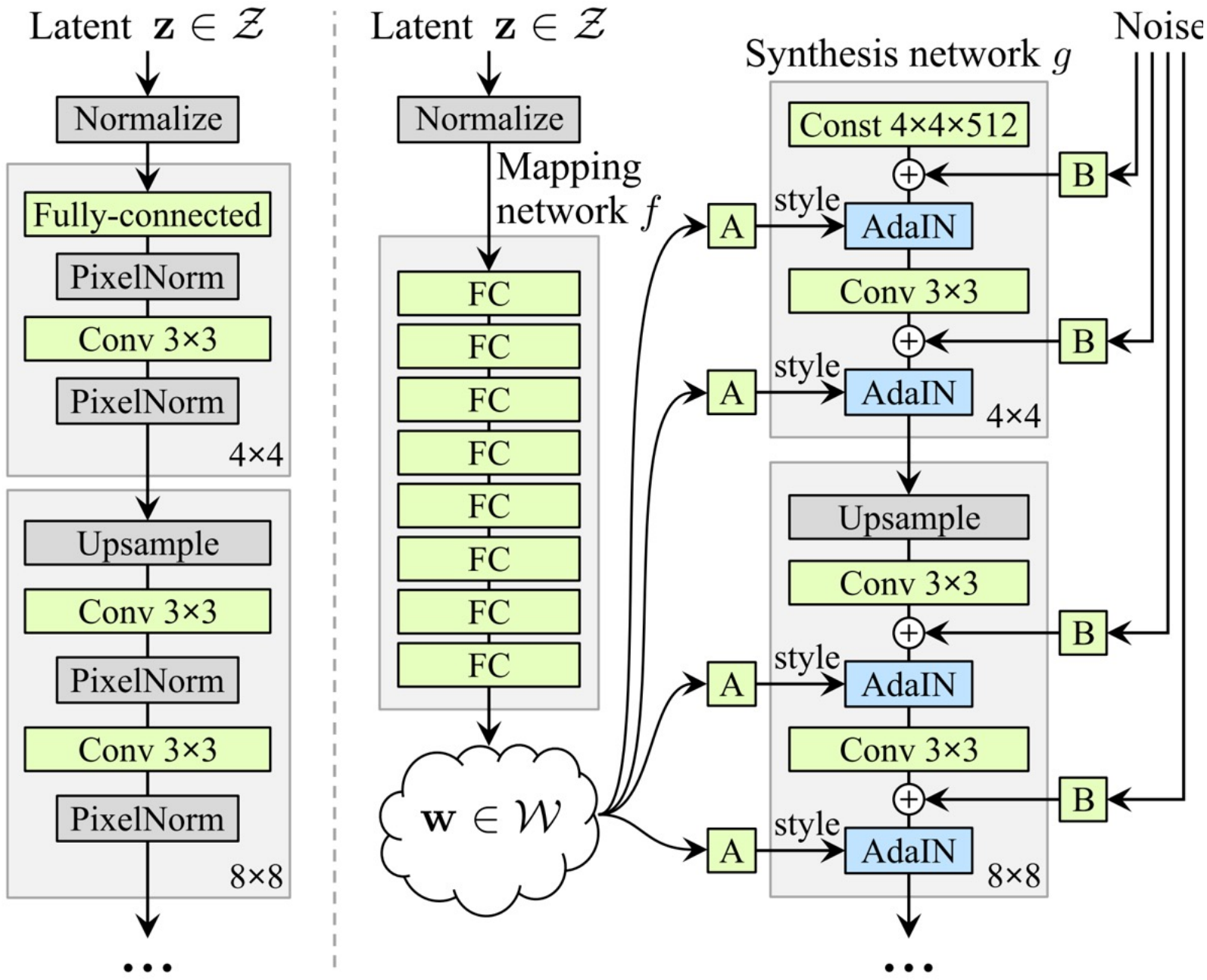
Cycle consistency loss:

$$\mathcal{L}_{\text{cyc}}(G, F) = \mathbb{E}_{x \sim p_{\text{data}}(x)}[\|F(G(x)) - x\|_1] + \mathbb{E}_{y \sim p_{\text{data}}(y)}[\|G(F(y)) - y\|_1]$$

Adversary loss:

$$\mathcal{L}_{\text{GAN}}(G, D, X, Y)$$
$$\textit{For } G, \text{ minimize } \mathbb{E}_{x \sim p_{\text{data}}(x)}[(D(G(x)) - 1)^2]$$
$$\textit{For } D, \text{ minimize } \mathbb{E}_{y \sim p_{\text{data}}(y)}[(D(y) - 1)^2] + \mathbb{E}_{x \sim p_{\text{data}}(x)}[D(G(x))^2]$$

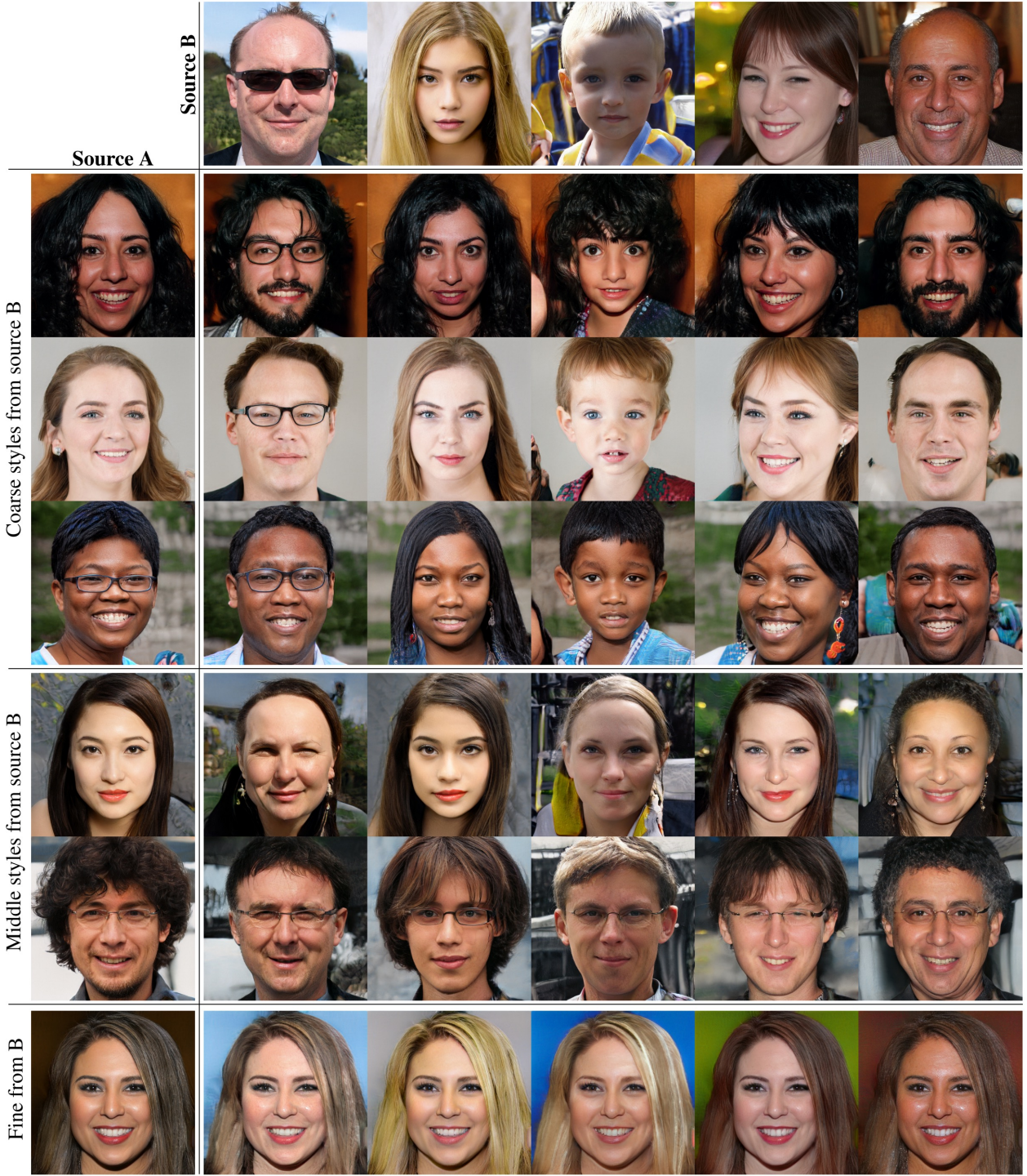
StyleGAN



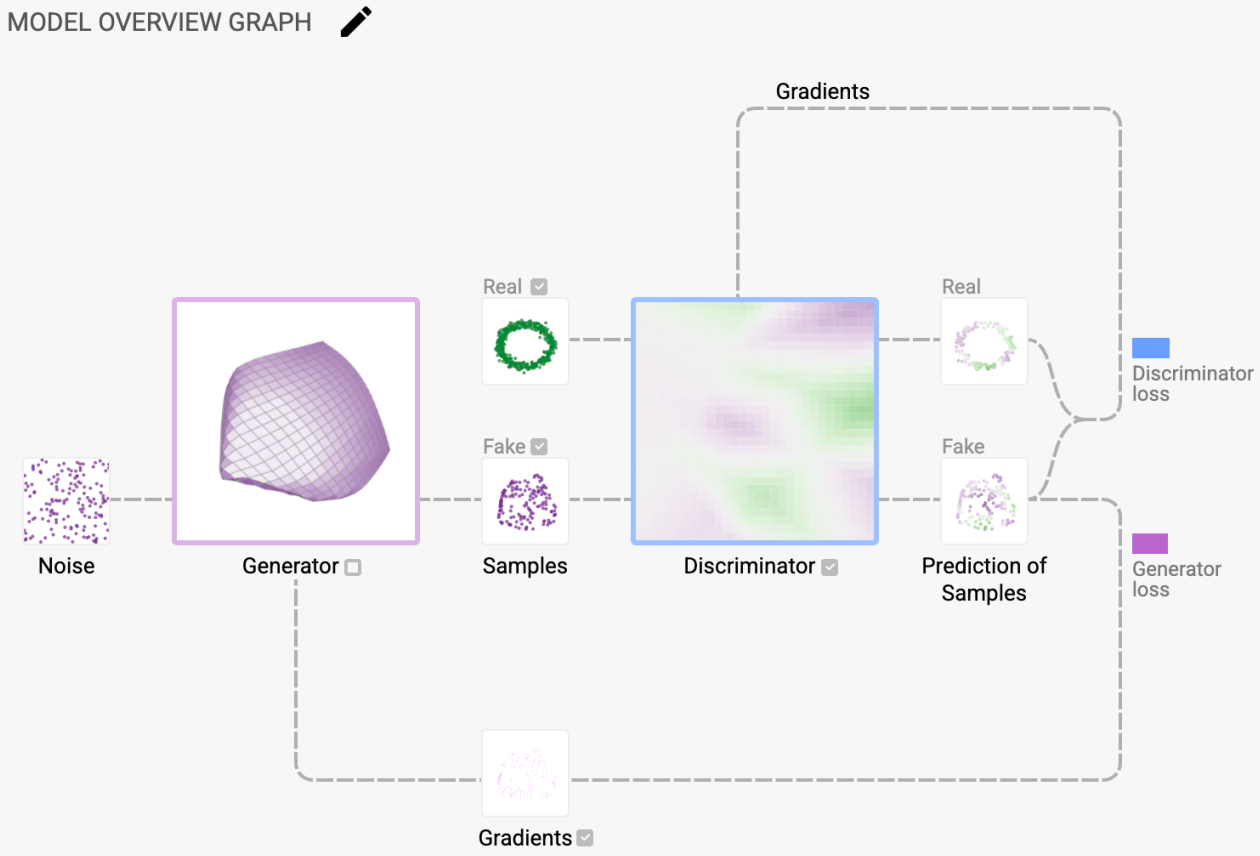
(a) Traditional

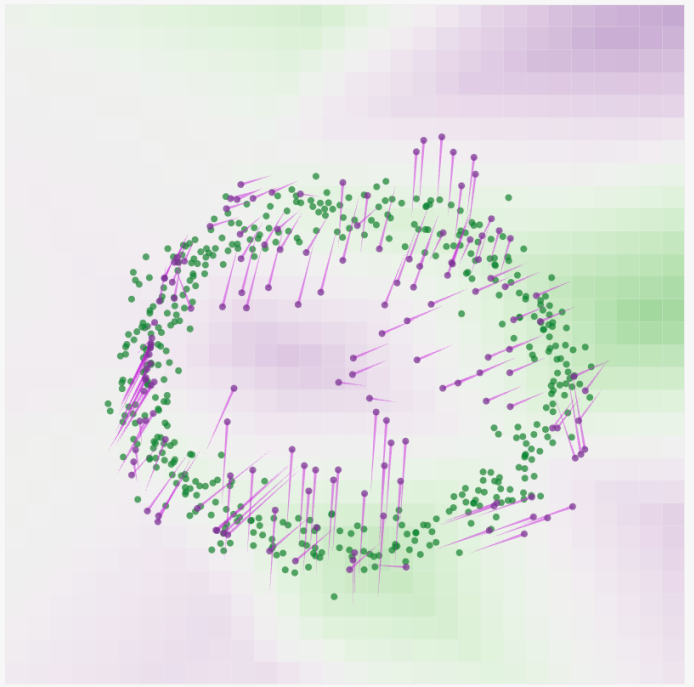
(b) Style-based generator

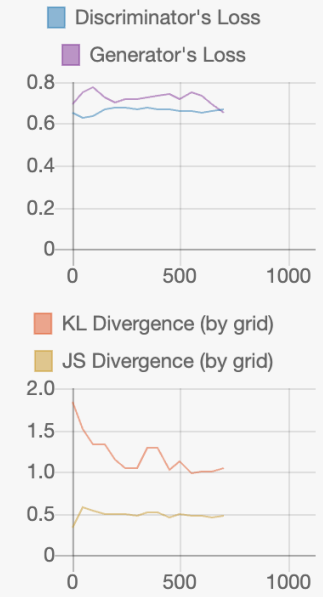



Source B
Source A
Coarse styles from source B
Middle styles from source B
Fine from B

Problem: Bad Convergence!

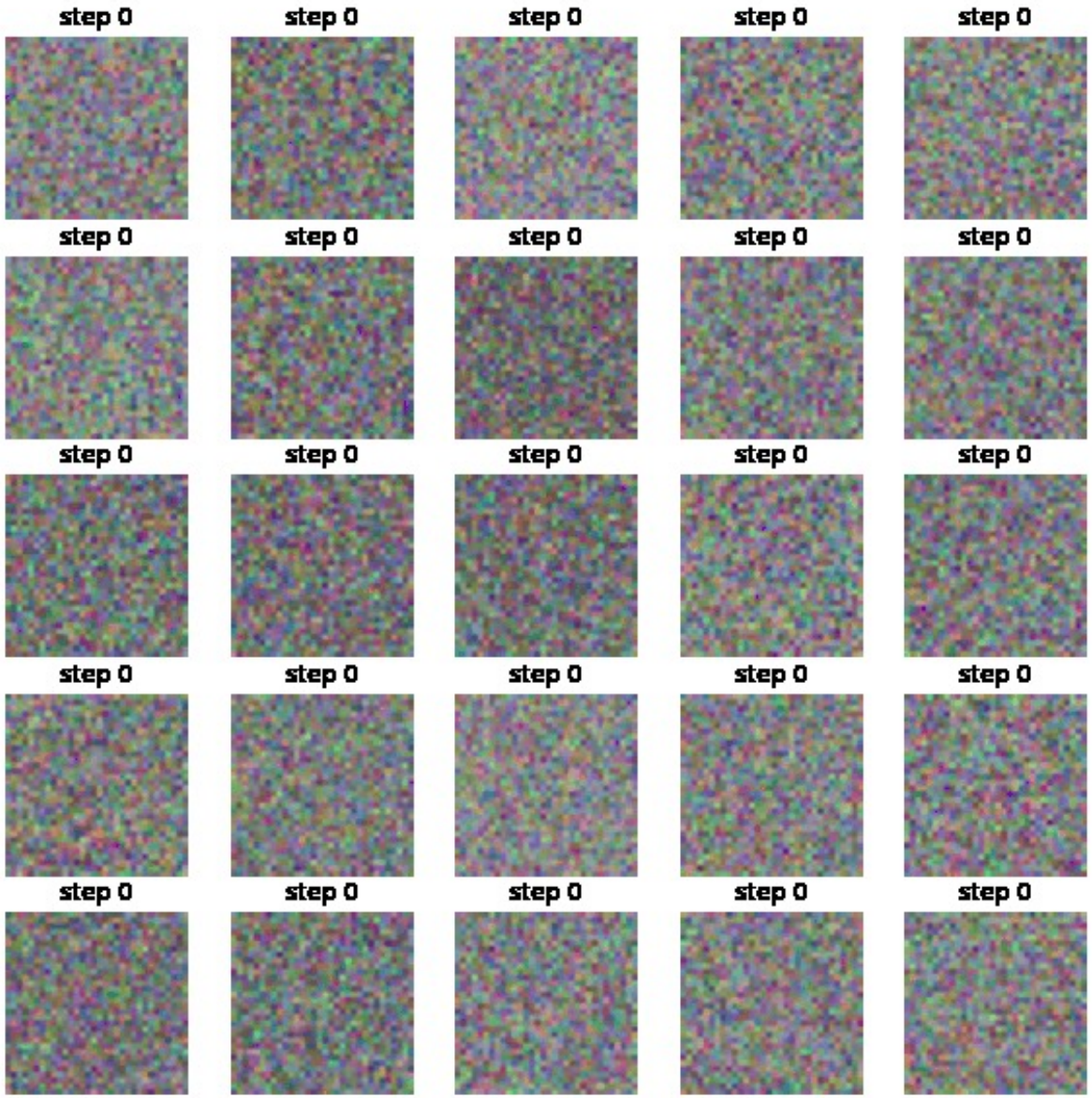




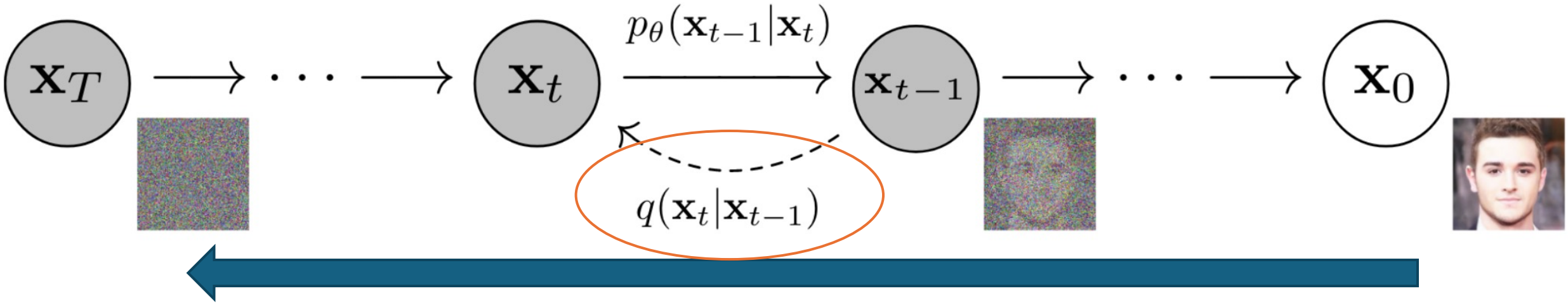


https://poloclub.github.io/ganlab/

Diffusion Model



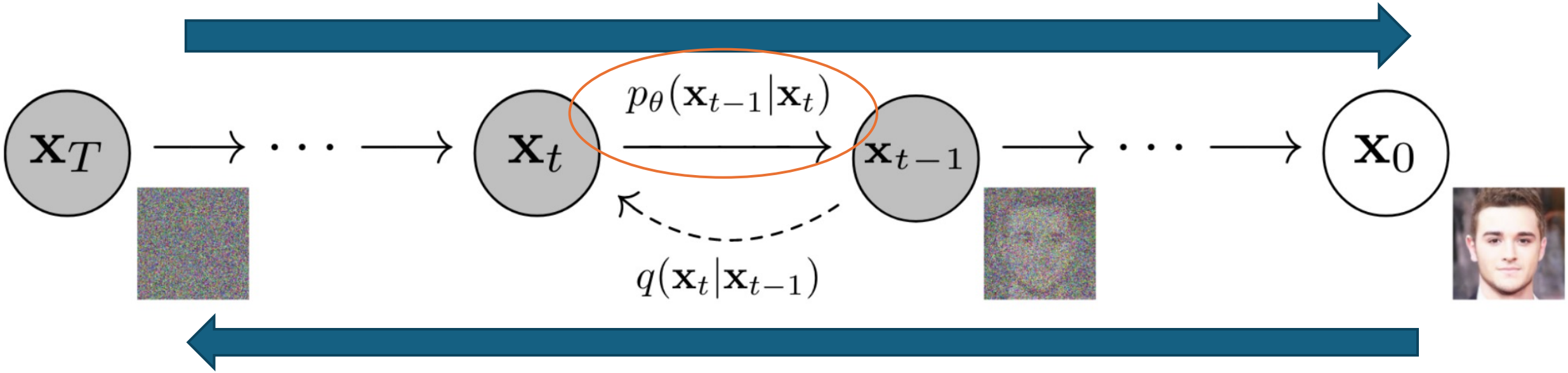
Method



Forward diffusion process: progressively destroy structure in real data

Method

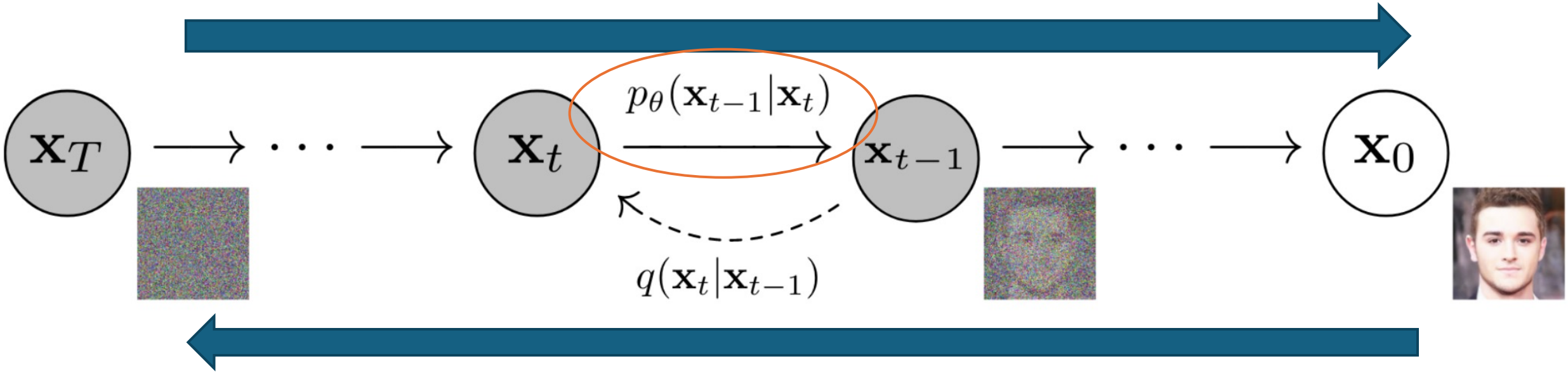
Revert diffusion process: progressively reconstruct the data



Forward diffusion process: progressively destroy structure in real data

Method

Revert diffusion process: progressively reconstruct the data



Forward diffusion process: progressively destroy structure in real data

Idea: Break down the complicated mapping functions in multiple manageable steps.

Method

Forward diffusion process

Real data x_0



Step 1 $x_1 = \sqrt{1-\beta_1}\, x_0 + \sigma_1 \epsilon, \quad \epsilon \in N(0,1)$

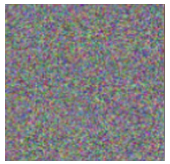


Step 2 $x_2 = \sqrt{1-\beta_2}\, x_1 + \sigma_2 \epsilon,$



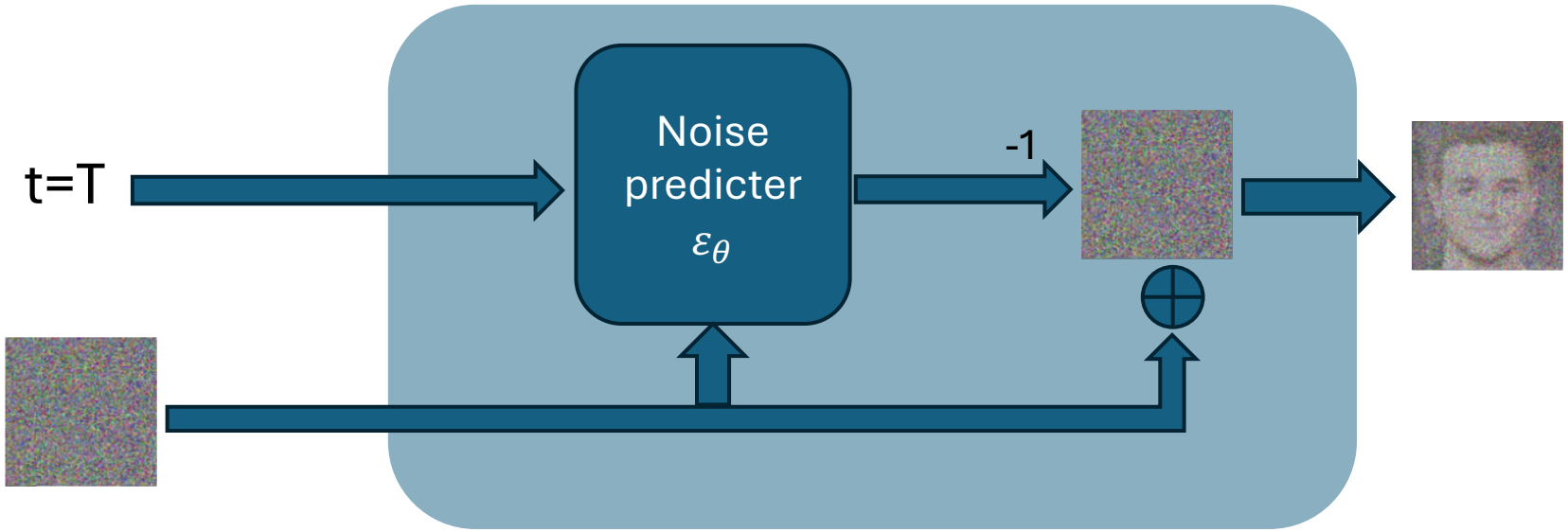
...

Step T $x_T = \sqrt{1-\beta_T}\, x_{T-1} + \sigma_T \epsilon,$



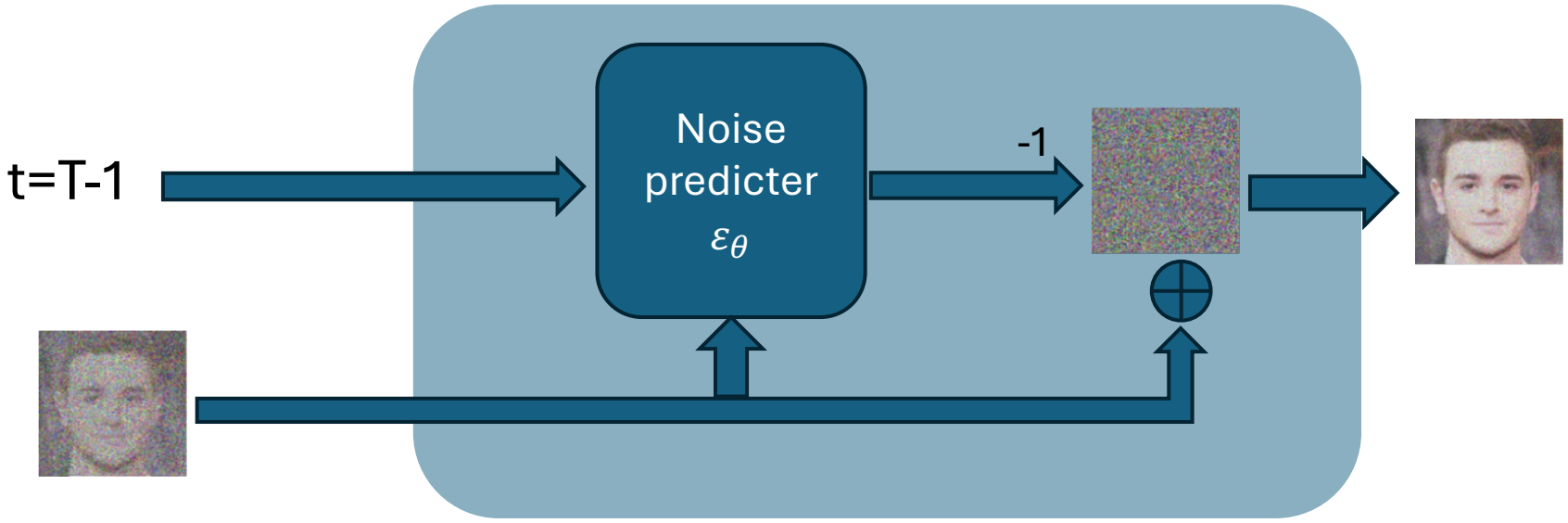
Method

Revert diffusion process



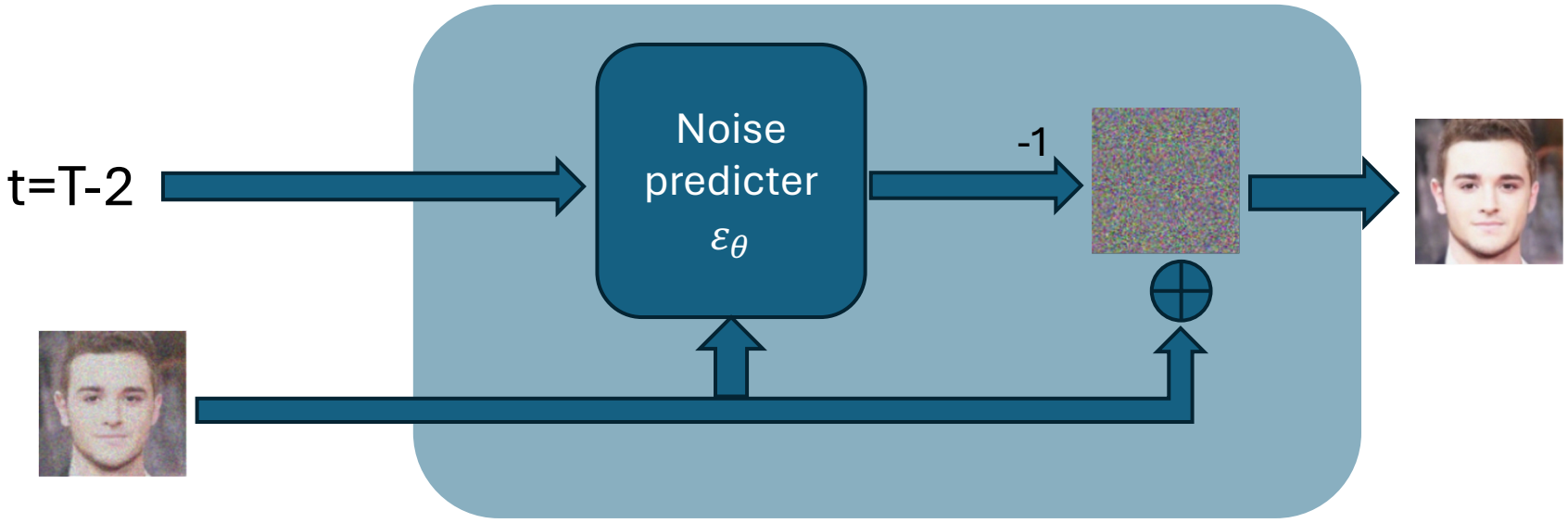
Method

Revert diffusion process



Method

Revert diffusion process



Method

Revert diffusion process

$$\alpha_t = 1 - \beta_t, \bar{\alpha}_t = \prod_{s=1}^{T} \alpha_s$$

Step T x_T

$\epsilon_\theta(x_T, t)$ is parameterized by nerual networks

Step T-1 $$x_{T-1} = \frac{1}{\sqrt{1-\beta_T}}\left(x_T - \frac{\beta_T}{\sqrt{1-\bar{\alpha}_T}}\epsilon_\theta(x_T, T)\right) + \sigma_T \epsilon$$

Remove the noise estimated by a neural networks

A randomness term that introduces more diversity

OUT OF THE SCOPE OF THIS CLASS. NO NEED TO UNDERSTAND HERE

Method

Revert diffusion process

$$\alpha_t = 1 - \beta_t, \bar{\alpha}_t = \prod_{s=1}^{T} \alpha_s$$

Step T x_T

$\epsilon_\theta(x_T, t)$ is parameterized by nerual networks

Step T-1 $$x_{T-1} = \frac{1}{\sqrt{1-\beta_T}}\left(x_T - \frac{\beta_T}{\sqrt{1-\bar{\alpha}_T}}\epsilon_\theta(x_T, T)\right) + \sigma_T \epsilon$$

Remove the noise estimated by a neural networks

A randomness term that introduces more diversity

⋮

Step 0 $$x_0 = \frac{1}{\sqrt{1-\beta_1}}\left(x_1 - \frac{\beta_1}{\sqrt{1-\bar{\alpha}_1}}\epsilon_\theta(x_1, T)\right) + \sigma_1 \epsilon$$

OUT OF THE SCOPE OF THIS CLASS. NO NEED TO UNDERSTAND HERE

Algorithm 1 Training

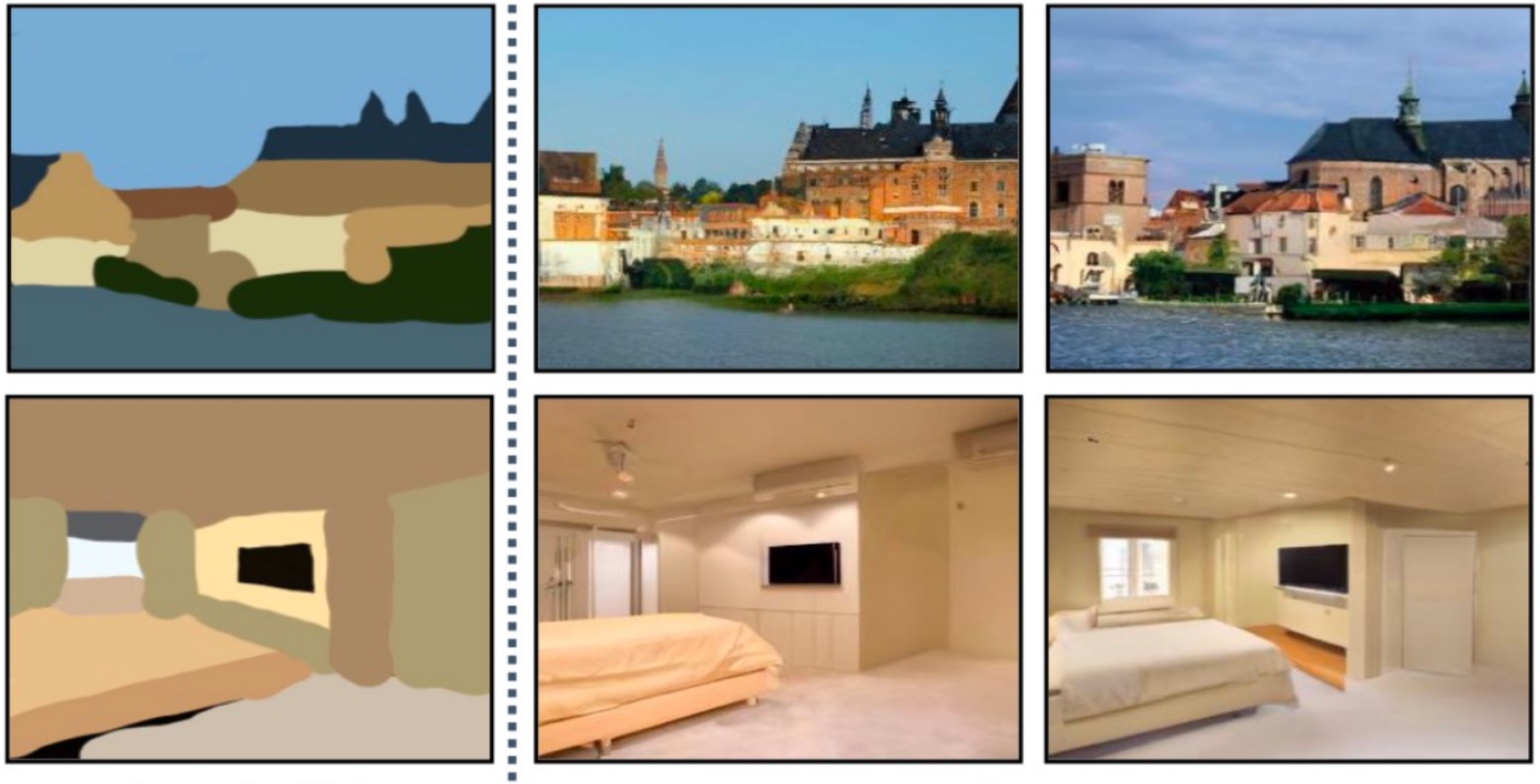
1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \text{Uniform}(\{1, \dots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \dots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

OUT OF THE SCOPE OF THIS CLASS. NO NEED TO UNDERSTAND HERE

Stroke Painting to Image

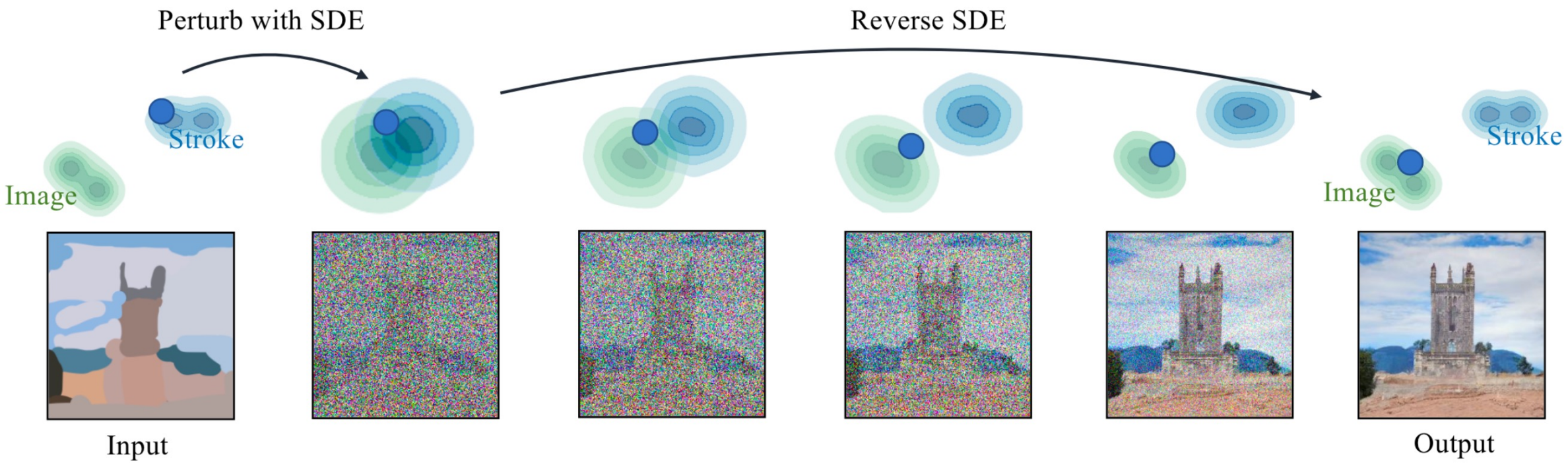


Input (guide)

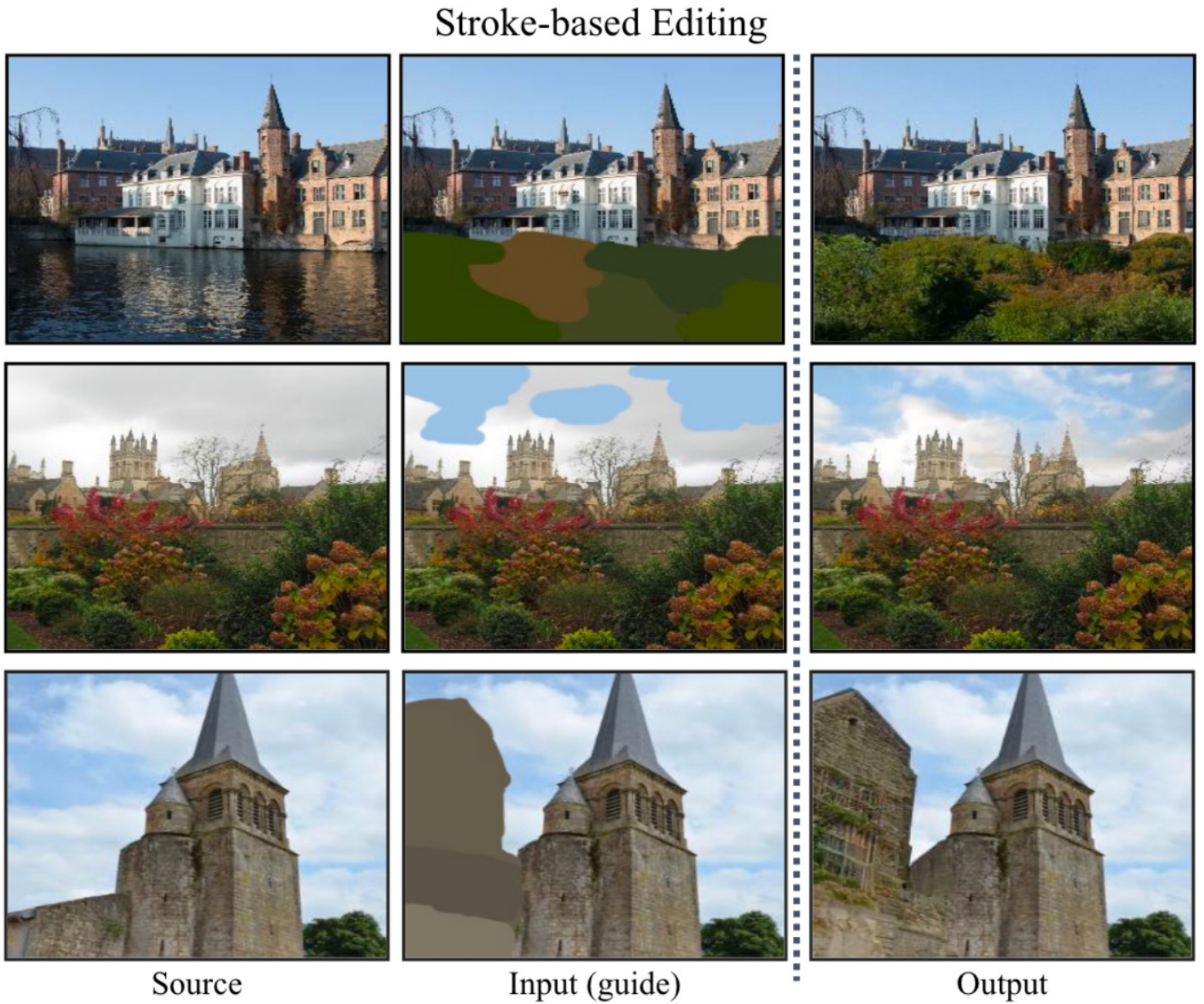
Output

idea

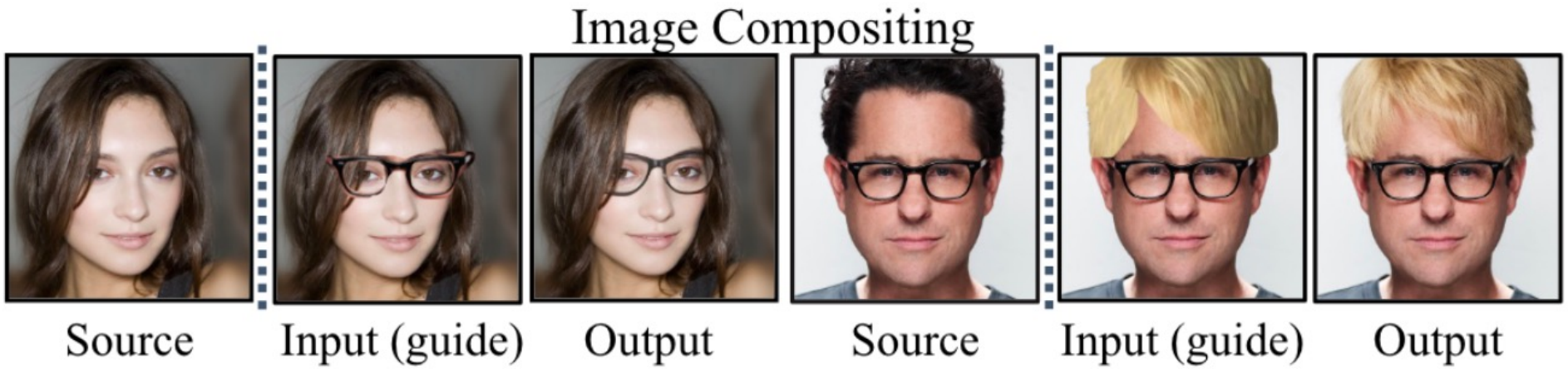
Assumption: a well-trained diffusion model



Applications

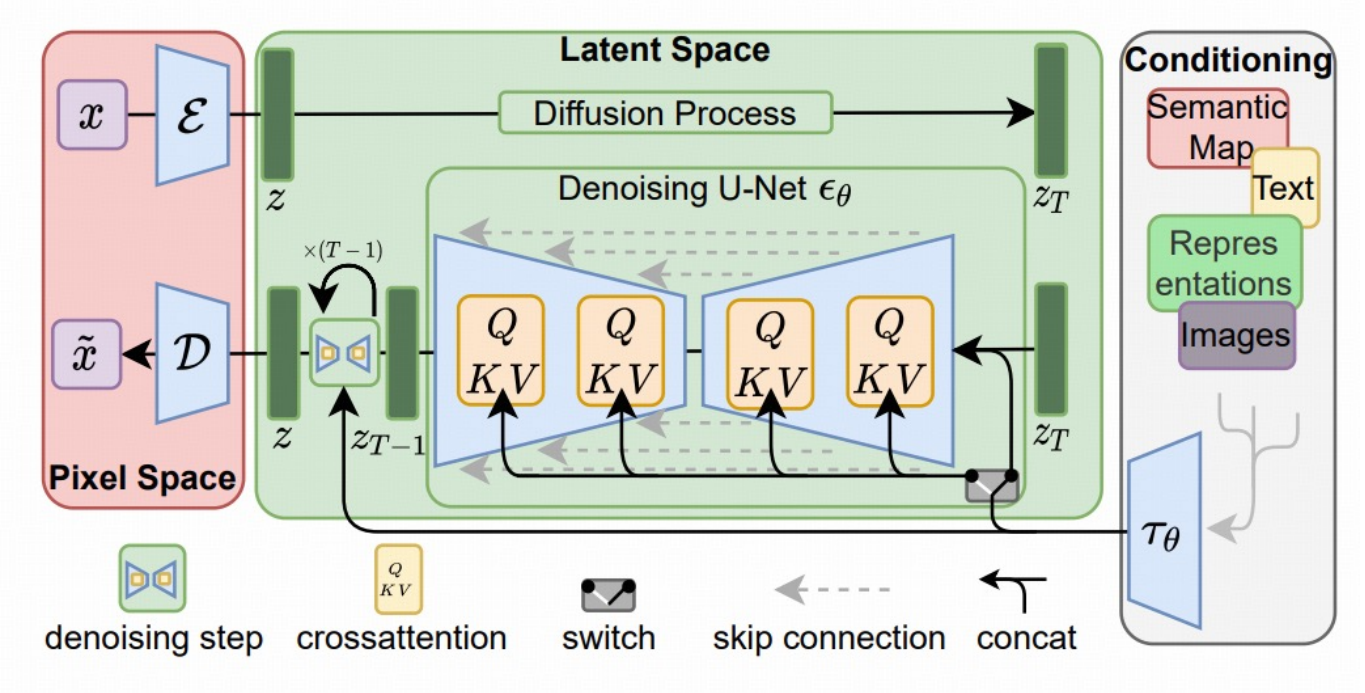


Applications

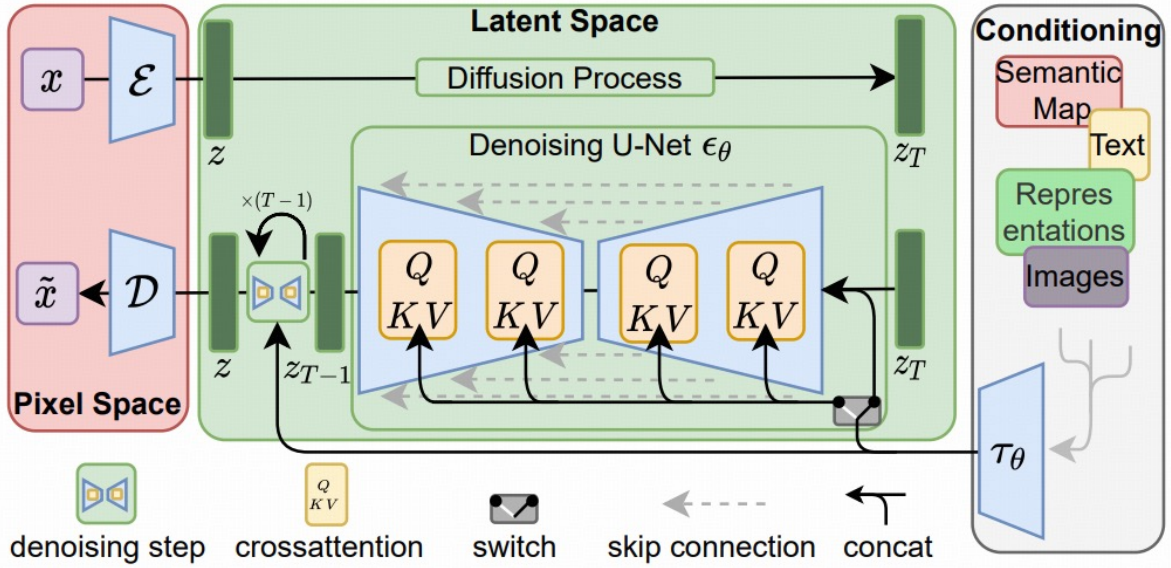


State-of-the-art Generative Models & Applications

- Text to Image Generation
  - Stable Diffusion Series (Runway, Stability AI)
    - **SD-1, SD-1.5, SD-2**
      - Control Net
      - SD-XL
    - SD-3



How Stable Diffusion Works: T



- The Problem: Generating images directly massive and slow.
- The Solution (The Autoencoder)
- The Denoising U-Net: It operates entirely within the Latent Space. It starts with a noisy latent tensor and iteratively predicts and removes noise step-by-step to recover a clean image representation.
- Conditioning (Guiding the Output): Text prompts, semantic maps, or reference images are passed through a domain-specific encoder. The U-Net uses a Cross-Attention mechanism at various layers, allowing the U-Net to "look at" the text prompt while deciding how to denoise the image.

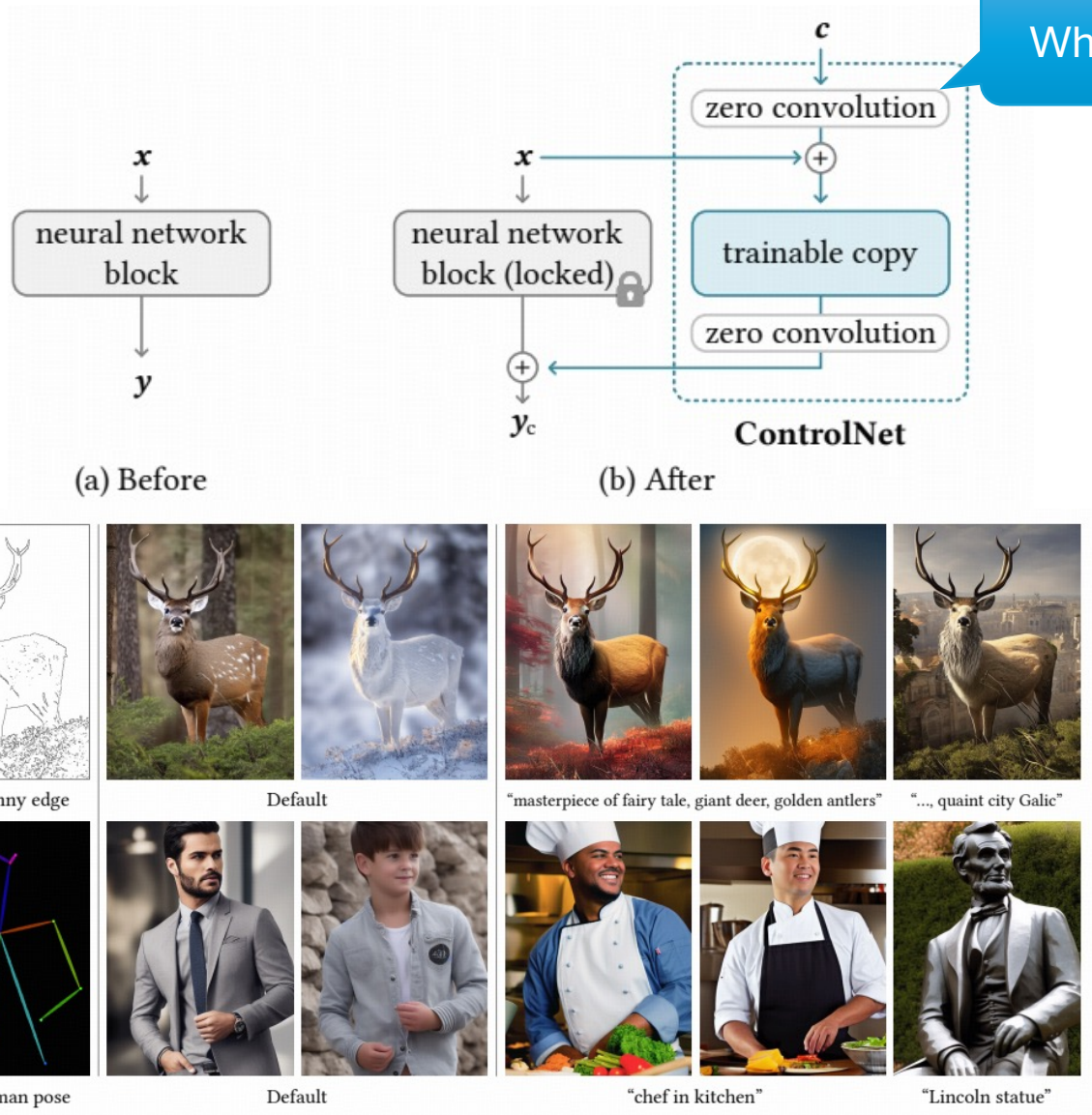
ControlNet: Taming the Diffusion Process

- The Limitation of Text: It is difficult to use words to describe the exact pose of a character or the exact perspective lines of a room.
- ControlNet: It is a neural network architecture that "locks" the pre-trained diffusion model and adds a trainable copy of the model's encoding layers.
- Visual Conditioning: Instead of just text, ControlNet allows you to feed the U-Net rigid spatial conditions.
- Common Use Cases:
  - OpenPose: Extracting a skeleton from a reference photo to force the AI to draw a character in that exact pose.
  - Canny Edge: Turning a simple sketch into a fully rendered, photorealistic image while strictly obeying the sketch lines.
  - Depth Maps: Controlling the 3D spatial layout of a generated scene.

State-of-the-art Generative Models & Applications

- Text to Image Generation
  - Stable Diffusion Series
    - SD-1, SD-1.5, SD-2
      - **Control Net**
      - SD-XL
    - SD-3

What is it?



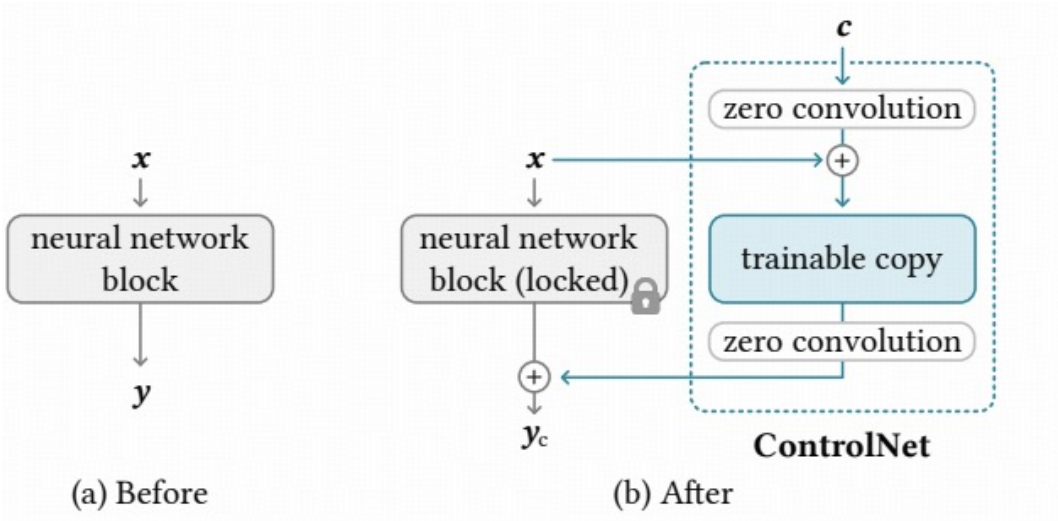
State-of-the-art Generative Models & Applications

Zero Convolution (1×1 convolution layers where both weights and biases are initialized to zero)

- Preserving the Knowledge of the Base Model: At the very start of training (iteration 0), the output of the Zero Conv layer is exactly 0.

$$y_c = y + \text{ControlNet}(x)$$

- Protecting the Gradient: Even though the weights start at zero, the **gradients** are not zero.





Source image
(for canny edge detection)



Canny edge (input)



Generated images (output)



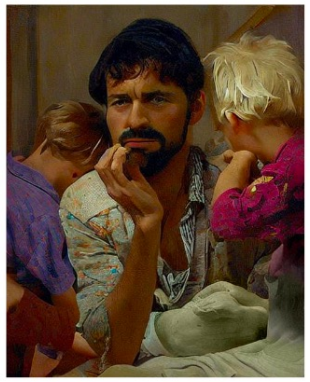


Input (Canny Edge)

Default

Automatic Prompt

User Prompt







“a man with beard sitting with two children”

“mother and two boys in a room, masterpiece, artwork”

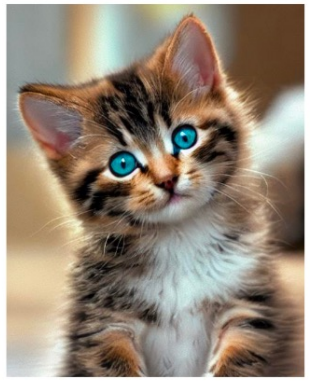


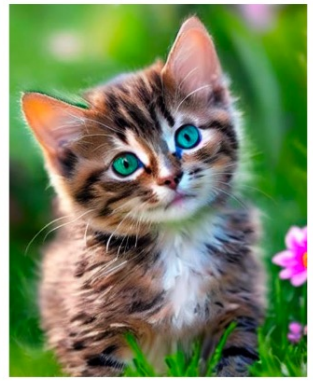




“a man in a suit and tie”

“a man in a white suit and tie”

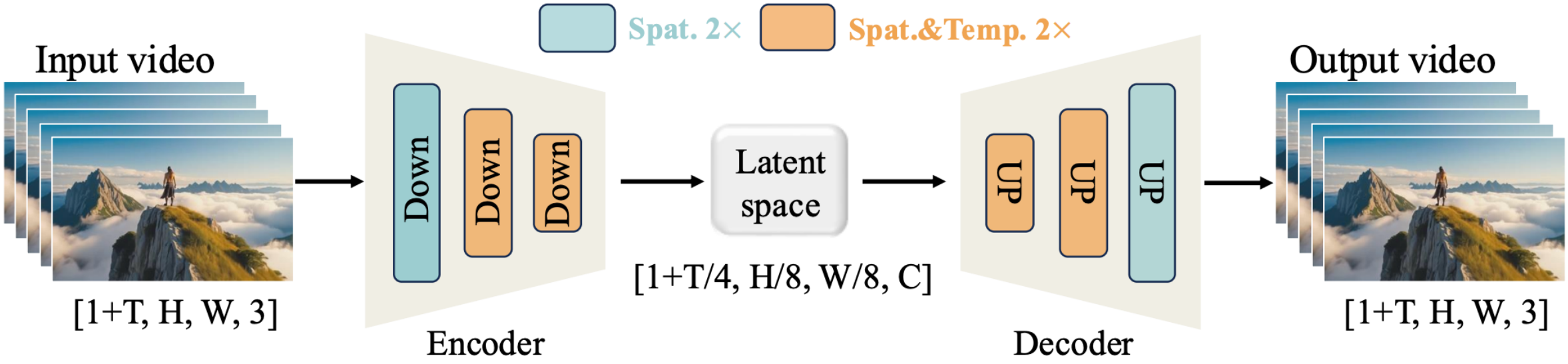



“a cat with blue eyes in a room”

“a cute cat in a garden, masterpiece, detailed wallpaper”

Video Generation



WAN: OPEN AND ADVANCED LARGE-SCALE VIDEO GENERATIVE MODELS

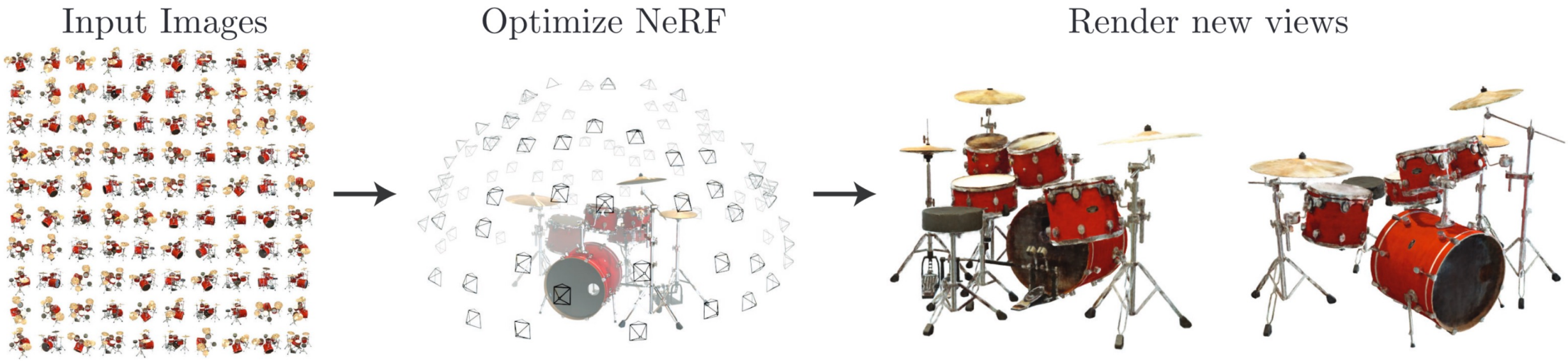




3D Reconstruction

Problem it solves

To get a scene representation based on a set of images.



Problem it solves

Input Images



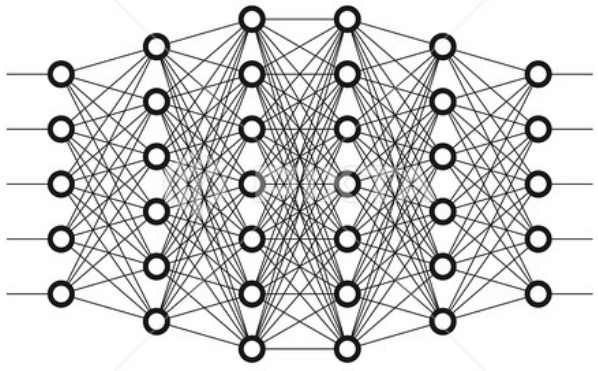
Optimize NeRF



Render new views







Use a neural network to represent the scene

Problem it solves

Input Images



Optimize NeRF

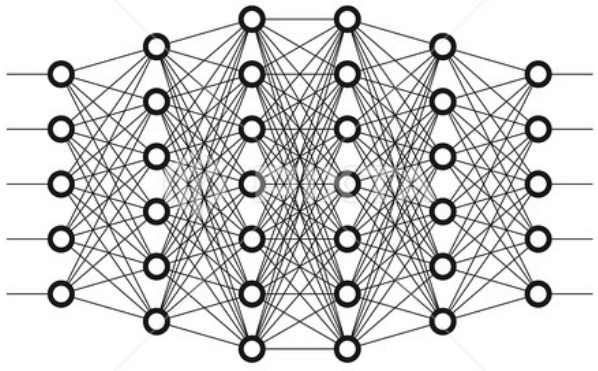


Render new views





(x, y, z, θ, ϕ)

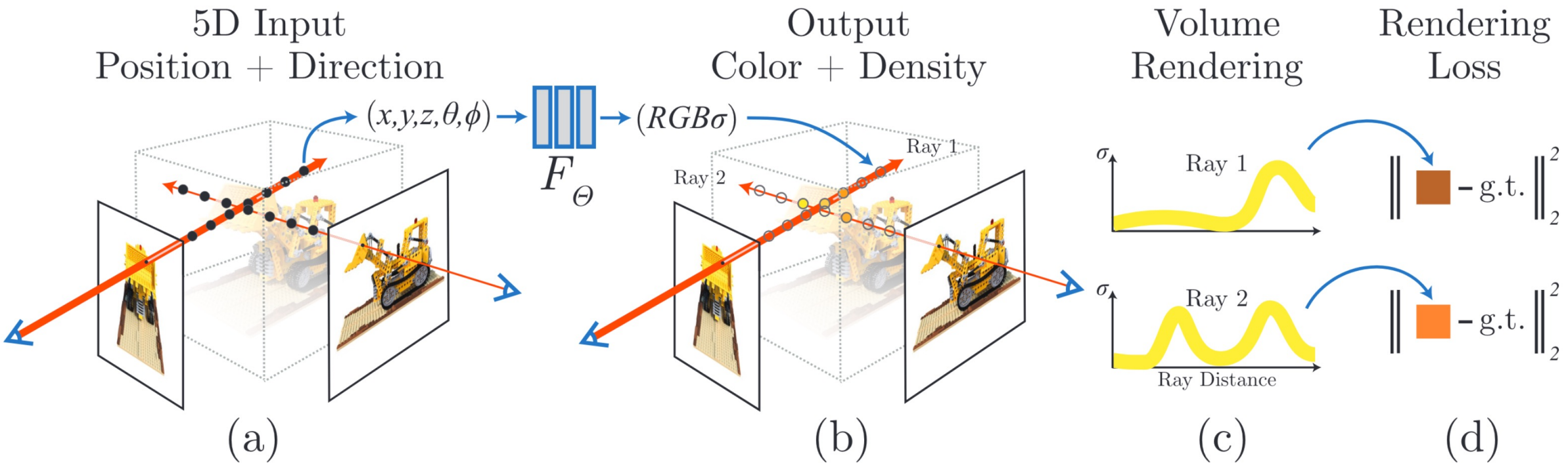


(r, g, b, σ)

Use a neural network to represent the scene

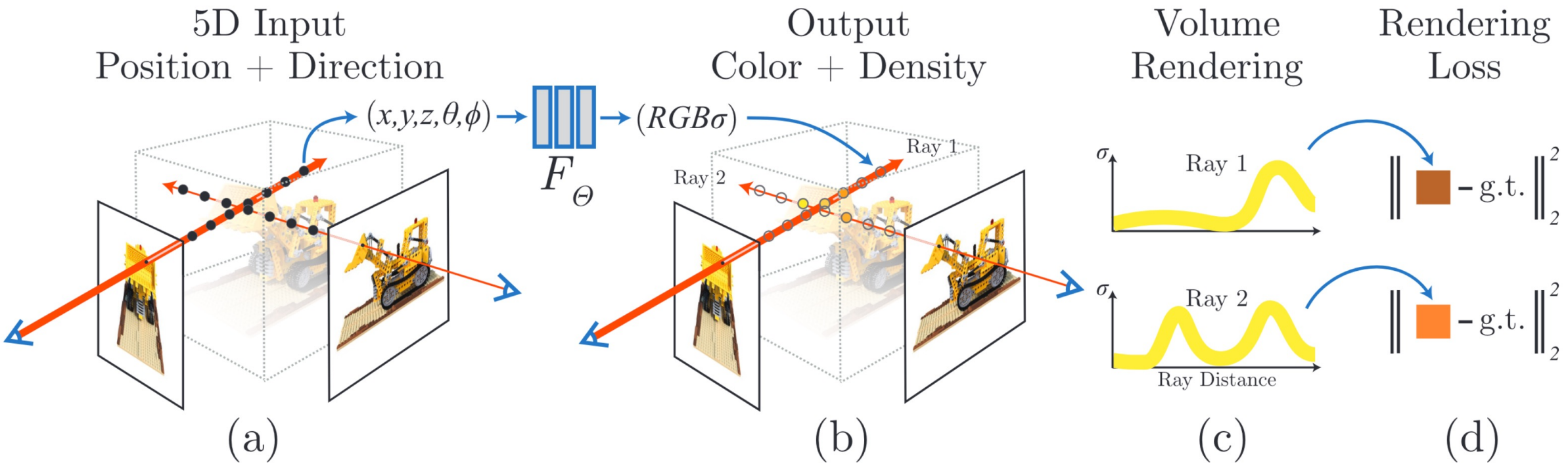
Basic idea

Overview



Basic idea

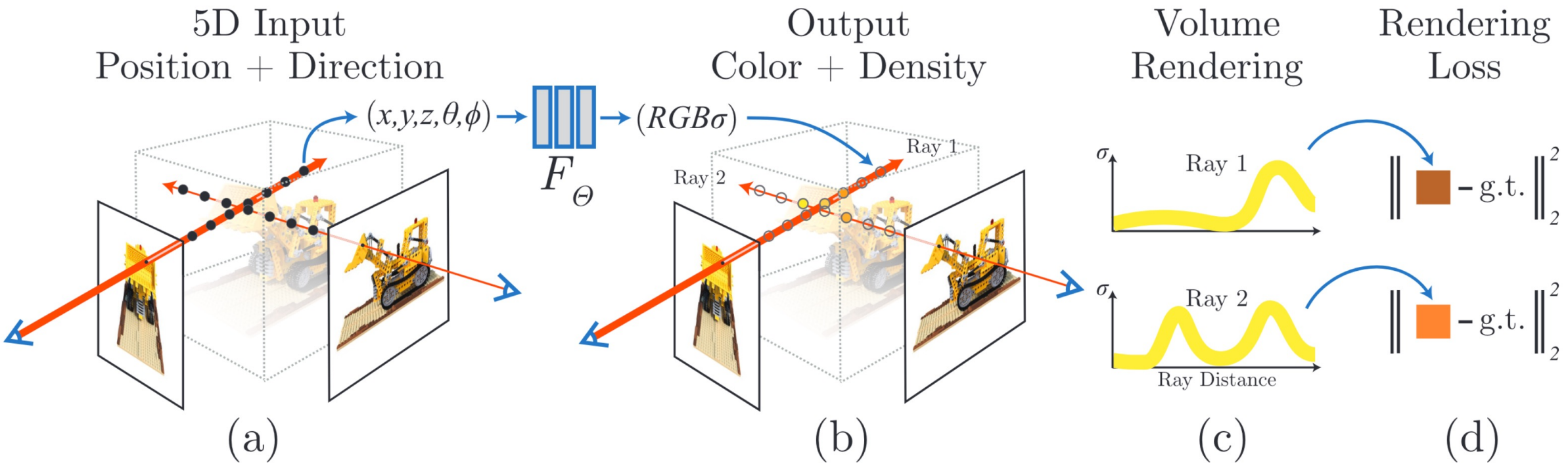
Overview



Sample along camera rays

Basic idea

Overview

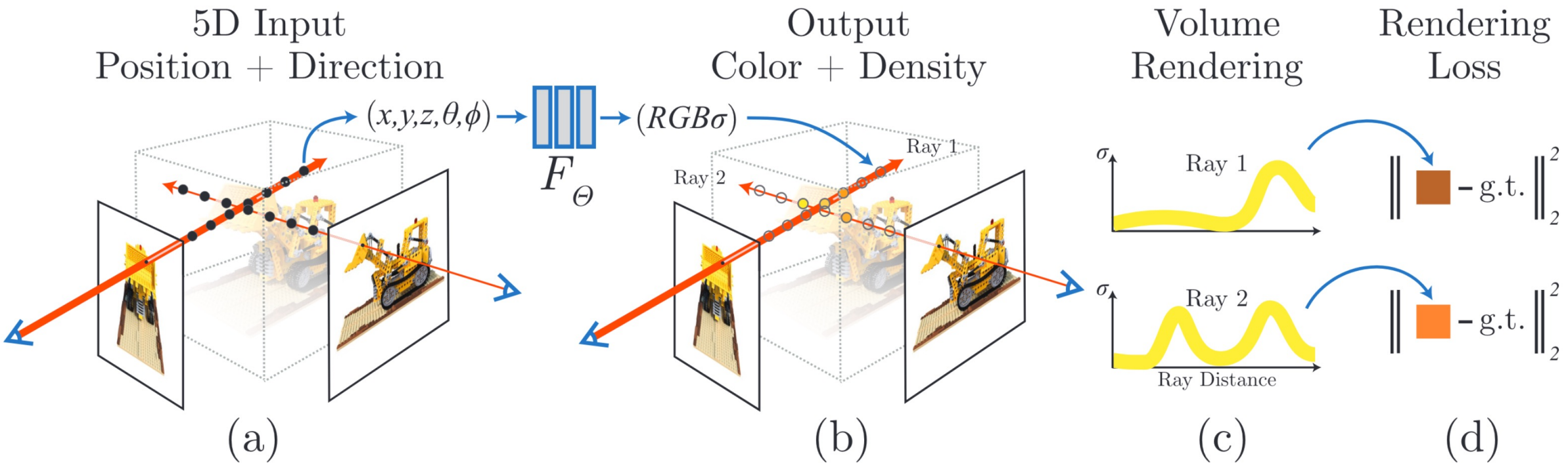


Sample along camera rays

Use MLP to predict color and volume density

Basic idea

Overview



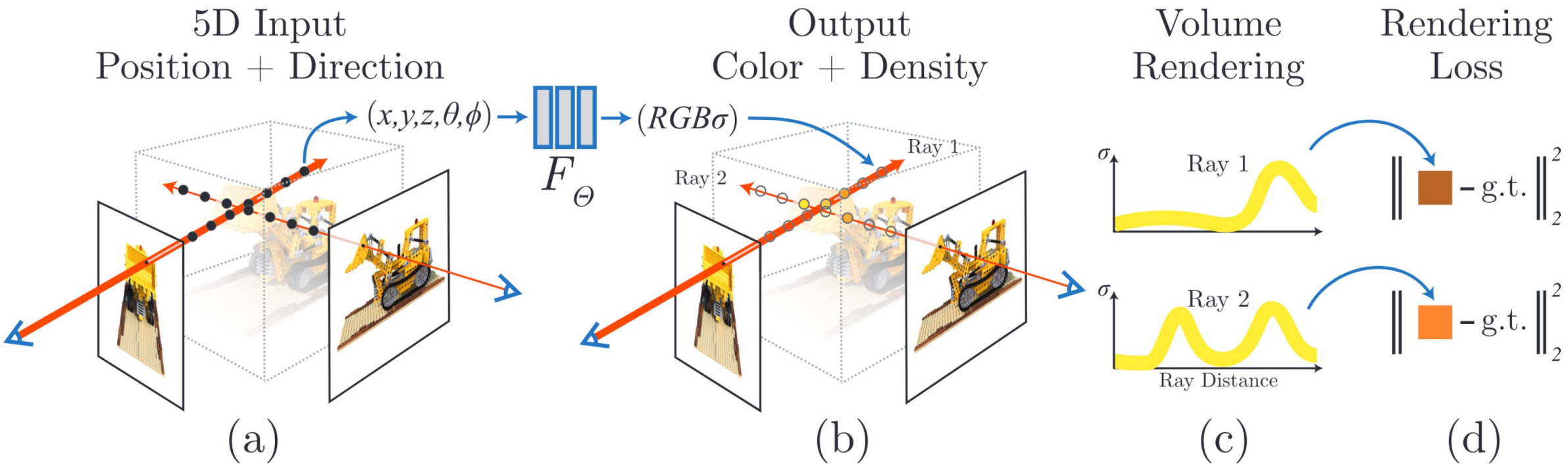
Sample along camera rays

Use MLP to predict color and volume density

Compose points along the ray into an image

Basic idea

Overview



Sample along camera rays

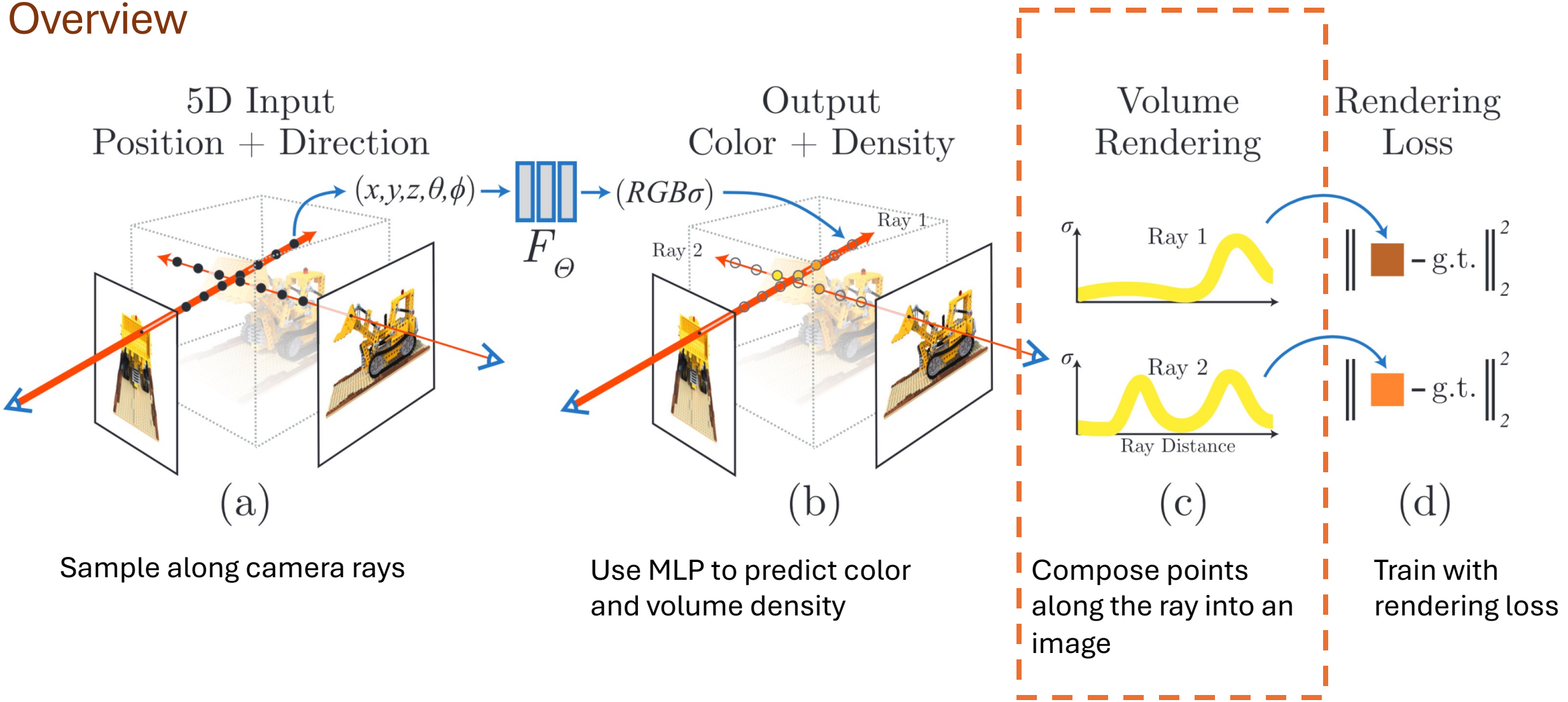
Use MLP to predict color and volume density

Compose points along the ray into an image

Train with rendering loss

Basic idea

Overview



Sample along camera rays

Use MLP to predict color and volume density

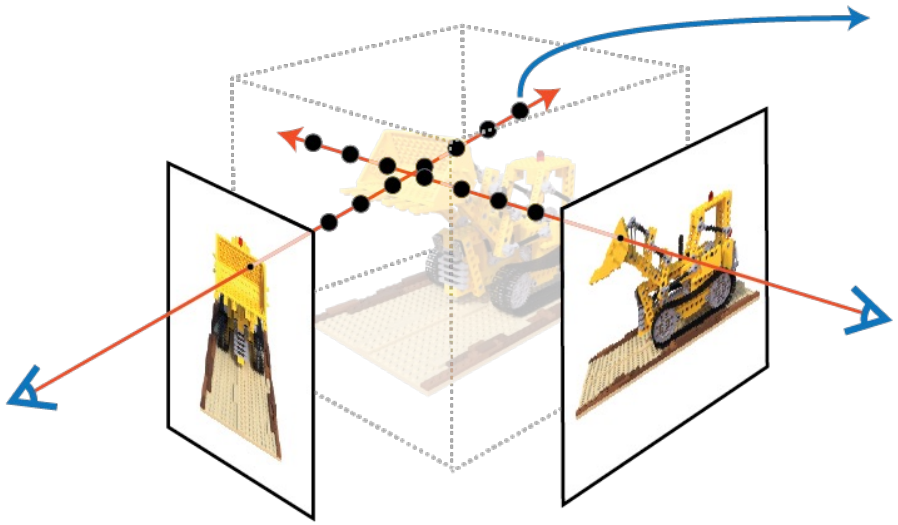
Compose points along the ray into an image

Train with rendering loss

Basic idea

Volume rendering

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t)\sigma(\mathbf{r}(t))\mathbf{c}(\mathbf{r}(t), \mathbf{d})dt\,, \text{ where } T(t) = \exp\left(-\int_{t_n}^{t} \sigma(\mathbf{r}(s))ds\right)$$

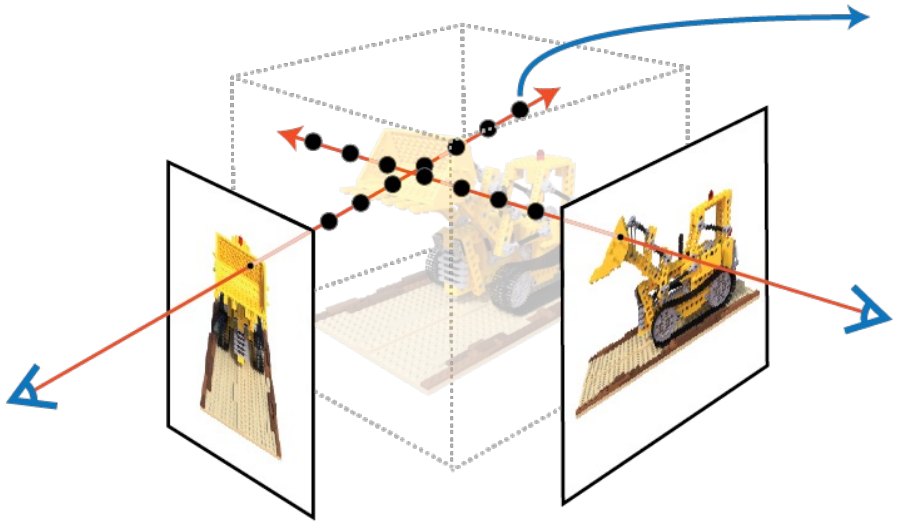


$\boldsymbol{r}(t) \rightarrow (x, y, z)$ The t-th point in ray $\boldsymbol{r}$

Basic idea

Volume rendering

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t)\sigma(\mathbf{r}(t))\mathbf{c}(\mathbf{r}(t), \mathbf{d})dt\,, \text{ where } T(t) = \exp\left(-\int_{t_n}^{t} \sigma(\mathbf{r}(s))ds\right)$$



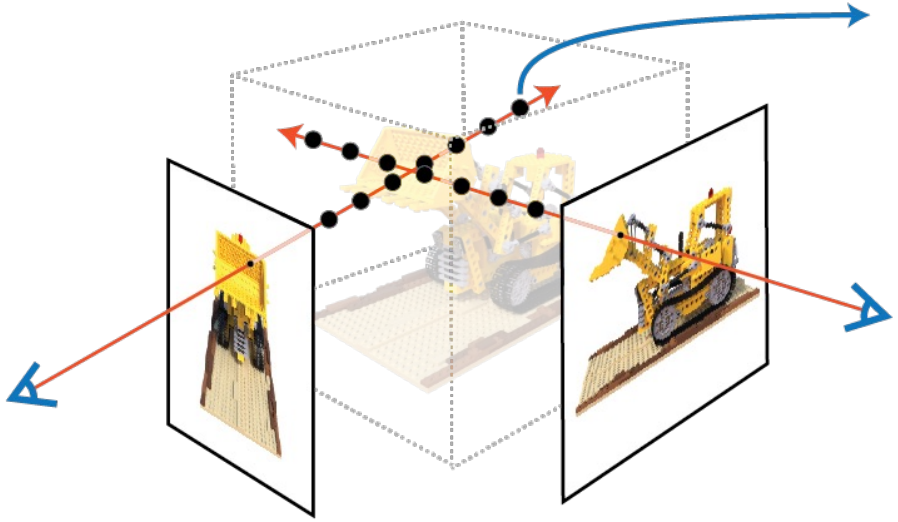
$\boldsymbol{r}(t) \rightarrow (x, y, z)$ The t-th point in ray $\boldsymbol{r}$

$\sigma(\boldsymbol{r}(t)) \rightarrow [0,1]$ The volume density

Basic idea

Volume rendering

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t)\sigma(\mathbf{r}(t))\mathbf{c}(\mathbf{r}(t), \mathbf{d})dt\,, \text{ where } T(t) = \exp\left(-\int_{t_n}^{t} \sigma(\mathbf{r}(s))ds\right)$$



$\boldsymbol{r}(t) \rightarrow (x, y, z)$ The t-th point in ray $\boldsymbol{r}$

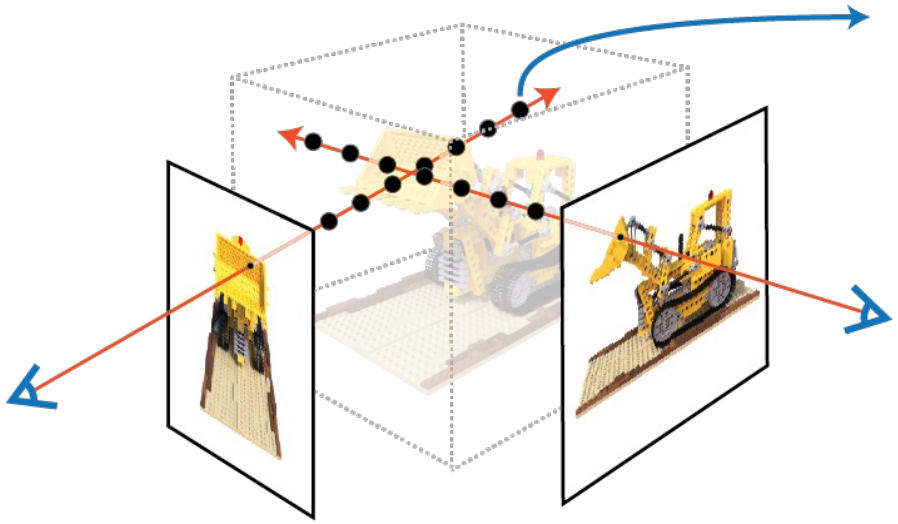
$\sigma(\boldsymbol{r}(t)) \rightarrow [0,1]$ The volume density

$\boldsymbol{c}(\boldsymbol{r}(t), d) \rightarrow (r, g, b)$ The color

Basic idea

Volume rendering

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t)\sigma(\mathbf{r}(t))\mathbf{c}(\mathbf{r}(t), \mathbf{d})dt\,, \quad \text{where } T(t) = \exp\left(-\int_{t_n}^{t} \sigma(\mathbf{r}(s))ds\right)$$



$\boldsymbol{r}(t) \rightarrow (x, y, z)$ The t-th point in ray $\boldsymbol{r}$

$\sigma(\boldsymbol{r}(t)) \rightarrow [0,1]$ The volume density

$\boldsymbol{c}(\boldsymbol{r}(t), d) \rightarrow (r, g, b)$ The color

$T(t) \rightarrow [1, 0]$ The probability that the ray travels without hitting any other particle

3D Gaussian Splatting





Real-time Rendering



100+ FPS





Why 3DGS

Point Cloud / Voxel: Discrete & easy to edit, but visually sparse (points) or memory-heavy (voxels).

Mesh: Continuous surface, but fixed topology makes gradient-based optimization difficult.

SDF (Signed Distance Field): Flexible topology, but struggles to capture high-frequency textures and complex details.

NeRF: High quality rendering, but painfully slow due to dense ray sampling.

Why 3DGS

Point Cloud / Voxel: Discrete & easy to edit, but visually sparse (points) or memory-heavy (voxels).

Mesh: Continuous surface, but fixed topology makes gradient-based optimization difficult.

SDF (Signed Distance Field): Flexible topology, but struggles to capture high-frequency textures and complex details.

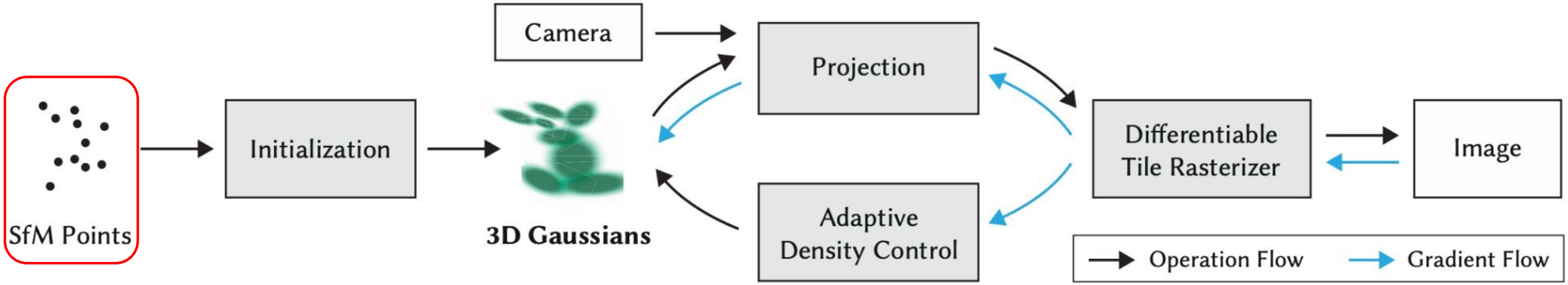
NeRF: High quality rendering, but painfully slow due to dense ray sampling.

3DGS:

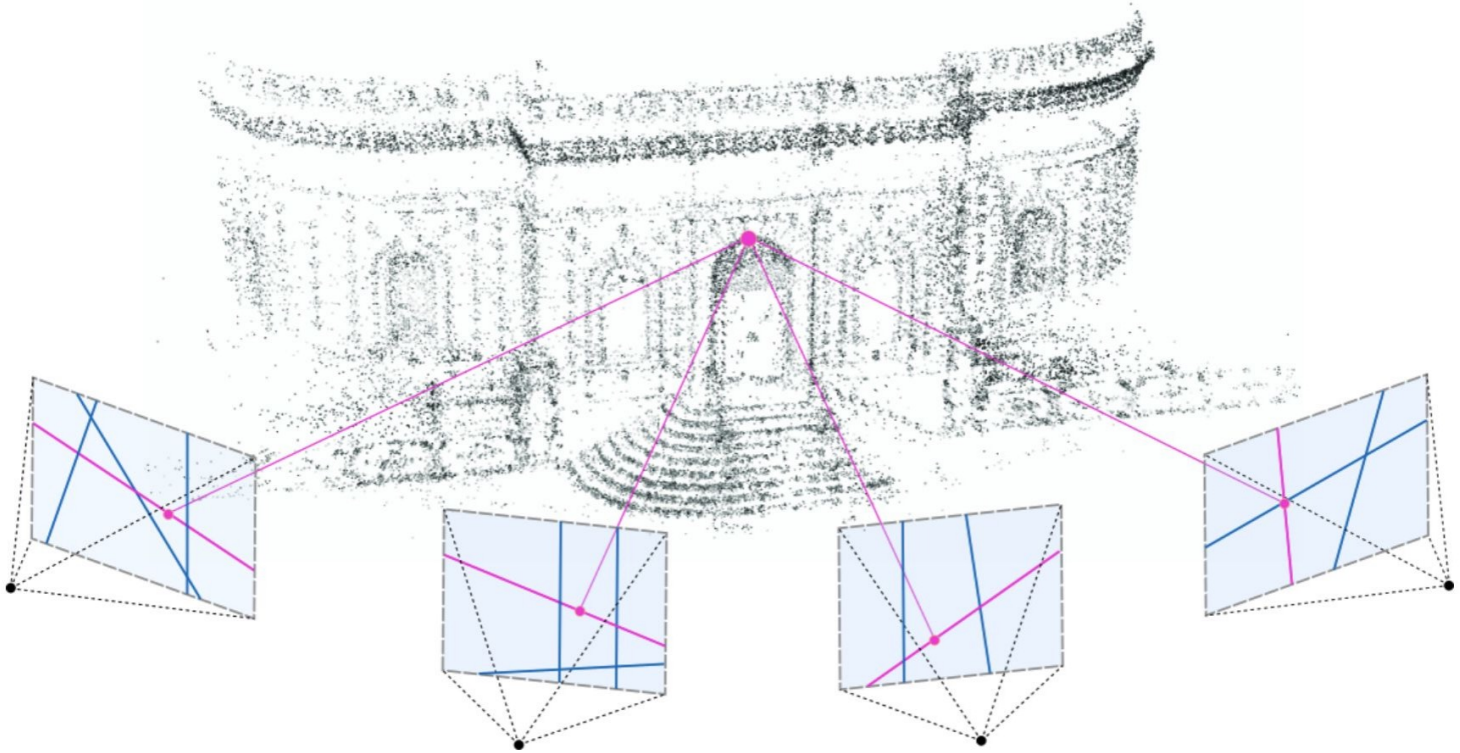
- Real-Time High-Fidelity Rendering—100+ FPS.
- Explicit Representation—flexible and easy to update.
- Fully Differentiable Pipeline—Highly optimizable.

3DGS Pipeline

Structure-from-Motion



Structure-from-Motion



Goal: Before training 3D Gaussians, we need to estimate both 3D structure (sparse points) and camera paramaters (K, R, t) from a set of 2D images.

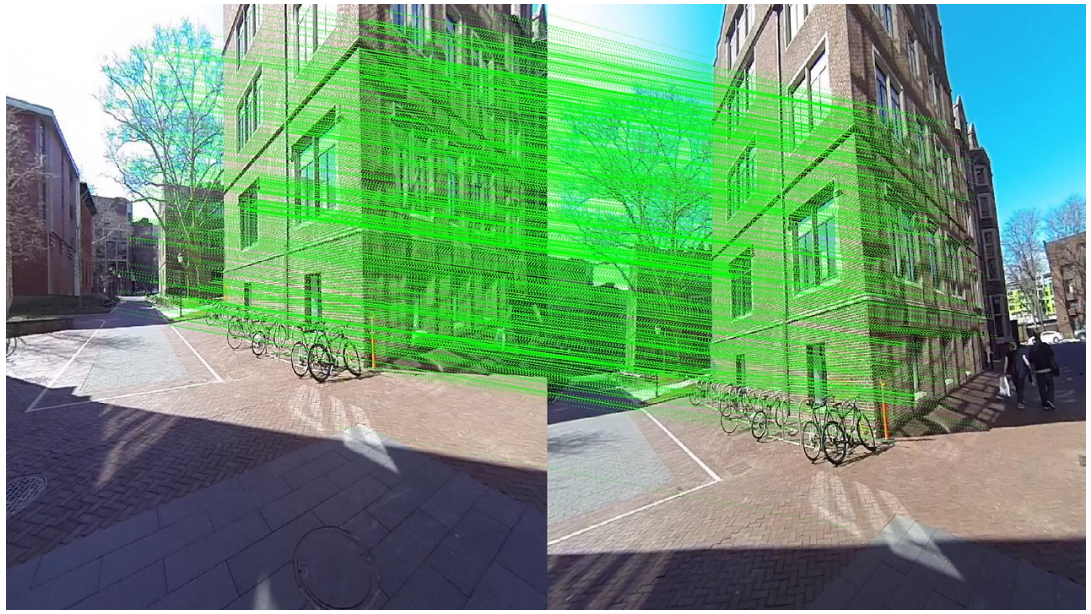
A classic yet complex algorithm. For more details, please refer to:

Structure-from-Motion Revisited

COLMAP Official Documentation

Structure-from-Motion

Feature Matching



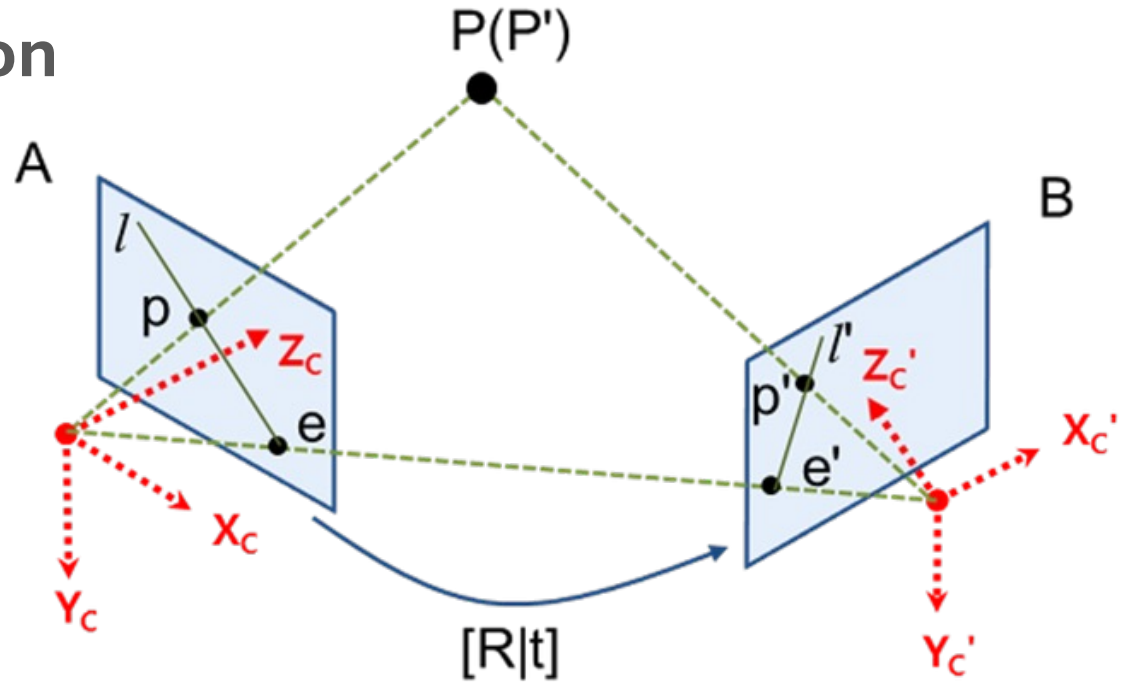
Extraction: Detect robust keypoints (e.g., SIFT,). Needs textured surfaces.

Matching: Pair points based on visual similarity.

Problem: Generates many false matches (Outliers).

Structure-from-Motion

Geometric Verification



Goal: Filter out false matches (Outliers) using mathematical constraints.

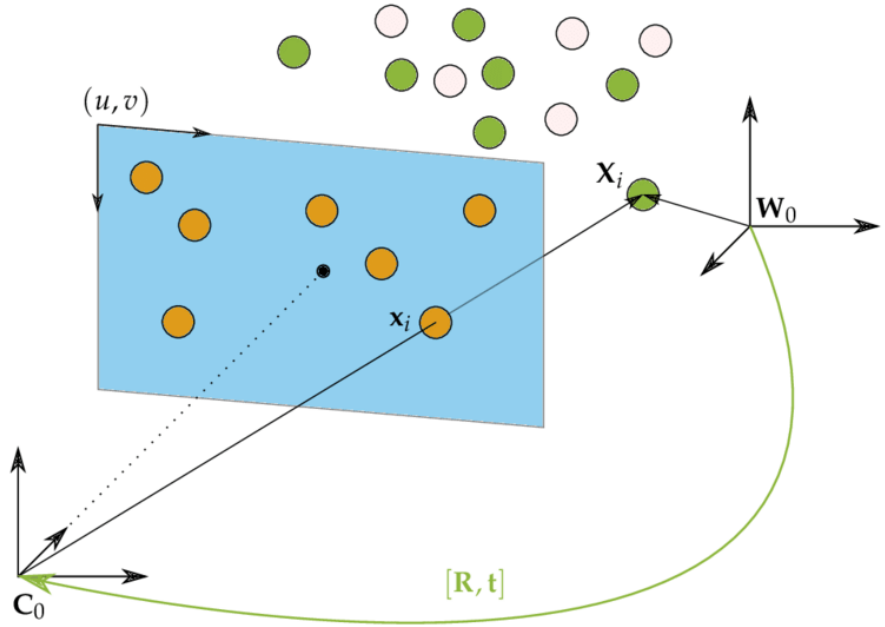
The Rule: Epipolar Geometry——physical points must satisfy 3D spatial constraints.

The Filter: RANSAC Algorithm——iteratively finds the mathematically correct matches.

Result: Keeps only the true matches (Inliers).

Structure-from-Motion

Image Registration



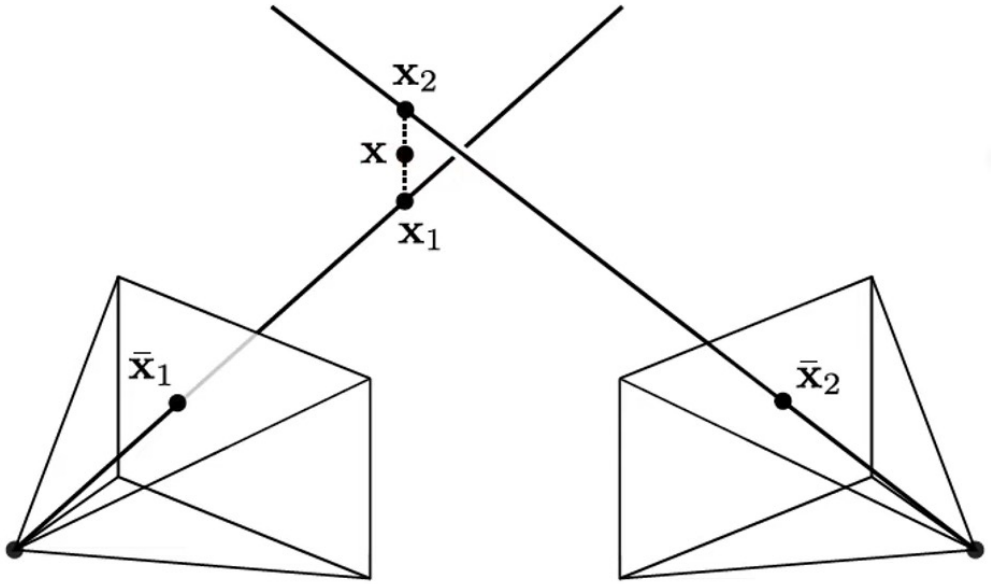
Initialization: Start with a good 2-image pair to build the initial 3D map.

Process (New Image Registration): Use known 3D points and their corresponding 2D pixel locations to calculate the new camera's pose.

Algorithm: PnP (Perspective-n-Point).

Structure-from-Motion

Triangulation



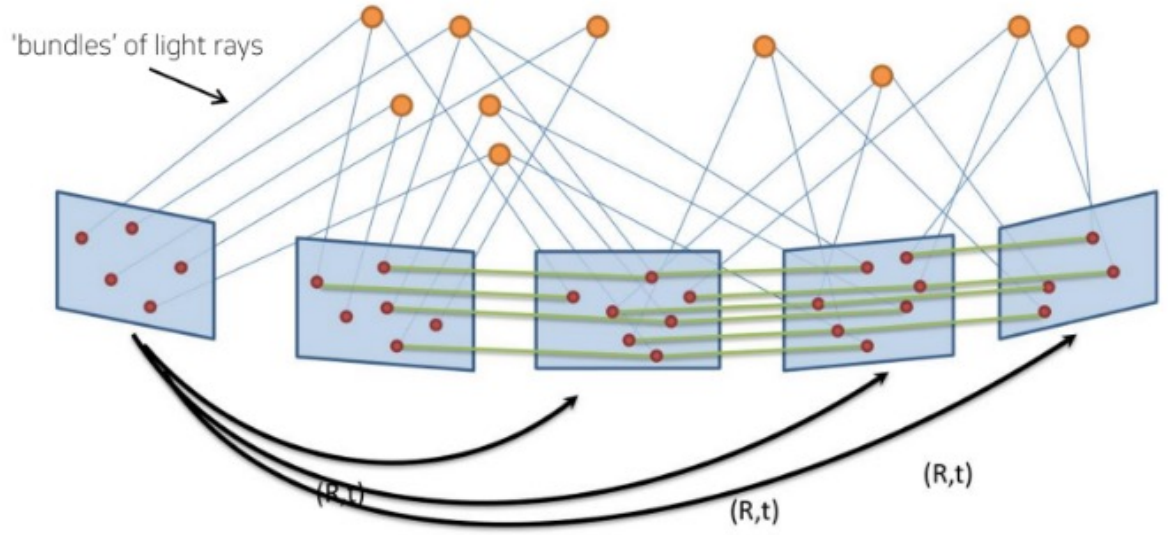
Goal: Determine the exact 3D coordinates of the matched features.

Mechanism: Intersect visual rays originating from multiple known camera positions.

Reality: Due to noise, rays rarely intersect perfectly, requiring mathematical approximation to find the closest point.

Structure-from-Motion

Bundle Adjustment (BA)



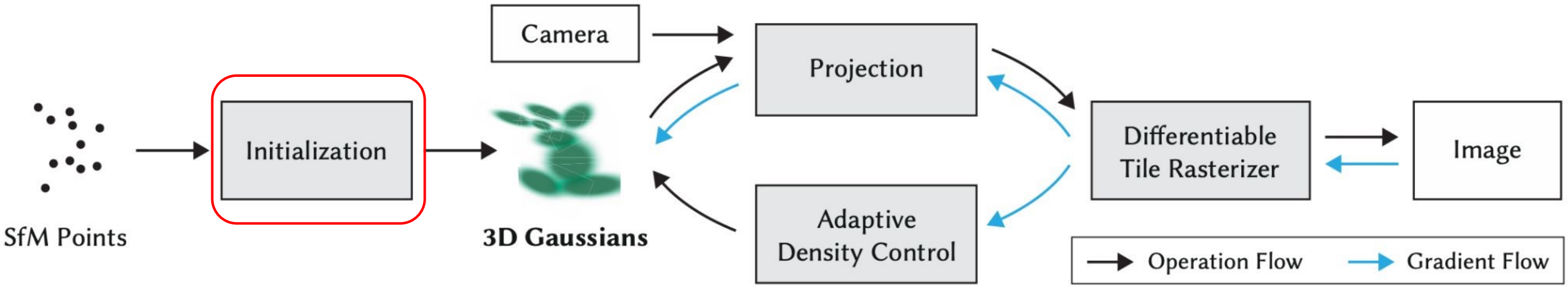
Goal: Jointly refines ALL camera poses and 3D point positions simultaneously.

Objective: Minimize the Reprojection Error (Distance between observed 2D pixels vs. projected 3D points).

Final Output: Camera Poses + Sparse Point Cloud (The starting point for 3DGS!)

3DGS Pipeline

Initial Spare Point Clouds



Initial Spare Point Clouds

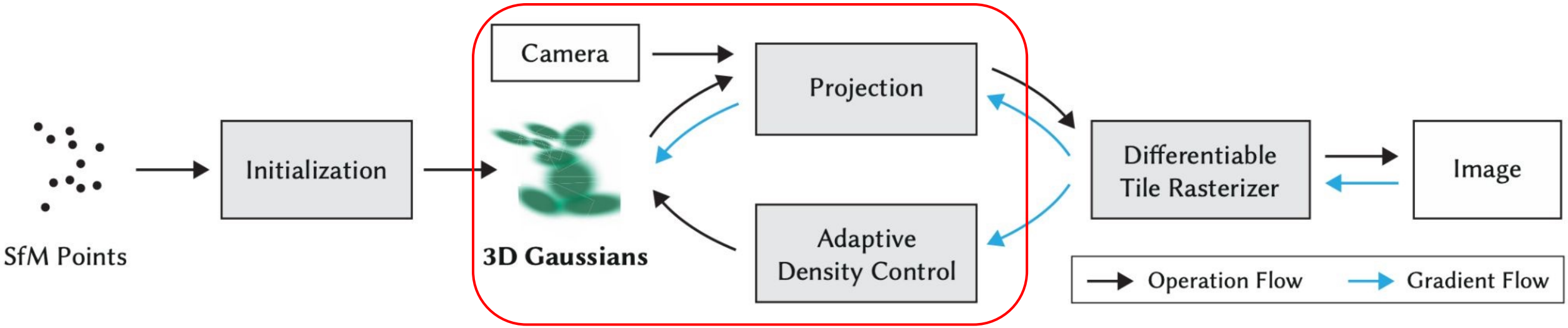




SfM sparse point cloud acts as the initial "skeleton" of the scene.

Training Pipeline

3D Gaussian Optimization

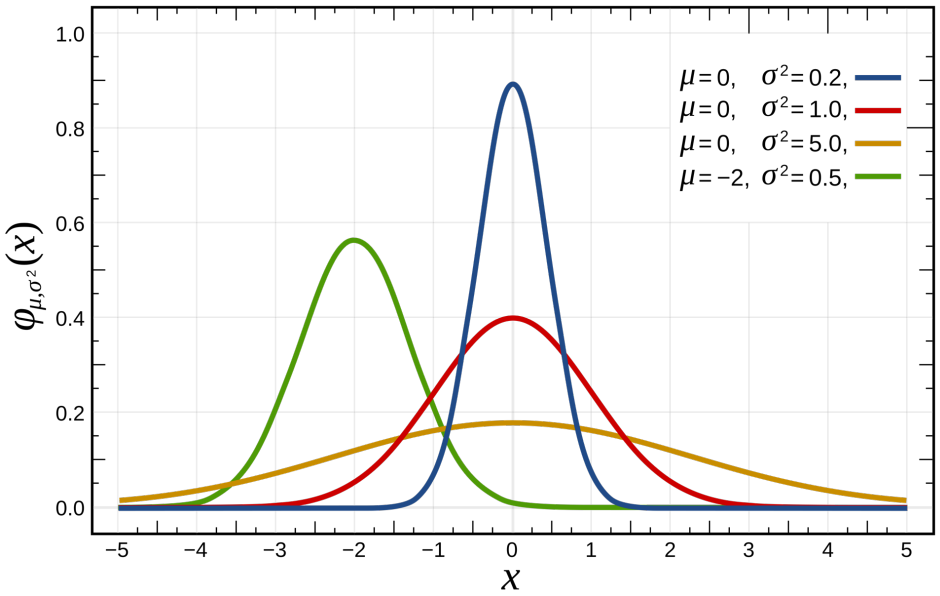


3D Gaussian Primitives



Final Scene Composed of Millions of 3D Gaussian Primitives

Fundamentals: 1D Gaussians



- Controlled by **Mean** (μ): Where is the center?
- Controlled by **Variance** (σ^2): How wide is the curve?
- Formula:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

3D Gaussians—Geometry

3D Gaussian Distribution (The Ellipsoid):

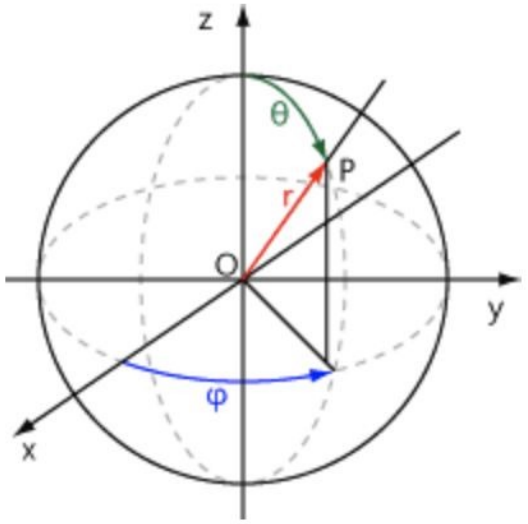
- Controlled by **3D Mean Vector** (μ): Position in (X, Y, Z).
- Controlled by **3×3 Covariance Matrix** (Σ): Defines the 3D volume, shape, and orientation.
- Formula:

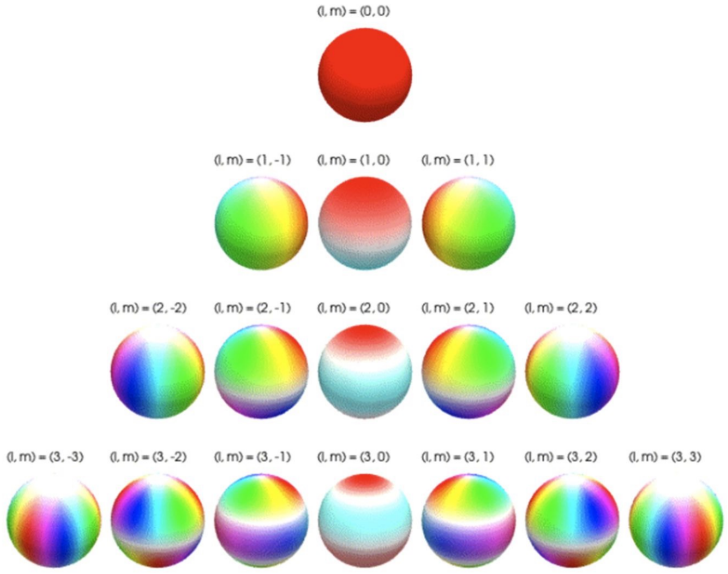
$$G(x) = e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)}$$

Modeling the scene as a continuous 3D Density Field.

3D Gaussians—Opacity and Color

- Opacity α: Transparency level $[0,1]$
- Color: Spherical Harmonics (SH) coefficients Spherical Harmonic Lighting

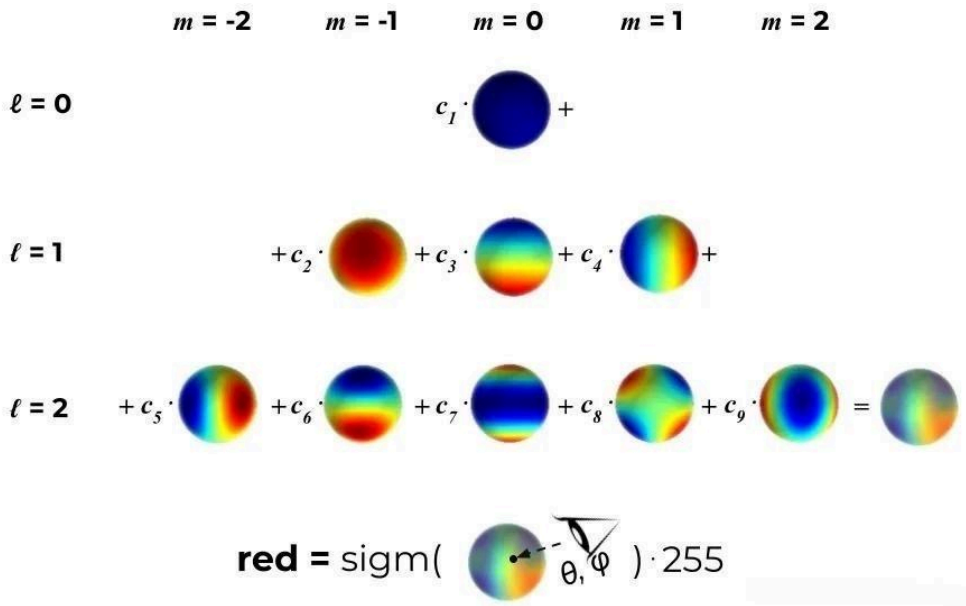




Why not just use simple RGB?

- **The Assumption:** If a 3D Gaussian only stores one fixed (R,G,B) value, its color looks exactly the same from every angle.
- **The Reality:** Real-world objects have highlights and reflections! The color of a specific point changes depending on your Viewing Direction.
- **The Goal:** We need a mathematical function that takes the camera's viewing direction as input, and outputs the correct RGB color for that specific angle.

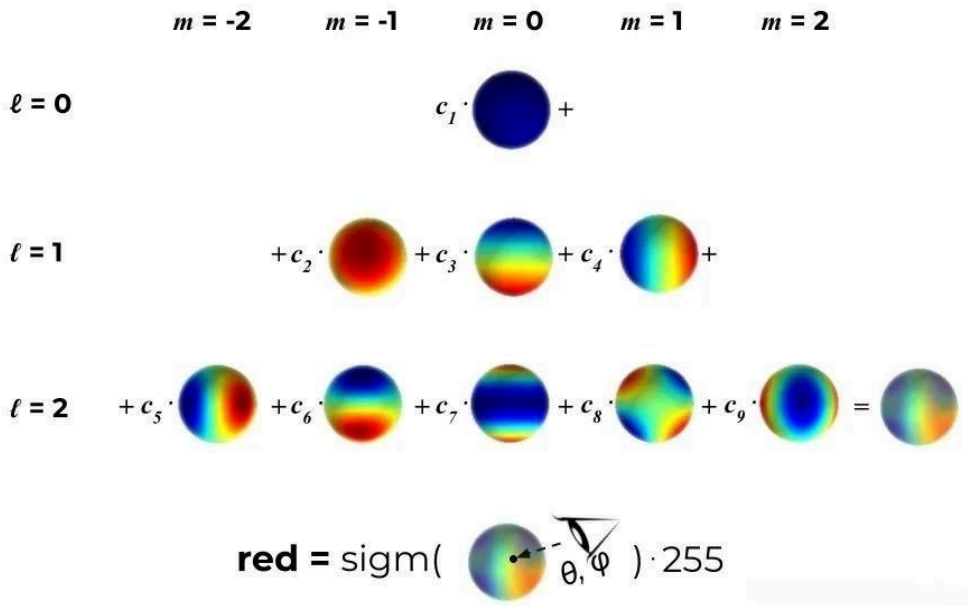
Spherical Harmonics



Spherical Harmonics decomposes a complex 3D spherical signal (lighting) into a set of basic 3D patterns (Basis Functions).

How to get color: Any complex view-dependent color can be approximated by summing up these base patterns, each multiplied by a Coefficient.

Spherical Harmonics



What we optimize: For each 3D Gaussian, the optimizer learns the SH Coefficients for Red, Green, and Blue.

3DGS typically uses up to Degree 3 (requiring 16 coefficients per color channel = 48 numbers total per Gaussian) to achieve photorealistic reflections.

Optimization: The Covariance Problem

The Challenge: How do you optimize a covariance matrix (Σ) using Gradient Descent?

The Mathematical Trap:

- A valid Σ must be **Positive Semi-Definite**.
- Not all symmetric matrices are valid covariance matrices.
- Blindly applying gradient updates (e.g., $\Sigma_{new} = \Sigma_{old} - learning_rate \times gradient$) will quickly generate invalid matrices, causing the program to crash!

Optimization: The Covariance Problem

The Elegant Solution: Decomposition

$$\Sigma = RSS^T R^T$$

For any valid Scale vector (S) and Rotation (R), this formula mathematically guarantees a valid covariance matrix.

Optimization: The Covariance Problem

The Elegant Solution: Decomposition

$$\Sigma = RSS^T R^T$$

For any valid Scale vector (S) and Rotation (R), this formula mathematically guarantees a valid covariance matrix.

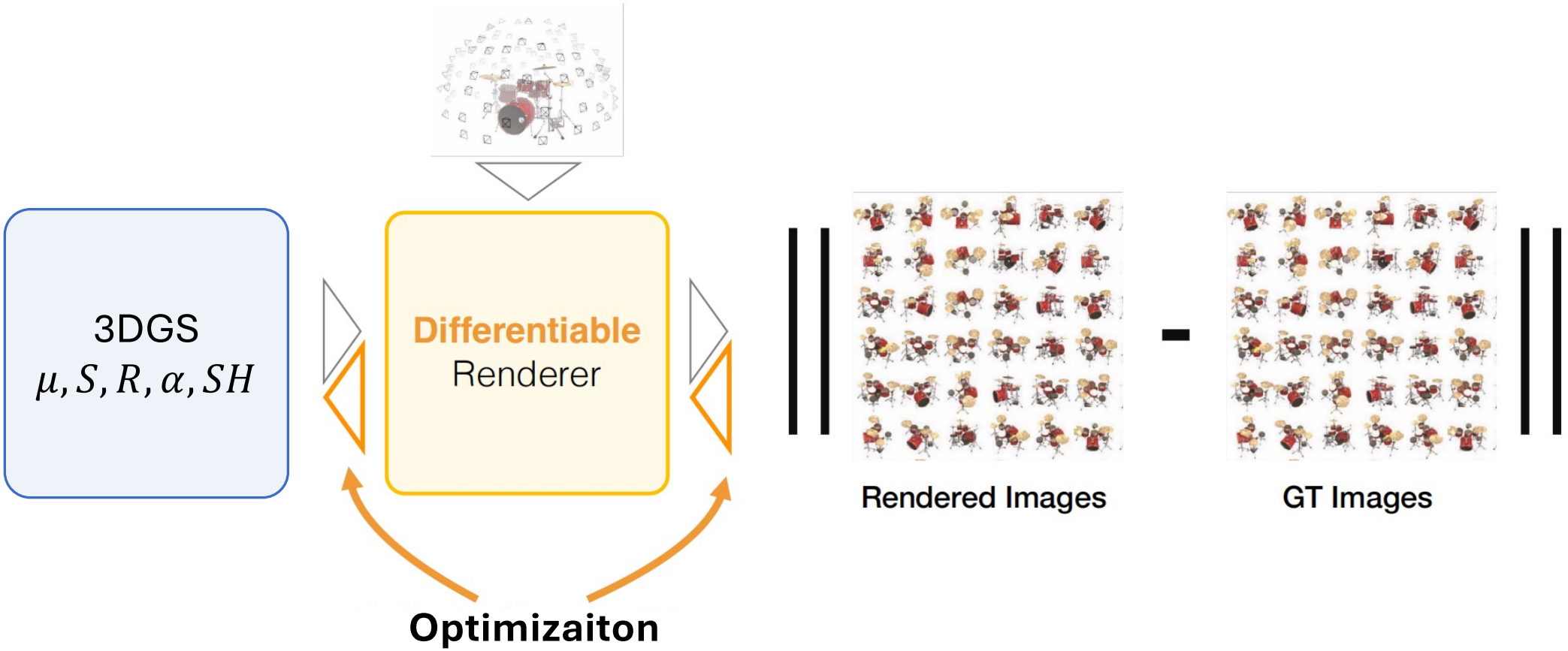
Implementation Detail: Rotation matrices (R) are hard to optimize directly (Gimbal lock). Therefore, 3DGS uses Quaternions to represent and optimize rotation.

3D Gaussian Optimization

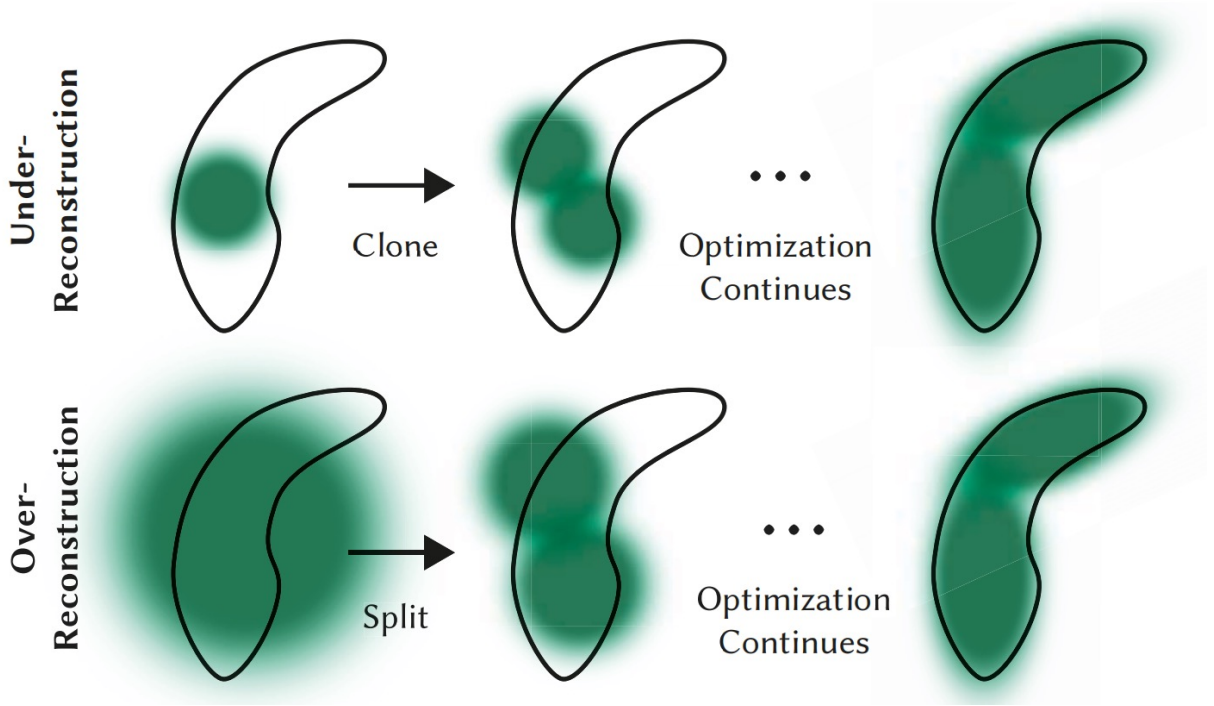
- Position (mean): μ
- Covariance: Σ (by S and R)
- Opacity: α
- Color: Spherical Harmonics (SH) coefficients

→ Gradient Descent

3D Gaussian Optimization

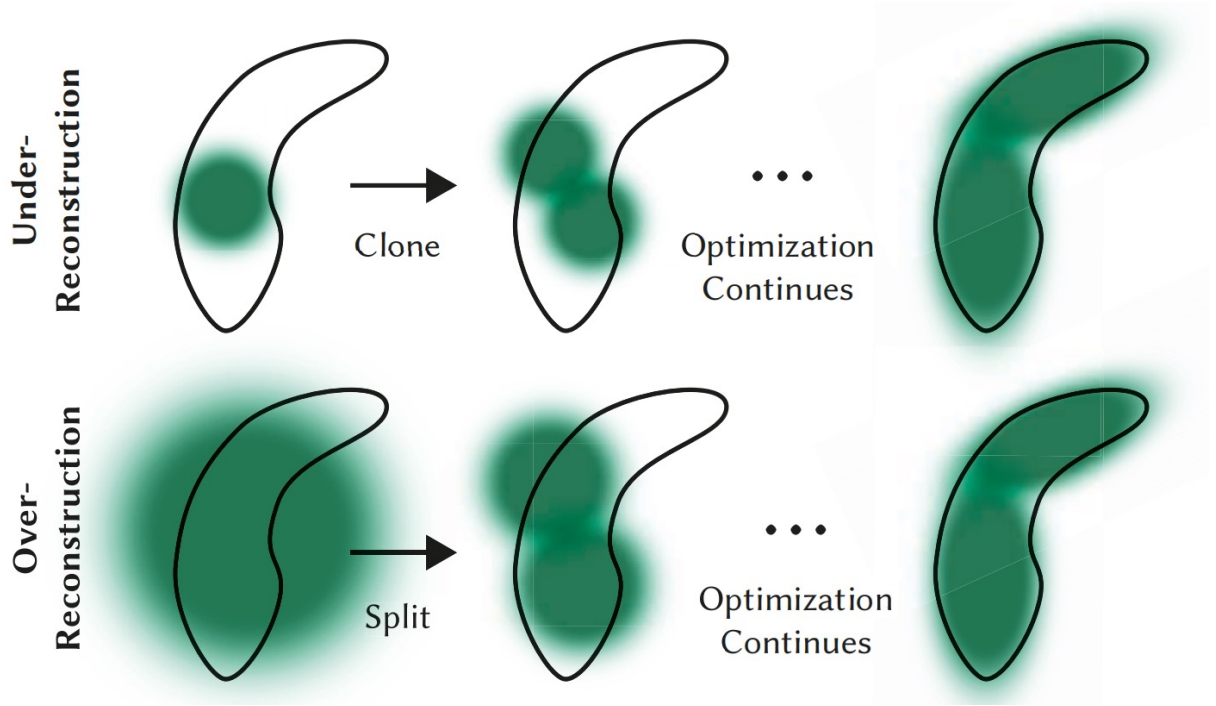


Adaptive Control of Gaussians—Densification



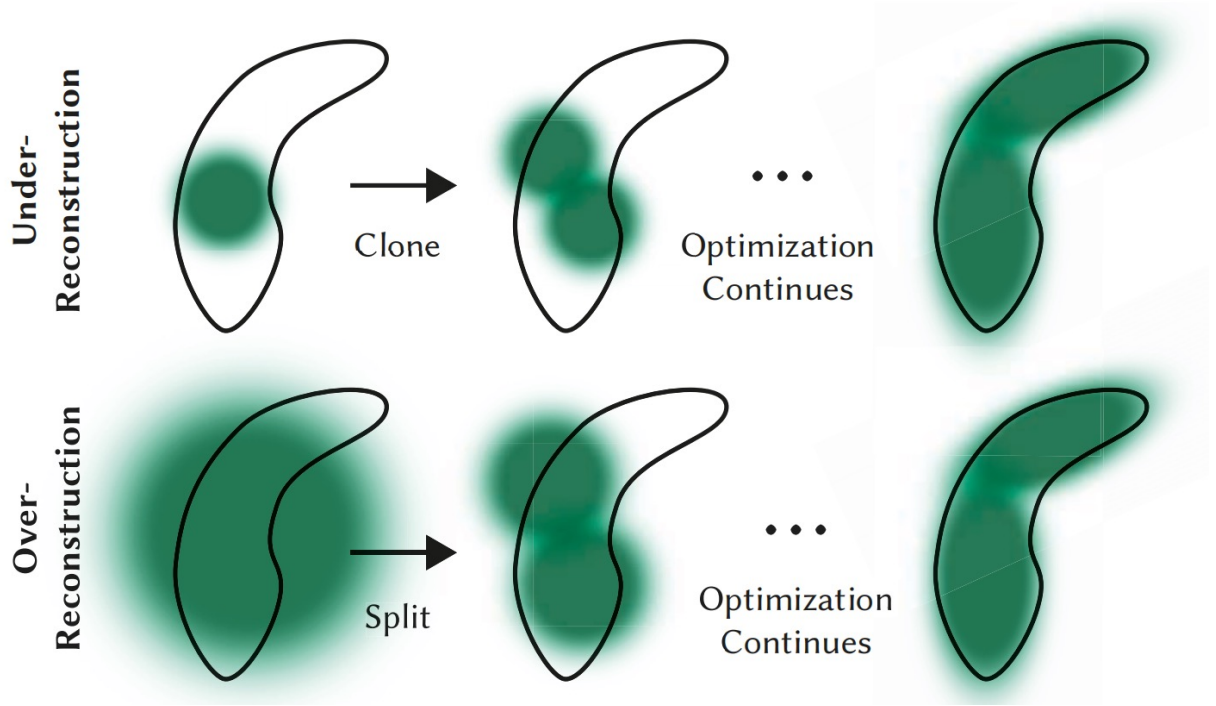
- **The Goal:** Increase the number of points where necessary.

Adaptive Control of Gaussians—Densification



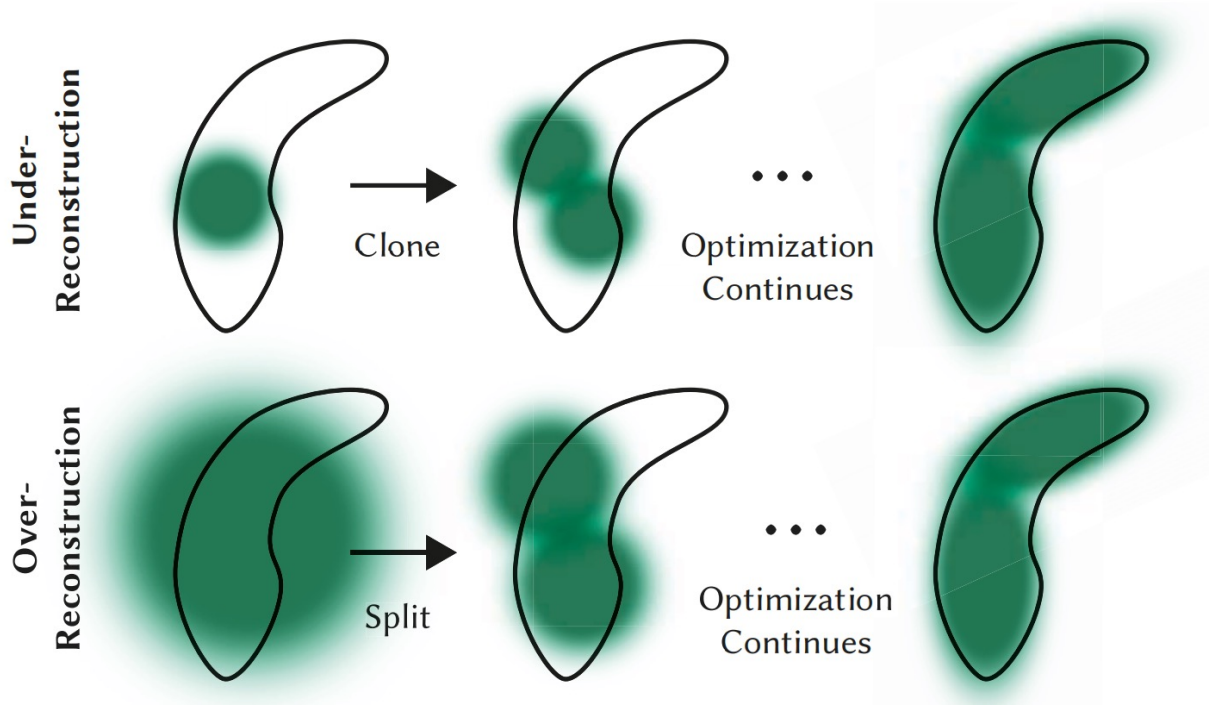
- **The Goal:** Increase the number of points where necessary.
- **The Indicator:** Points with high positional gradients.

Adaptive Control of Gaussians—Densification



- **The Goal:** Increase the number of points where necessary.
- **The Indicator:** Points with high positional gradients.
- **The Meaning:** High gradients correspond to regions that are not well reconstructed yet (the optimizer is struggling to fit the geometry).

Adaptive Control of Gaussians—Action



- **Clone** 3D gaussians: Under-reconstruction (missing geometric features)
- **Split** 3D gaussians: Over-reconstruction (covering large area)
- **Remove** 3D gaussians: Opacity is lower than threshold

Ablation Study



No densification/adaptive control



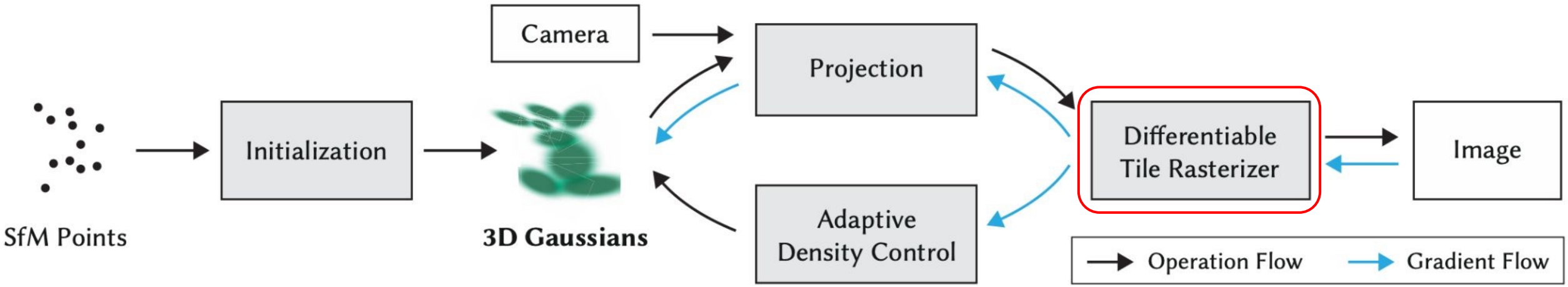


Full run

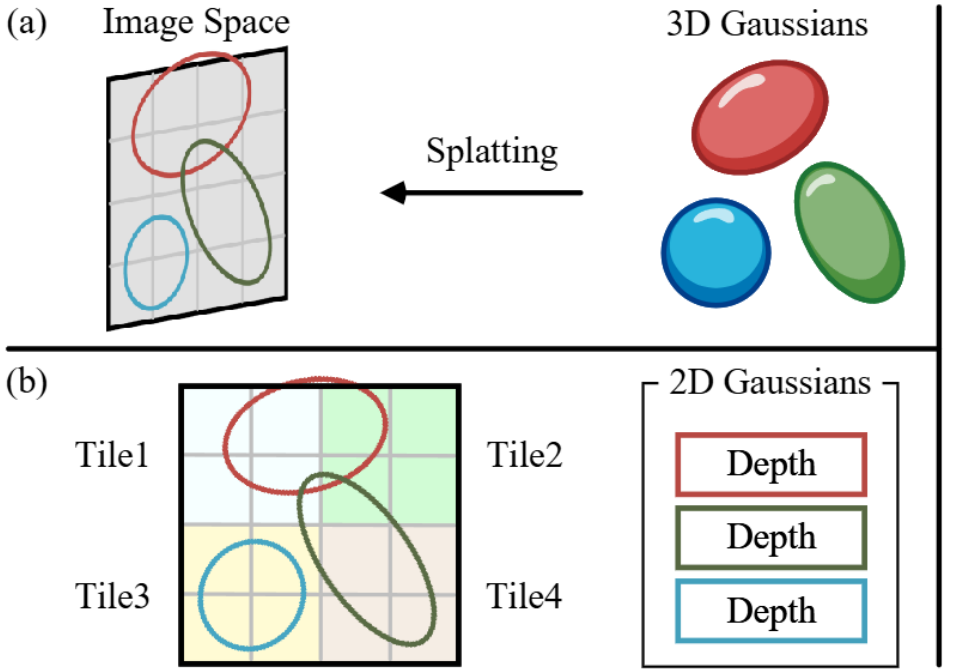


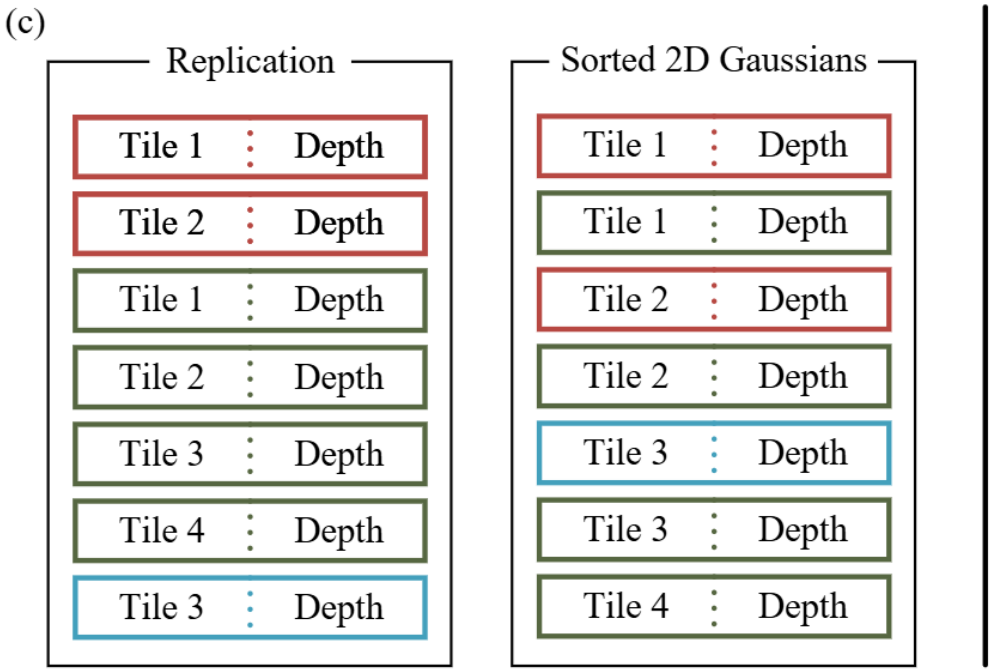
3DGS Pipeline

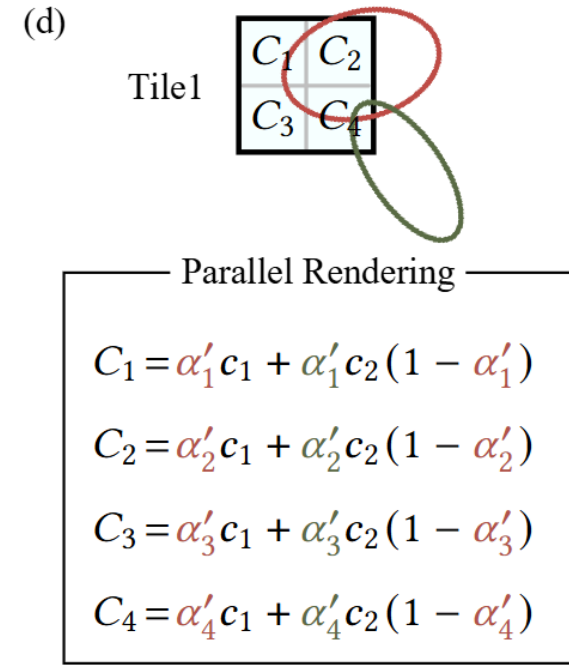
Fast Forward Rendering (Tile-based Rasterization)



Forward Rendering Process

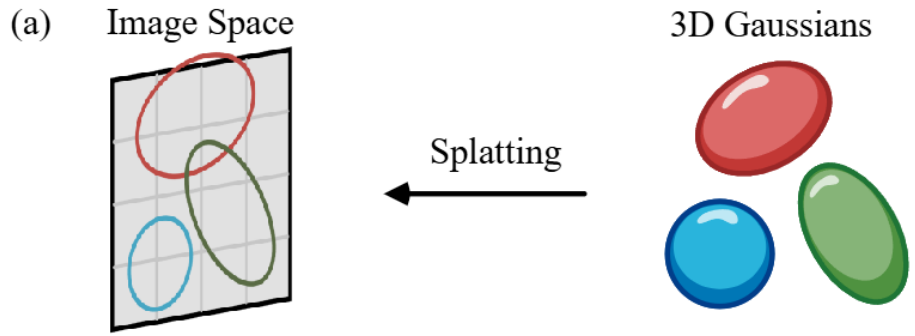


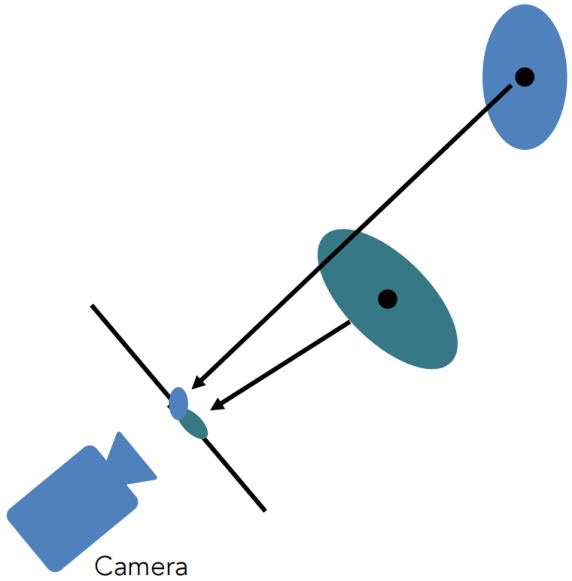




- **(a) Splatting (3D → 2D):** Project 3D Gaussians onto the 2D image plane.
- **(b) Tiling:** Divide the screen into non-overlapping patches (e.g., 16x16 pixel tiles).
- **(c) Replication & Sorting:** Duplicate Gaussians that overlap multiple tiles and sort them from front-to-back based on depth.
- **(d) Parallel Rendering (Alpha Blending):** Blend colors in depth order.

What is "Splatting"?

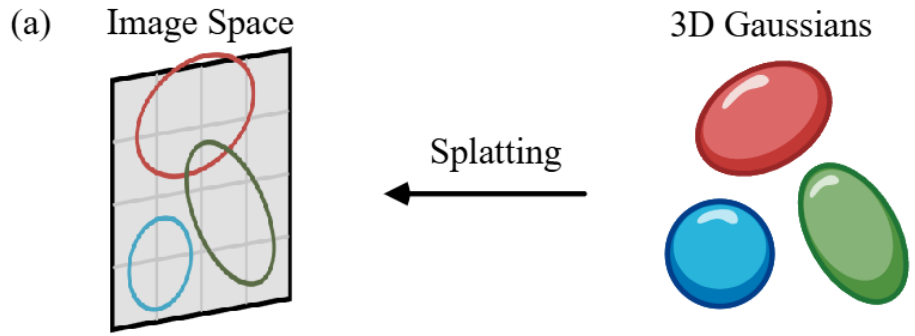


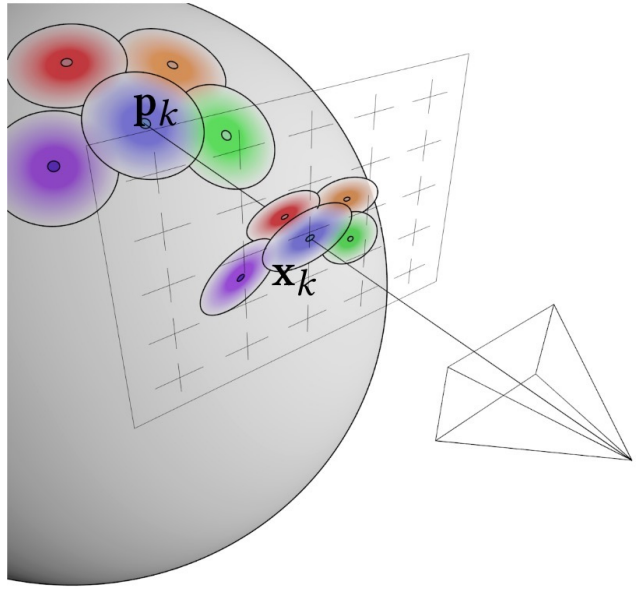


The Analogy (Throwing a Snowball):

- Imagine a 3D Gaussian as a soft, colored snowball.
- The image plane (screen) is a glass window.
- "Splatting" is the process of throwing that snowball at the window.

What is "Splatting"?

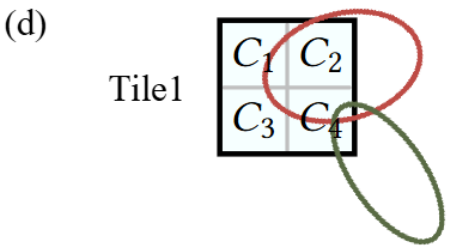




The Result: The 3D sphere flattens into a 2D colored splat (an ellipse) on the glass.

Goal: Calculate the exact 2D shape, position, and bounding box of every 3D snowball when it hits the screen.

Alpha Blending (Color Accumulation)



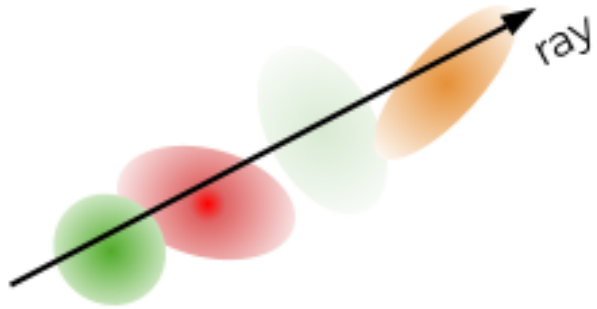
Parallel Rendering

$$C_1 = \alpha'_1 c_1 + \alpha'_1 c_2 (1 - \alpha'_1)$$

$$C_2 = \alpha'_2 c_1 + \alpha'_2 c_2 (1 - \alpha'_2)$$

$$C_3 = \alpha'_3 c_1 + \alpha'_3 c_2 (1 - \alpha'_3)$$

$$C_4 = \alpha'_4 c_1 + \alpha'_4 c_2 (1 - \alpha'_4)$$



Accumulate color pixel by pixel using the sorted 2D Gaussians:

$$C = \sum_{i \in \mathcal{N}} c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j)$$

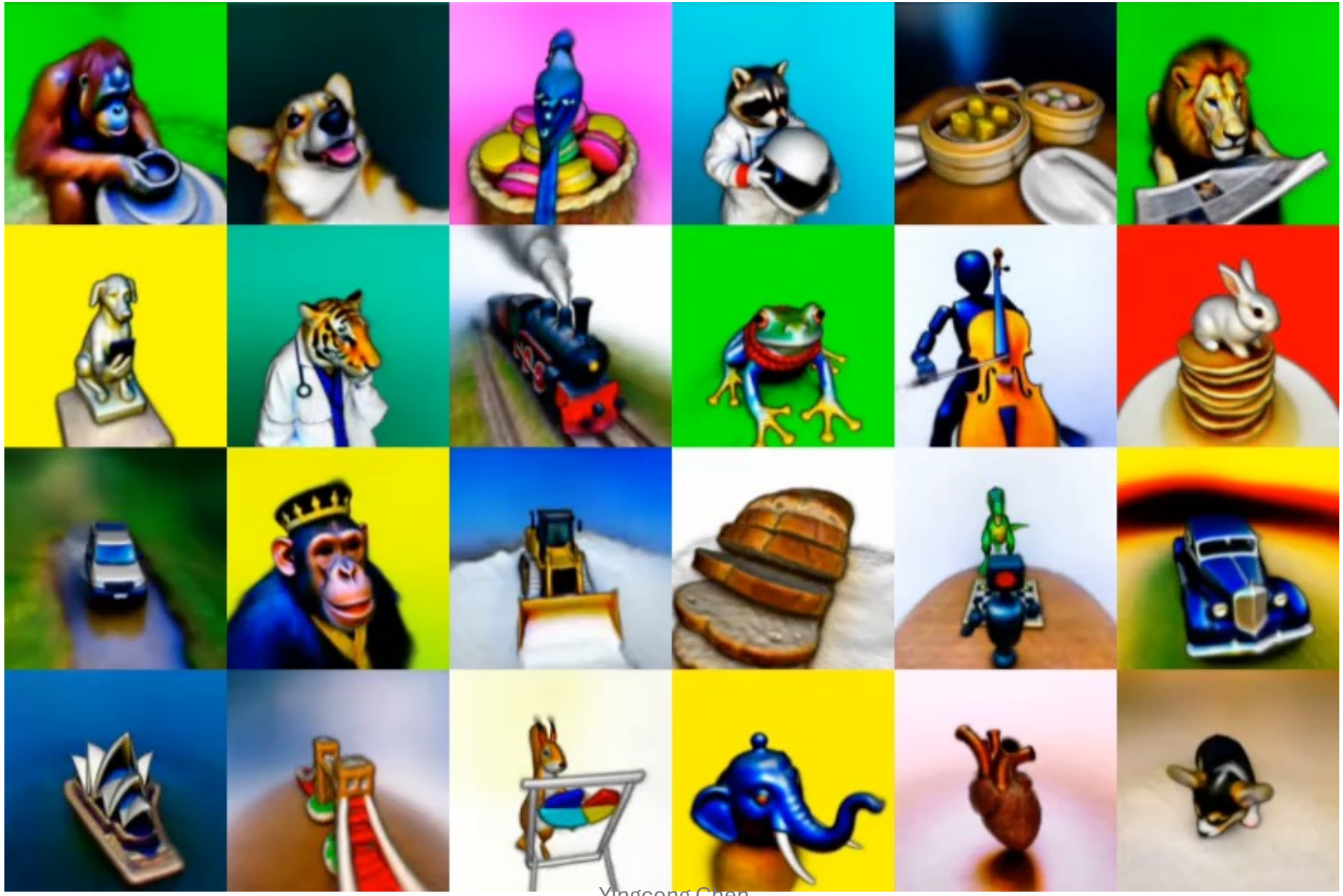
The Math Translated: Final Color = (Current Color × Opacity) × (Light blocked by objects in front).



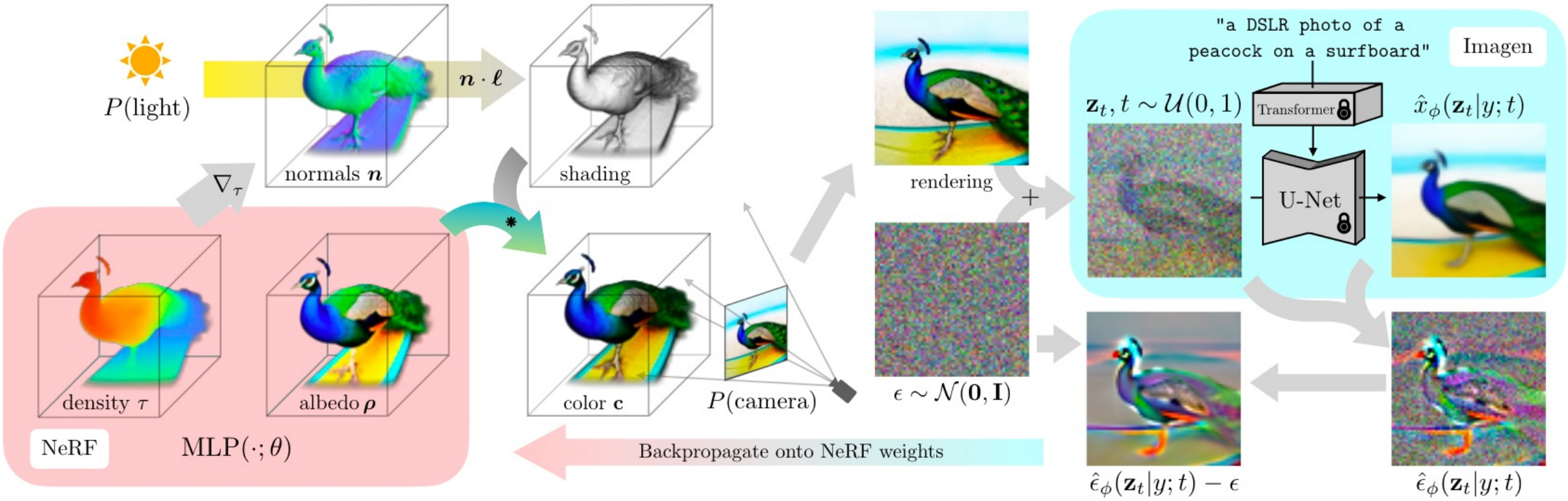
https://antimatter15.com/splat/

When Generation Meets 3D Reconstruction

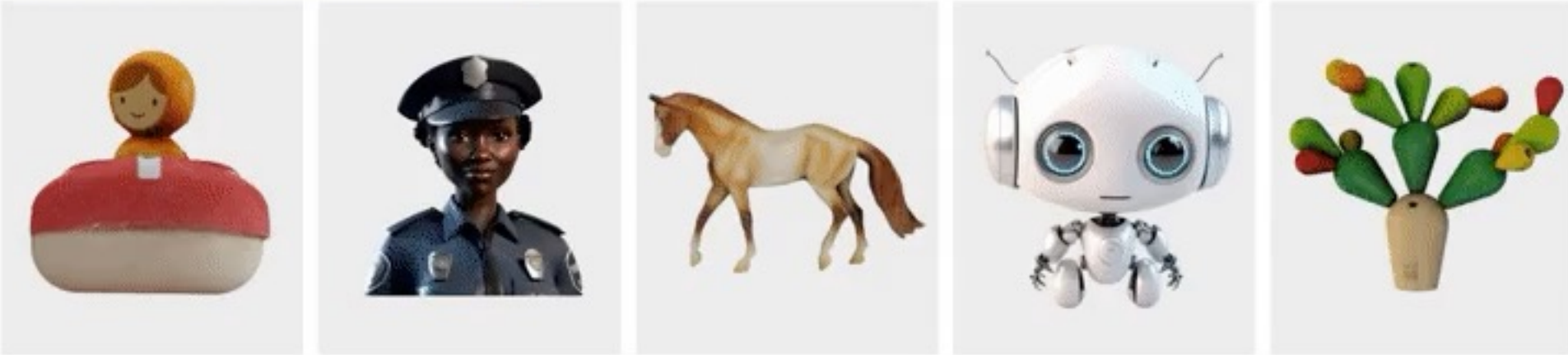
Dreamfusion



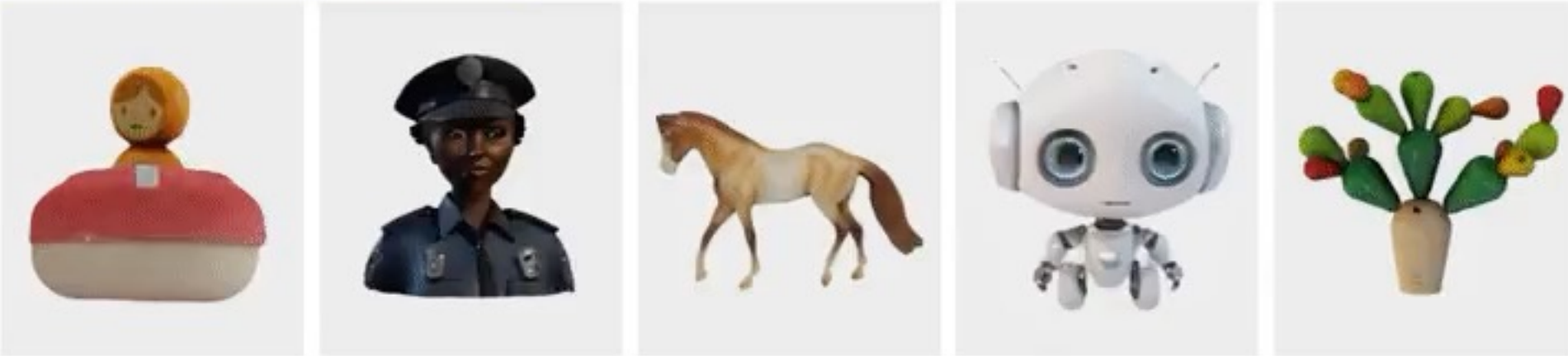
How to train the model?



Input images



TripoSR outputs



Next Steps: World Models

- https://openai.com/index/video-generation-models-as-world-simulators/
- https://gamengen.github.io
- https://research.nvidia.com/labs/dir/cosmos-predict1
- https://dreamzero0.github.io



Anonymous
Class Feedback

Q&A